Arquitectura x86-64 y Lenguaje Ensamblador

Diego Feroldi

Arquitectura del Computador [*]
Departamento de Ciencias de la Computación
FCEIA-UNR



[*]Actualizado 6 de noviembre de 2026 (D. Feroldi, feroldi@fceia.unr.edu.ar)

Índice

1. Arquitectura x86-64 **2**
- 1.1. Registros de propósito general . . . 2
- 1.2. Registros especiales . . . 4
  - 1.2.1. Puntero de instrucciones . . . 4
  - 1.2.2. Registros de segmentos . . . 4
  - 1.2.3. Registro de banderas . . . 5
- 1.3. Registros SSE . . . 6
- 1.4. Lenguaje de máquina . . . 7

2. Lenguaje Ensamblador de x86-64 **8**
- 2.1. Sintaxis AT&T . . . 11
- 2.2. Sufijos de tamaño . . . 12
- 2.3. Directivas al Ensamblador . . . 14
- 2.4. Etiquetas . . . 16
- 2.5. Etiquetas globales . . . 16
- 2.6. Operandos inmediatos . . . 17
- 2.7. Operandos en memoria . . . 18

3. Instrucciones básicas **19**
- 3.1. Instrucciones de transferencia de datos . . . 19
  - 3.1.1. Instrucción `MOV` . . . 19
  - 3.1.2. Instrucción `XCHG` . . . 21
- 3.2. Instrucciones aritméticas . . . 21
  - 3.2.1. Instrucción `ADD` . . . 21
  - 3.2.2. Instrucción `ADC` . . . 22
  - 3.2.3. Instrucción `SUB` . . . 22
  - 3.2.4. Instrucción `SBB` . . . 23
  - 3.2.5. Instrucción `INC` . . . 23
  - 3.2.6. Instrucción `DEC` . . . 24
  - 3.2.7. Instrucción `IMUL` . . . 24
  - 3.2.8. Instrucción `MUL` . . . 26
  - 3.2.9. Instrucción `IDIV` . . . 27
  - 3.2.10. Instrucción `DIV` . . . 27
  - 3.2.11. Instrucción NEG . . . 28
- 3.3. Instrucciones lógicas . . . 28
  - 3.3.1. Instrucción `AND` . . . 28
  - 3.3.2. Instrucción `OR` . . . 28
  - 3.3.3. Instrucción `XOR` . . . 29
  - 3.3.4. Instrucción `NOT` . . . 29
- 3.4. Instrucciones rotación y desplazamiento . . . 29
  - 3.4.1. Instrucción `SAL/SHL` . . . 29
  - 3.4.2. Instrucción `SAR` . . . 30
  - 3.4.3. Instrucción `SHR` . . . 30
  - 3.4.4. Instrucción `ROL` . . . 30
  - 3.4.5. Instrucción `ROR` . . . 31
  - 3.4.6. Instrucción `RCL` . . . 31

3.4.7. Instrucción RCR . . . 32
3.5. Instrucciones de comparación . . . 32
3.5.1. Instrucción CMP . . . 32
3.5.2. Instrucción TEST . . . 33
3.6. Instrucciones de acceso y manipulación del registro de banderas . . . 33
3.6.1. Modificación individual de banderas . . . 33
3.6.2. Acceso al registro de banderas . . . 34
3.7. Instrucciones para saltos incondicionales . . . 34
3.7.1. Instrucción JMP . . . 34
3.8. Instrucciones para saltos condicionales . . . 35
3.9. Instrucciones de saltos iterativos . . . 35
3.9.1. Instrucción LOOP . . . 35
3.10. Instrucciones para llamadas a procedimientos . . . 37
3.10.1. Instrucción CALL . . . 37
3.11. Instrucciones para retornos de procedimientos . . . 38
3.11.1. Instrucción RET . . . 38
3.12. Instrucciones para llamadas al sistema . . . 38
3.12.1. Instrucción SYSCALL . . . 38
3.13. Instrucciones de conversión . . . 38
3.13.1. Instrucciones CXX/CXXE . . . 38
3.13.2. Instrucciones CXTX . . . 39
3.13.3. Instrucciones MOVSXX . . . 40
3.13.4. Instrucciones MOVZXX . . . 40
3.14. Instrucciones para cálculo de direcciones . . . 41
3.14.1. Instrucción LEA . . . 41
3.15. Instrucciones de gestión de pila y marcos de activación . . . 43
3.15.1. Instrucción PUSH . . . 43
3.15.2. Instrucción POP . . . 44
3.15.3. Instrucción ENTER . . . 45
3.15.4. Instrucción LEAVE . . . 45
3.16. Instrucciones de entrada/salida por puertos . . . 46

4. Comparaciones, Saltos y Estructuras de Control 46
4.1. Saltos incondicionales . . . 46
4.2. Saltos condicionales . . . 47
4.3. Estructuras de Control . . . 48
4.4. Iteraciones . . . 50

5. Manejo de Arreglos y Cadenas 52
5.1. Copia y manipulación de datos . . . 53
5.1.1. Instrucción LODS . . . 53
5.1.2. Instrucción STOS . . . 53
5.1.3. Instucción MOVS . . . 53
5.2. Búsquedas y Comparaciones . . . 55
5.2.1. Instrucción SCAS . . . 55
5.2.2. Instrucción CMPS . . . 55
5.3. Iteraciones con instrucciones de cadena . . . 56

6. Acceso a datos en memoria **57**
6.1. Definición de variables globales 57
6.2. Endianness 58
6.3. Organización de datos en memoria 59
6.4. Acceso a memoria: valor vs dirección 61
6.5. Modos de direccionamiento de memoria 63
6.5.1. Modo de direccionamiento directo (absoluto) 63
6.5.2. Modo de direccionamiento general 64
6.6. Desreferenciación de memoria 66

7. Gestión de la pila **67**
7.1. Definición de pila 67
7.2. Uso de registros en la gestión de la pila 68
7.3. Operaciones sobre la pila 70
7.4. Alineación de la pila en x86-64 71

8. Funciones **72**
8.1. Llamada a funciones 72
8.2. Retorno de funciones 73
8.3. Convención de llamada en `x86-64` 73
8.4. Prólogo y epílogo de una función 75
8.5. Ejemplo completo de llamada a función 77
8.6. Zona roja en x86-64 77

9. Instrucciones de punto flotante **80**
9.1. Copias de datos 81
9.2. Conversiones de datos 82
9.3. Operaciones en punto flotante 83
9.4. Comparaciones en punto flotante 84
9.5. Convención de llamada en punto flotante 86
9.6. Almacenamiento de registros XMM en la pila 87

10. Instrucciones SIMD **88**
10.1. Instrucciones para operaciones con flotantes empaquetados 88
10.2. Instrucciones para operaciones con enteros empaquetados 90
10.3. Otras extensiones 91

Apéndice A: Compilando código ensamblador con GCC **92**

Apéndice B: Depurando el código con GDB **94**

Apéndice C: Otras opciones útiles para compilar **95**

Notas generales:

- Este apunte de clase resume las principales características de la arquitectura x86-64 y su lenguaje ensamblador. Durante la asignatura, utilizaremos principalmente x86-64, sin importar si el fabricante es Intel o AMD.
- Este apunte no pretende ser una referencia completa del lenguaje ensamblador ni de la arquitectura, sino que debe usarse como material complementario a lo visto en las clases teóricas. Para obtener información más detallada, consultar las referencias. En particular, se recomienda consultar [14] para una información más detallada sobre las instrucciones.
- Para compilar y depurar los ejemplos mostrados en el apunte, consulte los Apéndices 10.3 y 10.3.

1. Arquitectura x86-64

> *"x86-64 es el resultado de décadas de evolución: una arquitectura que crece sin olvidar su pasado."*

En este apunte se aborda la arquitectura x86-64 analizando sus principales características. En particular, se estudian la arquitectura del conjunto de instrucciones (ISA), el conjunto de registros, el modelo de memoria y los aspectos fundamentales del lenguaje ensamblador asociado. El objetivo es proporcionar una base conceptual y práctica que permita comprender cómo se ejecutan los programas a bajo nivel en sistemas actuales.

La arquitectura x86-64 es el resultado de una evolución progresiva de la arquitectura x86 a lo largo de varias décadas:

- En 1978, Intel introdujo el procesador Intel 8086, dando origen a la arquitectura x86 como una arquitectura de 16 bits.
- En 1985, con el lanzamiento del procesador Intel 80386, la arquitectura evolucionó a 32 bits. Esta versión fue conocida inicialmente como i386 o x86-32, y posteriormente como IA-32.
- Entre 1999 y 2003, AMD extendió la arquitectura de 32 bits a una de 64 bits, denominándola x86-64 en sus primeros documentos y, más tarde, AMD64.
- Intel adoptó estas extensiones poco tiempo después bajo los nombres IA-32e o EM64T, y finalmente Intel 64.

La arquitectura x86-64 (también conocida como **AMD64** o **Intel 64**) de 64 bits ofrece un soporte significativamente mayor para el espacio de direcciones virtuales y físicas. Proporciona registros de propósito general de 64 bits, así como buses de datos y direcciones también de 64 bits, lo que permite que las direcciones de memoria (punteros) sean valores de 64 bits. Aunque cuenta con registros de 64 bits, también permite operaciones con valores de 256, 128, 32, 16 y 8 bits.

La arquitectura x86 ha evolucionado a lo largo del tiempo incorporando distintos modos de operación, entre ellos el modo real (utilizado en los primeros procesadores), el modo protegido (introducido para soportar multitarea y protección de memoria) y el modo de 64 bits o *long mode*.

En este apunte se considerará exclusivamente el **modo de 64 bits**, que es el empleado por los sistemas operativos actuales. Este modo incluye dos submodos: el modo de 64 bits propiamente dicho, en el que se ejecutan programas diseñados para arquitecturas x86-64, y el *compatibility mode*, que permite ejecutar programas de 32 bits sin necesidad de recompilación. Esta capacidad garantiza la compatibilidad con software legado dentro de un entorno de ejecución moderno.

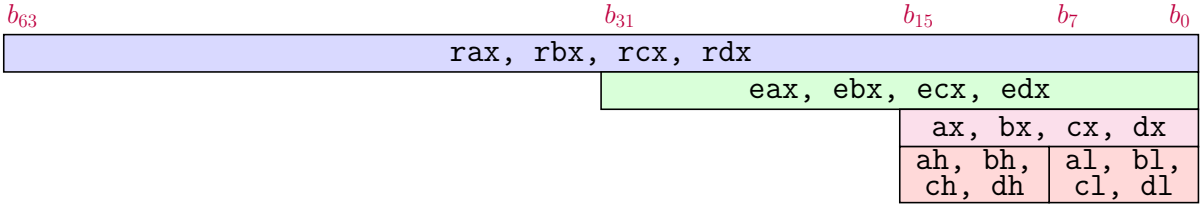
1.1. Registros de propósito general

La arquitectura x86-64 posee 16 registros de propósito general (cada uno de 64 bit): `rax, rbx, rcx, rdx, rsi, rdi, rbp, rsp` y `r8-r15`[1]. Los 8 primeros registros se denominan de manera parecida a los 8 registros de propósito general de 32 bits disponibles en

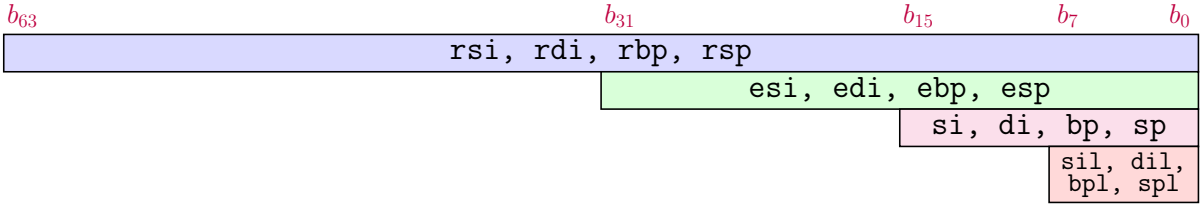
[1] Si bien el registro `rsp` está dentro del grupo de registros de propósito general, veremos en la Sección 7.1 que su uso es bastante particular.

la arquitectura IA-32 (`eax, ebx, ecx, edx, esi, edi, ebp y esp`) dado que son extensiones de los mismos. En cambio, los registros `r8` al `r15` son registros nuevos. Además, dependiendo de la versión cuenta con registros adicionales para control, punto flotante, etc. Dentro del conjunto de registros disponibles hay algunos que en realidad tienen un uso especial como el `rsp` y el `rbp` que son utilizados para manipular la pila (como veremos en la Sección 7.1).

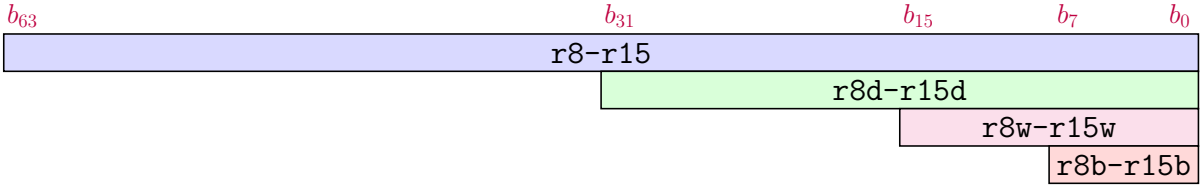
La mayoría de los registros de 64 bits se dividen en subregistros de 32, 16 y 8 bits. Por ejemplo, el registro `rax` de 64 bits contiene en sus 32 bits inferiores al subregistro `eax` (donde `e` proviene de *extended*), en sus 16 bits inferiores al subregistro `ax`, y este, a su vez, se divide en dos registros de 8 bits: `ah` (por *high*) y `al` (por *low*), como se muestra en la Figura 1a. Por razones históricas, esta última división en registros de 8 bits solo ocurre en `rax`, `rbx`, `rcx` y `rdx`; en el resto, solo existe la parte baja de 8 bits, como se observa en la Figura 1b.



(a) Registros `rax, rbx, rcx, rdx` y sus subregistros.



(b) Registros `rsi, rdi, rbp, rsp` y sus subregistros.



(c) Registros `r8-r15` y sus subregistros.

Figura 1: Registros y subregistros de propósito general en x86-64.

Los registros introducidos en la versión de 64 bits (`r8-r15`) se dividen en un subregistro de 32 bits (por ejemplo, `r8d`, de *doble word*), un subregistro de 16 bits (por ejemplo, `r8w`, de *word*) y un subregistro de 8 bits (por ejemplo, `r8b`, de *byte*), como se muestra en la Figura 1c.

Observaciones

- El registro `rsp` se utiliza para señalar el tope actual de la pila. Aunque forma parte del conjunto de registros de propósito general, no debe emplearse para almacenar datos ni para otros fines. Su uso se analizará en detalle en la Sección 7.1.

- De manera similar, el registro `rbp` se usa como registro auxiliar para facilitar el manejo de la pila.
- En GDB, los registros de propósito general de x86-64 se pueden ver usando:

  ```
  info registers
  ```

  Si se desea inspeccionar únicamente algunos registros específicos, se puede usar:

  ```
  info register rax rbx rcx
  ```

  También se puede abreviar como:

  ```
  i r rax rbx rcx
  ```

1.2. Registros especiales

Existen varios registros más que no son de uso general y, por lo tanto, no pueden ser utilizados por las instrucciones habituales.

1.2.1. Puntero de instrucciones

El puntero de instrucciones o contador de programa (en inglés *Instruction Pointer* o *Program Counter*) apunta o guarda la dirección de memoria de la próxima instrucción a ejecutar. En la arquitectura x86-64, este registro se denomina `rip`.

Cada vez que se lee la siguiente instrucción desde la memoria, este registro se actualiza con la dirección de la próxima instrucción a ejecutar. Su contenido también puede modificarse durante la ejecución de una instrucción de ruptura de secuencia, como una llamada a una subrutina o un salto, ya sea condicional o incondicional.

1.2.2. Registros de segmentos

Los registros de segmento contienen los selectores que se utilizan para acceder a los segmentos de memoria. Son seis registros de 16 bits cada uno:

ss (Stack segment): Indica cuál es el segmento utilizado para la pila.

cs (Code Segment): Indica cuál es el segmento de código. En este segmento debe alojarse el código ejecutable del programa. En general este segmento es marcado como sólo lectura.

ds (Data Segment): Indica cuál es el segmento de datos. Allí se alojan los datos del programa (como variables globales).

es, fs, gs: Estos registros tienen un uso especial en algunas instrucciones (las de cadena) y también pueden ser utilizados para referir a uno o más segmentos extras.

Observación

En modo de 64 bits se utiliza un modelo de segmentación plana de la memoria virtual. Esto significa que el espacio de memoria virtual de 64 bits se trata como un único

espacio de direcciones plano (no segmentado), lo que reduce la utilidad de los registros de segmentos. El tema de **Segmentación** se abordará por separado cuando estudiemos **Memoria Virtual**.

1.2.3. Registro de banderas

El procesador incluye un registro especial denominado `RFLAGS` o registro de banderas, que refleja el estado del procesador. Este registro contiene banderas que proporcionan información sobre el resultado de las últimas operaciones realizadas, principalmente operaciones enteras, y también ciertos bits de control que permiten modificar el comportamiento del procesador.

Aunque `RFLAGS` tiene 64 bits en la arquitectura x86-64, la mayoría de las banderas utilizadas se encuentran en los bits bajos del registro, manteniendo compatibilidad con el registro `EFLAGS` de la arquitectura de 32 bits. Los bits no utilizados están reservados.

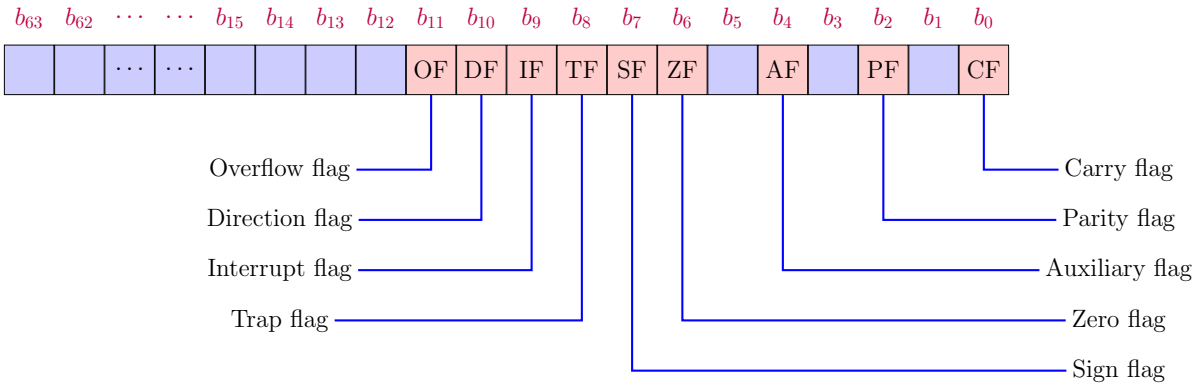


Figura 2: Esquema simplificado del registro `rflags` mostrando las banderas más relevantes.

En la Figura 2 vemos el registro `eflags` (la versión de 32 bits del `rflags`). Describimos brevemente las banderas más utilizadas:

CF *Carry Flag*: indica un acarreo en las operaciones aritméticas sin signo. En las instrucciones de suma se activa cuando se produce un acarreo fuera del bit más significativo, mientras que en las instrucciones de resta indica que fue necesario realizar un préstamo (*borrow*).

ZF *Zero Flag*: en 1 si el resultado de la última operación es cero, es decir, si todos los bits del resultado son iguales a 0.

OF *Overflow Flag*: en 1 si la última operación produjo un desbordamiento en la interpretación con signo, es decir, si el resultado no es representable utilizando la cantidad de bits del operando destino.

SF *Sign Flag*: en 1 si el bit más significativo del resultado es 1. Para números representados en complemento a dos, esto indica un resultado negativo.

DF *Direction Flag*: indica la dirección de procesamiento en las instrucciones de manejo de cadenas. Si está en 0, los registros índice se incrementan después de cada operación; si está en 1, se decrementan.

PF *Parity Flag*: en 1 si la cantidad de bits en 1 del byte menos significativo del resultado de la operación es par. Este flag se utiliza principalmente para detectar errores de transmisión y tiene poca relevancia en las aplicaciones modernas.

El resto de los bits no se utilizan o corresponden a banderas más específicas que no son relevantes para esta asignatura.

Observaciones

- El registro `rflags` no es de propósito general por lo cual no puede ser accedido ni modificado por instrucciones regulares (`add`, `mov`, etc) de manera directa. En cambio, sí puede ser modificado de manera indirecta por instrucciones tales como las de comparación, aritméticas, etc. Es decir, las banderas del registro `rflags` se modifican como resultado de ciertas instrucciones, lo cual veremos más adelante que tiene importantes utilidades.
- En GDB, es posible visualizar el contenido del registro de banderas mediante el comando `info registers eflags` (o su forma abreviada `i r eflags`), que muestra su valor en formato hexadecimal e indica además qué banderas están activadas, lo que facilita el análisis.

  ```
  i r eflags
  eflags         0x213              [ CF AF IF ]
  ```

  El valor hexadecimal corresponde al contenido completo del registro, mientras que la lista entre corchetes indica las banderas que se encuentran activas. En este ejemplo, las banderas `CF`, `AF` y `IF` están en 1, mientras que las restantes están en 0.

  Aunque en arquitectura x86-64 el registro se denomina `RFLAGS` y tiene 64 bits, GDB suele referirse a él como `eflags` por compatibilidad con la arquitectura de 32 bits.

1.3. Registros SSE

La arquitectura x86-64 proporciona 16 registros de 128 bits (`xmm0`–`xmm15`), conocidos como registros SSE (*Streaming SIMD Extensions*), donde SIMD significa *Single Instruction, Multiple Data*. Estos registros permiten realizar operaciones vectoriales, es decir, aplicar una misma instrucción sobre múltiples datos en paralelo.

Su uso resulta especialmente relevante en aplicaciones de gráficos, multimedia, cómputo científico y en general en tareas que requieren alto rendimiento. Además, los registros `xmm` se utilizan para operaciones en punto flotante.

En la arquitectura x86-64, los registros `xmm` forman parte del soporte para procesamiento vectorial introducido por SSE y extendido posteriormente por otras tecnologías como AVX.

La Figura 3 muestra la estructura de los registros SSE y los distintos tipos de datos que puede contener un registro `xmm`, tanto de tipo punto flotante como de tipo entero.

Vemos, por ejemplo, que un registro `xmm` puede almacenar un vector de cuatro `float` de 32 bits o un vector de dos `quadword` de 64 bits.

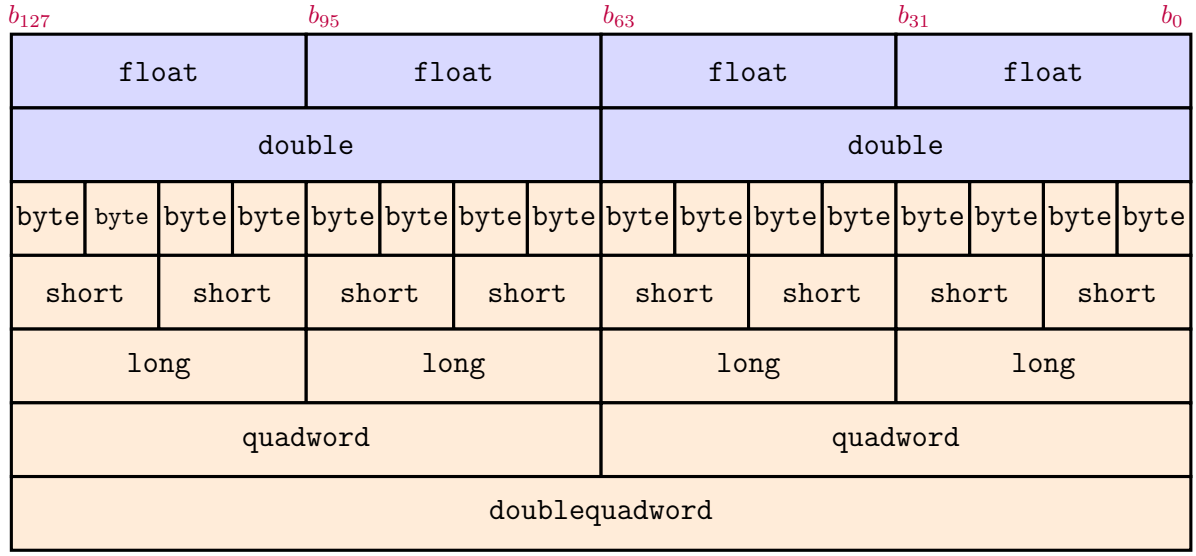


Figura 3: Registros vectoriales SSE (`xmm0`–`xmm15`): estructura y características de sus 128 bits.

Intel AVX (*Advanced Vector Extensions*) proporciona, además, 16 registros AVX de 256 bits de ancho (`ymm0-ymm15`). Los 128 bits inferiores de `ymm0-ymm15` actúan como alias de los respectivos registros SSE de 128 bits (`xmm0-xmm15`). La utilidad de los registros SSE se analizará con más detalle en la Sección 9.

1.4. Lenguaje de máquina

> *"El lenguaje de máquina es el único que la CPU realmente entiende; todo lo demás es traducción."*

Los procesadores son dispositivos de hardware encargados de ejecutar el programa alojado en memoria. Actualmente, un programador escribe el código en un lenguaje de programación de alto nivel, como C, Java, Haskell, entre otros. Sin embargo, la CPU no ejecuta directamente el programa escrito en ese lenguaje, sino que este debe ser traducido (o compilado) a **lenguaje de máquina**.

El lenguaje de máquina (o código de máquina) está compuesto por secuencias de bytes que codifican operaciones de bajo nivel destinadas a manipular datos, administrar la memoria, interactuar con dispositivos de almacenamiento y comunicarse a través de redes. Un compilador genera este código a través de una serie de etapas, guiadas por las reglas del lenguaje de programación, el conjunto de instrucciones de la máquina de destino y las convenciones impuestas por el sistema operativo.

Sin embargo, el lenguaje de máquina resulta muy críptico para las personas. Por ejemplo, la secuencia `10001001 11111000` copia el valor del registro `edi` al registro `eax`. Para facilitar la tarea de programación en los años 50, se introdujo el lenguaje ensamblador, que ofrece una representación más legible para los humanos.

2. Lenguaje Ensamblador de x86-64

"El lenguaje ensamblador es la forma más cercana en que un humano puede dialogar con la CPU."

El **lenguaje ensamblador** es una forma simbólica del lenguaje de máquina que permite escribir instrucciones mediante mnemónicos legibles por humanos, en lugar de secuencias de bits. Por ejemplo, la instrucción

```
movl %edi, %eax    # Lenguaje de máquina: 10001001 11111000 (0x89 0xF8)
                   # 0x89: MOV r/m32, r32; 0xF8: %edi -> %eax
```

es equivalente a la instrucción binaria `10001001 11111000`, que copia el valor del registro `edi` al registro `eax`. Claramente, esta representación es mucho más legible para una persona.

Ejemplo

El siguiente fragmento de código realiza la suma de dos enteros:

```
0x0000000000001125 <+0>:  89 f8   movl %edi, %eax
0x0000000000001127 <+2>:  01 f0   addl %esi, %eax
0x0000000000001129 <+4>:  c3      ret
```

En la columna izquierda se muestran las direcciones de memoria; en la columna central, el código en lenguaje de máquina (en formato hexadecimal, para mayor compacidad); y en la columna derecha, su equivalente en lenguaje ensamblador x86-64 (sintaxis AT&T).

Si bien todavía no sabemos cómo utilizar estas instrucciones, ya podemos observar que el lenguaje ensamblador resulta mucho más comprensible que el lenguaje de máquina. De hecho, presenta una sintaxis basada en una operación seguida de sus argumentos, donde las operaciones —llamadas instrucciones— tienen nombres representativos (por ejemplo, `movl` para "mover" y `addl` para "sumar"). Más adelante analizaremos el significado de cada instrucción de ensamblador y cómo se utiliza.

Los primeros ensambladores surgieron a principios de la década de 1950, cuando los programadores comenzaron a reemplazar los códigos numéricos de las instrucciones de máquina por mnemónicos simbólicos. Uno de los primeros ejemplos fue el ensamblador desarrollado por Kathleen Booth alrededor de 1950, mientras que sistemas como SAP de IBM (1954) contribuyeron a popularizar esta forma de programación.

Es importante distinguir entre el lenguaje ensamblador (*assembly language*) y el ensamblador (*assembler*).

El **lenguaje ensamblador** (*assembly language*) es un lenguaje simbólico de bajo nivel que representa instrucciones de máquina mediante mnemónicos (por ejemplo, `mov`, `add`, `lea`) y nombres simbólicos.

El **ensamblador** (*assembler*) es el programa que traduce ese código ensamblador a código máquina (*machine code*) ejecutable por el procesador. Ejemplos de ensambladores son `as` (GNU Assembler) o `nasm`.

En síntesis, el *assembly language* es la notación que usa el programador, mientras que el *assembler* es la herramienta que la convierte en instrucciones binarias.

El lenguaje ensamblador es específico de cada máquina. Por ejemplo, el código escrito para un procesador x86-64 no se ejecutará en un procesador diferente, como un procesador ARM, que es popular en tabletas y teléfonos inteligentes. El lenguaje ensamblador es un lenguaje de "bajo nivel" y proporciona la interfaz de instrucción básica para el procesador de la computadora. Para un programador, el ensamblador es lo más cercano al procesador. Los programas escritos en un lenguaje de alto nivel se traducen al lenguaje ensamblador para que el procesador pueda ejecutarlos. En este sentido, el lenguaje de alto nivel actúa como una abstracción entre el código y las instrucciones reales del procesador.

El ensamblador ofrece al programador control directo sobre los recursos del sistema, lo que implica configurar registros del procesador, acceder a ubicaciones de memoria e interactuar con otros componentes de hardware. Esto requiere una comprensión profunda del funcionamiento del procesador y la memoria.

Al programar en un lenguaje de alto nivel como C, no es necesario preocuparse por la implementación detallada del programa a nivel de máquina. En cambio, al escribir programas en código ensamblador, como se hacía en los primeros días de la informática, el programador debe especificar las instrucciones de bajo nivel que el programa utilizará para realizar un cálculo. La mayoría de las veces, resulta mucho más productivo y confiable trabajar en un nivel más alto de abstracción proporcionado por un lenguaje de alto nivel. La verificación de tipos realizada por un compilador ayuda a detectar muchos errores de programación y asegura que los datos se manejen de manera consistente.

La Figura 4 ilustra el proceso de traducción desde un código fuente —en este caso, escrito en C— hasta su forma binaria en lenguaje de máquina. Aunque este proceso se presenta en dos pasos, algunos compiladores eliminan el paso intermedio y generan directamente el código binario ejecutable.

Con los compiladores y optimizadores modernos, el código generado suele ser tan eficiente como el que escribiría manualmente un programador experto en ensamblador. Además, un programa escrito en un lenguaje de alto nivel puede compilarse y ejecutarse en múltiples plataformas, mientras que el código ensamblador es altamente específico de la máquina para la que fue desarrollado.

Entonces, ¿por qué deberíamos aprender lenguaje ensamblador? Este lenguaje ofrece varios beneficios:

- **Velocidad**: Los programas en lenguaje ensamblador son generalmente los programas más rápidos.
- **Espacio**: Los programas en lenguaje ensamblador suelen ser los más pequeños.
- **Capacidad**: Se pueden hacer cosas en lenguaje ensamblador que son difíciles o imposibles en lenguajes de alto nivel.
- **Conocimiento**: El conocimiento del lenguaje ensamblador ayuda a escribir mejores programas, incluso cuando se use lenguajes de alto nivel.

Aunque los compiladores hacen la mayor parte del trabajo en la generación de código ensamblador, poder leerlo y comprenderlo es una habilidad importante para los programadores avanzados. Al invocar al compilador con los parámetros de línea de comandos apropiados, el compilador generará un archivo que muestra su salida en forma de código de ensamblaje. Al leer este código, podemos entender las capacidades de optimización del compilador y analizar las ineficiencias subyacentes en el código.

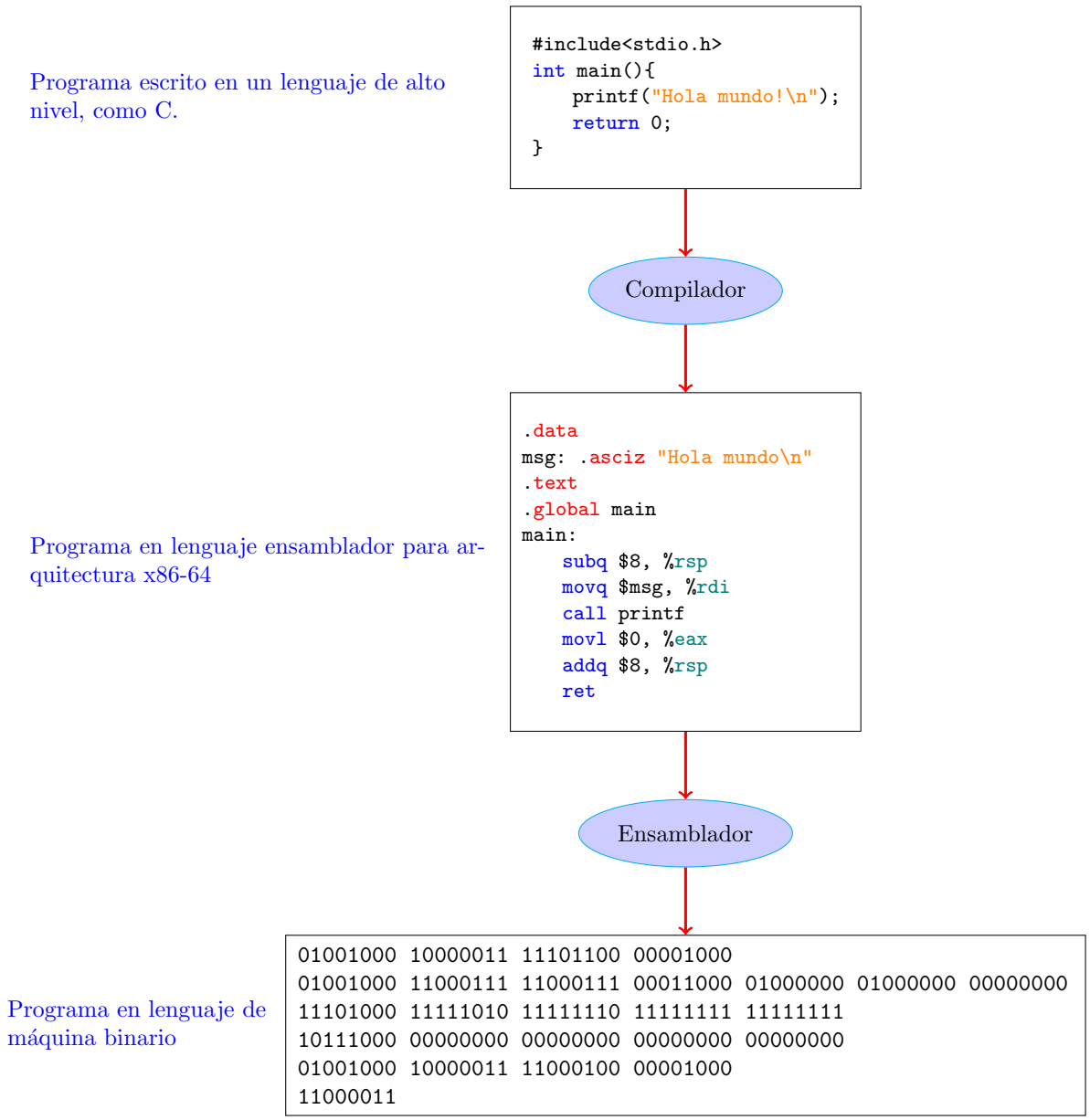


Figura 4: Programa en C compilado a lenguaje ensamblador y luego ensamblado a lenguaje de máquina binario.

Los programadores que buscan maximizar el rendimiento de una sección crítica de código a menudo prueban diferentes variaciones del código fuente, cada vez que compilan y examinan el código de ensamblaje generado para tener una idea de qué tan eficientemente se ejecutará el programa. Además, hay momentos en que la capa de abstracción proporcionada por un lenguaje de alto nivel oculta información sobre el comportamiento en tiempo de ejecución de un programa que debemos comprender. Por ejemplo, cuando se escriben programas concurrentes utilizando un paquete de subprocesos, es importante saber qué región de la memoria se usa para contener las diferentes variables del programa. Esta información es visible a nivel de código ensamblador.

Otro ejemplo es que muchas de las formas en que se pueden atacar los programas, permitiendo que gusanos y virus infesten un sistema, implican matices de la forma en que los programas almacenan la información en tiempo de ejecución. Muchos ataques

implican la explotación de las debilidades en los programas del sistema para sobrescribir la información y, por lo tanto, tomar el control del sistema. Comprender cómo surgen estas vulnerabilidades y cómo protegerse de ellas requiere un conocimiento de la representación de los programas a nivel de máquina.

La necesidad de aprender lenguaje ensamblador varía según el contexto. En la computación de propósito general, su estudio se orienta principalmente a la lectura y comprensión del código generado por los compiladores. En cambio, en sistemas embebidos, el ensamblador mantiene un papel más activo: no solo se utiliza con fines de análisis, sino también para implementar componentes críticos como rutinas de inicialización, manejo de interrupciones o secciones donde se requiere un control preciso del hardware y de los tiempos de ejecución.

2.1. Sintaxis AT&T

En esta sección se presentan las principales características de la sintaxis del lenguaje ensamblador x86-64 utilizando la notación AT&T, a modo introductorio. A lo largo del apunte se profundizará progresivamente en estos conceptos.

Las instrucciones en lenguaje ensamblador están formadas por un *mnemónico* que identifica la operación a realizar (por ejemplo, suma, resta o comparación), seguido de uno o más operandos que especifican los datos involucrados en la operación.

Para las instrucciones de dos operandos en la sintaxis AT&T, el formato general es:

```
mnemónico{S} <operando origen>, <operando destino>
```

- El mnemónico identifica la operación que debe realizar la instrucción (por ejemplo, `MOV`, `ADD`, etc.).
- Las instrucciones incluyen un sufijo opcional (`S`) que indica el tamaño de los operandos o del dato manipulado: `b` (byte), `w` (palabra de 16 bits), `l` (32 bits) y `q` (64 bits). Por ejemplo, la instrucción `mov`, utilizada para transferir datos, se escribe como `movq` cuando se emplea el sufijo `q` (quadword), que indica que la operación se realiza sobre operandos de 64 bits.
- Los operandos en una instrucción pueden ser:
  - Registros
  - Direcciones de memoria
  - Valores inmediatos
- En general, las instrucciones tienen dos operandos, donde el operando destino se ubica a la derecha del operando origen.
- Existen instrucciones con un solo operando, sin operandos o incluso con tres operandos, aunque se trata de excepciones.
- En algunos casos, las instrucciones utilizan operandos implícitos.
- El nombre de los registros comienza con el prefijo `%`. Por ejemplo, el registro `rax` se escribe como `%rax`.

- Los comentarios de línea comienzan con `#` (a partir de `#` comienza un comentario hasta el fin de línea).
- Las constantes (valores inmediatos) se prefijan con `$`. Así, la constante 5 se escribe como `$5`. Un caso particular que veremos luego son las etiquetas.
- Las direcciones de memoria se escriben sin ninguna decoración. Por lo tanto, la expresión `3000` refiere a la dirección de memoria 3000, y no a la constante 3000, que —como se explicó antes— se escribiría como `$3000`.

Observación

En este apunte utilizaremos la sintaxis de AT&T de lenguaje ensamblador ya que es la utilizada por defecto en GNU Assembler (GAS). Si bien originalmente GCC solo trabajaba con la sintaxis AT&T, actualmente se puede trabajar también con la sintaxis Intel utilizando la bandera `-masm=intel` al momento de compilar.

Las principales diferencias entre ambas sintaxis son las siguientes:

	Intel	AT&T
Orden de los operandos	destino ← origen	origen → destino
Comentarios	;	#
Operadores	Sin sufijo: `add`	Con sufijo: `addq`
Registros	`eax, ebx`, etc.	`%eax, %ebx`, etc.
Valores inmediatos	`0x100`	`$0x100`
Direccionamiento indirecto	`[eax]`	`(%eax)`
Direc. (forma general)	`[base+(índice*scale)+K]`	`K(base, índice, scale)`

Como ejemplo comparativo la instrucción `addq %ebx, %eax` en sintaxis AT&T es equivalente a la instrucción `add eax, ebx` en sintaxis Intel.

Ejemplos

Aquí se presentan instrucciones con sintaxis AT&T con diferente cantidad de operandos. Su funcionamiento se analiza en detalle más adelante.

```
movq $5, %rax     # Instrucción con dos operandos (un valor inmediato y
   un registro) y con sufijo q
incl %eax         # Instrucción con un solo operando y con sufijo l
ret               # Instrucción sin operandos explícitos
mulq %rbx         # Instrucción con un operando explítico, que usa como
   operando implícito el registro rax
imulq $15, %rax, %rbx    # Instrucción con tres operandos
```

2.2. Sufijos de tamaño

En lenguaje ensamblador x86-64 con sintaxis AT&T es fundamental indicar explícitamente el tamaño de los operandos mediante sufijos, ya que omitirlos puede dar lugar a ambigüedades o a comportamientos no deseados.

Los sufijos posibles son los siguientes:

Sufijo	Tamaño (bytes)	Tipo de dato	Directiva (.data)	GDB
`b`	1	Entero	`.byte`	`b`
`w`	2	Entero	`.word` / `.short`	`h`
`l`	4	Entero	`.long`	`w`
`q`	8	Entero	`.quad`	`g`
`s`	4	Flotante	`.float`	`w`
`d`	8	Flotante	`.double`	`g`

Observación

En sintaxis **AT&T**, el sufijo de una instrucción (por ejemplo, `b`, `w`, `l`, `q`) indica explícitamente el tamaño del dato sobre el que opera: 1, 2, 4 u 8 bytes, respectivamente. Este tamaño debe ser consistente con los operandos involucrados, en particular con el registro destino o fuente.

Cuando se omite el sufijo, el ensamblador intenta inferir el tamaño a partir de los operandos. Sin embargo, esta inferencia puede ser ambigua o inducir errores sutiles, especialmente cuando se combinan memoria y subregistros.

Ejemplos

Algunos ejemplos correctos:

```
movq a, %rax      # copia 8 bytes desde memoria en 'a' hacia el registro
   rax
movw a, %ax       # copia 2 bytes
movb a, %al       # copia 1 byte
movq %rax, %rbx   # ambos operandos son registros de 8 bytes
movb %al, %bl     # ambos operandos son registros de 1 byte
```

Contra-ejemplos (ambigüedad o error):

```
mov a, %rax       # El ensamblador infiere movq, pero no es explícito
mov a, %ax        # El ensamblador infiere movw
mov %eax, %rdx      # Error: los operandos son registros de diferente
   tamaño
```

Aunque en algunos casos pueden ensamblar, dependen de inferencias implícitas que conviene evitar.

Conclusión: especificar siempre el sufijo evita ambigüedades y, sobre todo, errores semánticos asociados a escrituras parciales de registros. En código de bajo nivel, estas diferencias no son meramente sintácticas, sino que afectan directamente el estado del procesador.

Observación

Los sufijos `b`, `w`, `l` y `q` se utilizan para datos enteros, mientras que `s` y `d` corresponden a números en punto flotante. Aunque algunos tamaños coinciden (por ejemplo, `l` y `s` son ambos de 4 bytes), no son intercambiables, ya que representan tipos de datos distintos.

En la terminología de x86, los tamaños de datos suelen denominarse *byte* (1 byte), *word* (2 bytes), *double word* (4 bytes) y *quad word* (8 bytes).

Observación

La columna **GDB** indica el modificador de formato utilizado por el depurador `gdb` en comandos como `x` (examine). Por ejemplo, `x/g` muestra datos de 8 bytes ("giant word"), mientras que `x/w` muestra datos de 4 bytes ("word"). Estos modificadores no forman parte de la sintaxis del ensamblador, sino que corresponden a convenciones propias de `gdb`.

2.3. Directivas al Ensamblador

Las instrucciones y los datos no son los únicos elementos que componen un programa en lenguaje ensamblador. Existen también palabras clave especiales, denominadas *directivas*, que indican al ensamblador cómo debe procesar el código fuente (por ejemplo, la definición de datos o la organización del programa).

A diferencia de las instrucciones, las directivas no se traducen directamente a código máquina. En general, comienzan con el prefijo ".".

Dentro de las directivas, se destacan las siguientes:

Segmentos Las directivas de segmento indican al ensamblador en qué sección del programa debe ubicarse el código o los datos que siguen. Las más utilizadas son `.text`, que contiene el código ejecutable, `.data`, que almacena datos inicializados, y `.bss`, destinada a datos no inicializados.

Ejemplos

```
.data
(Desde este punto, un tramo con datos inicializados.)
.bss
(Desde este punto, un tramo con datos sin inicializar.)
.text
(Desde este punto, un tramo con instrucciones.)
```

Inicialización de datos Estas directivas permiten definir valores constantes que se almacenan directamente en memoria durante el ensamblado. A diferencia de las instrucciones, no generan código ejecutable, sino que reservan e inicializan datos.

Dentro de esta categoría se destacan:

asciz, ascii Permiten inicializar una lista de cadenas con y sin carácter nulo al final de cada una.

Ejemplos

```
.asciz "Hola mundo"   # Se almacena la cadena Hola mundo\000 (11
   caracteres, con el carácter nulo al final).
.ascii "Hola mundo"   # Se almacena la misma cadena de caracteres
   sin el cero final.
```

```
.string "Hola mundo"  # Equivalente al primer ejemplo
.string "Hola mundo", "abcd", "1234" # La directiva string puede
   procesar múltiples cadenas si se las separa por comas.
```

byte Inicializa una lista de bytes.

Ejemplos

```
.byte 'a', 'b'
.byte 97
.byte 0x61
```

El primer ejemplo es un arreglo de bytes.

double, float Inicializa una lista de valores de punto flotante de doble y simple precisión, respectivamente.

Ejemplos

```
 .double 3.1415, 2.16
 .float 5.3
```

El primer ejemplo es un arreglo de *doubles*.

short, long, quad Emite una lista de valores enteros de 2, 4 y 8 bytes, respectivamente.

Ejemplos

```
.short 20, 30, 40, 50
.long 50, 200
.quad 0, 0xff, 0xbeef
```

space Emite un bloque de tamaño fijo inicializado en cero o en un valor pasado como argumento.

Ejemplos

```
.space 128
.space 5000, 0
.zero 5000
```

En el primer ejemplo se reservó un bloque de memoria de 128 bytes pero no está inicializado (puede tener cualquier valor). En el segundo ejemplo se reservó un bloque de 5000 bytes y está inicializado explícitamente en 0. El tercer ejemplo es equivalente al segundo. Estas directivas son útiles para obtener un bloque de memoria de tamaño dado (ya sea inicializado o no).

Observación

Es importante notar que todas estas directivas toman como argumento una **lista** de valores a inicializar.

Un error muy común es no indicar ningún elemento en esa lista, por ejemplo:

```
.data
.long
```

lo cual **NO** reserva espacio. La versión correcta sería `.long 0` o, alternativamente,

```
.data
.space 8
```

2.4. Etiquetas

Las etiquetas son una parte fundamental del lenguaje ensamblador, ya que hacen referencia a elementos dentro del programa. Su función principal es facilitar al programador la tarea de referenciar diferentes partes del programa, como constantes, variables o posiciones del código, que se utilizan como operandos en las instrucciones o directivas.

Por ejemplo, cuando se define una variable en C (`long i;`), se le indica al compilador que reserve un espacio de memoria para un entero y que este espacio se referenciará mediante el identificador `i`. Tanto en C como en ensamblador, nombrar un espacio de memoria es útil para el programador, pero esta información no es utilizada directamente por la computadora; en su lugar, una etiqueta se convierte en una **dirección de memoria**.

En ensamblador con sintaxis AT&T una etiqueta es un nombre seguido del símbolo "`:`".

Ejemplos

```
a: .quad 126
b: .long 455
c: .byte 0x22
f: .float 3.14
g: .double 0.1
msg: .ascii "Hola mundo\n"
```

En este ejemplo se crean variables de distintos tipos. Por ejemplo, una variable de tipo `quad` (8 bytes) se inicializa con el valor 126 y se almacena en una dirección de memoria etiquetada como `a`.

2.5. Etiquetas globales

La directiva `.global` indica que la etiqueta nombrada a continuación es de alcance global.

Ejemplos

```
.global main
.global sum
```

De no especificar esta directiva la etiqueta desaparece luego del proceso de compilación. Las etiquetas globales deben ser utilizadas, por ejemplo, con las etiquetas que definan funciones que serán llamadas fuera del archivo compilado. Por ejemplo, cuando se enlaza un programa C con uno escrito en ensamblador, las funciones incluidas en ensamblador deben ser definidas como globales (siendo `main` el caso más común). Esto se verá en detalle en el Apéndice 10.3.

2.6. Operandos inmediatos

En ciertas instrucciones, un operando de origen, llamado operando inmediato, se incluye como parte de la instrucción en lugar de acceder a él desde un registro o una ubicación de memoria. En la sintaxis que veremos en este apunte (AT&T) cada operando inmediato debe ir precedido por un signo peso para indicar que es un valor inmediato. Los valores también se pueden expresar en varios formatos diferentes, siendo el formato decimal y el formato hexadecimal los más usuales. Estos valores no se pueden cambiar después de que el programa es ensamblado y *linkeado* en el archivo de programa ejecutable. En el modo de 64 bits, el tamaño máximo de un operando inmediato es de 32 bits, excepto en la instrucción `mov`, que puede copiar un inmediato de 64 bits en un registro de propósito general.

Ejemplos

```
movl $0, %eax          # copia el valor 0 al registro eax
movb $0x80, %bl        # copia el valor hexadecimal 80 al registro bl
addb $0xff, %ah        # suma 0xff con el valor en ah y lo guarda en ah
movl $0x11223344, %eax       # copia el valor 0x11223344 al registro eax
movq $0x1122334455667788, %rax    # copia el valor 0x1122334455667788 al
   registro rax
movq $0xbeef, a        # Copia 8 bytes (por el sufijo q) en memoria, en la
    dirección etiquetada como a.
```

Observación

Es interesante ver el equivalente en lenguaje de máquina de la instrucción `movq $0x1122334455667788, %rax` del ejemplo anterior. Esto se puede lograr utilizando GDB, con el comando `disassemble/r`. Obtenemos el siguiente equivalente en lenguaje de máquina (en formato hexadecimal):

```
48 b8 88 77 66 55 44 33 22 11
```

Aquí se ve de manera explícita que el valor inmediato está contenido dentro de la propia instrucción. El motivo por el cual se ve invertido lo veremos en detalle en la Sección 6.2.

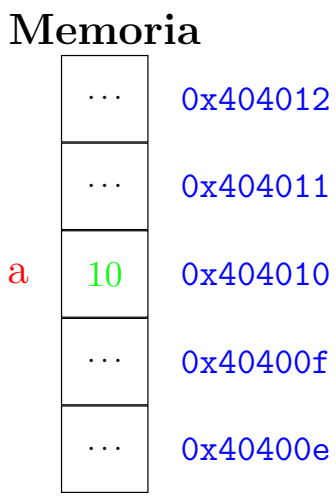


Figura 5: Representación esquemática de la memoria donde se muestra la asociación entre una etiqueta (`a`), su dirección (`0x404010`) y el valor almacenado (`10`).

2.7. Operandos en memoria

Además de registros y valores inmediatos, las instrucciones pueden operar sobre datos almacenados en memoria. La forma más simple de acceso es mediante una etiqueta que representa una dirección.

Por ejemplo:

```
movb $10, a
```

En este caso, `a` es una etiqueta asociada a una dirección de memoria, y la instrucción copia el valor inmediato `10` en esa dirección (1 byte, por el sufijo `b`).

La Figura 5 ilustra una vista simplificada de la memoria principal, donde cada celda se identifica mediante una dirección única y almacena un byte. En particular, se destaca la etiqueta `a`, que en el contexto de assembler constituye un símbolo sin tipo asociado que referencia la dirección `0x404010`; en dicha dirección se almacena el valor `10`.

Observación

Notar que la instrucción `movb $10, a` es equivalente a `movb $10, 0x404010`. En realidad, el ensamblador (y posteriormente el enlazador) resuelven la etiqueta sustituyéndola por la dirección de memoria correspondiente. En la práctica, resulta mucho más conveniente trabajar con etiquetas que con direcciones absolutas.

De manera análoga, también es posible leer desde memoria:

```
movb a, %al
```

Esta instrucción se interpreta como leer 1 byte desde la dirección asociada a la etiqueta `a` y cargar ese valor en el registro `%al`.

En la Sección 6.5 veremos formas de acceso a la memoria más complejas.

3. Instrucciones básicas

En esta sección se presentan las instrucciones más comunes para operar con valores enteros, organizadas según su función, con el objetivo de facilitar su comprensión. Luego, en la Sección 9 se verán la instrucciones para operar con datos en punto flotante. Debe tenerse en cuenta que el conjunto de instrucciones de la arquitectura x86-64 es muy amplio.

El **conjunto de instrucciones** (*Instruction Set Architecture*, ISA) de la arquitectura x86-64 dispone de varios cientos de instrucciones en su especificación básica. Al considerar las extensiones incorporadas a lo largo de su evolución, como SSE, AVX y AVX-512, el ISA x86-64 supera el millar de instrucciones. Esta gran cantidad de instrucciones es consecuencia de la compatibilidad hacia atrás y de la incorporación progresiva de nuevas capacidades al procesador.

Por lo tanto, este listado constituye un subconjunto reducido, aunque suficientemente representativo para los objetivos de un curso básico. Para un tratamiento más detallado y exhaustivo, se recomienda consultar la documentación técnica [14].

3.1. Instrucciones de transferencia de datos

Una operación muy común es la de copiar valores de un lugar a otro. Un programa debe intercambiar valores con la memoria, registros, etc. La arquitectura x86-64 ofrece varias instrucciones para hacer copias de datos siendo la más importante la instrucción `mov`.

3.1.1. Instrucción `MOV`

La instrucción `mov` es la instrucción genérica para copiar un dato desde un origen a un destino. Esta instrucción toma la forma

```
mov{b,w,l,q} <operando origen>, <operando destino>
```

donde `b,w,l,q` es un sufijo opcional que indica el tamaño de los operandos (los cuales deben coincidir), según lo descrito en la Tabla de la Sección 2.2.

Observación

El operando destino es el argumento de la derecha, por lo que la instrucción

```
movq %rax, %rbx  # rbx <-- rax
```

Es decir, copia el valor de `rax` a `rbx`. Después de ejecutar la instrucción, el valor de `rbx` será igual al de `rax`. Es importante destacar que el valor del registro `rax` permanece sin cambios. En realidad, más que un movimiento, es una copia de datos.

El operando origen puede ser un valor inmediato, un registro de propósito general o un valor en memoria. El operando destino puede ser un registro de propósito general o un valor en memoria. Los dos operandos no pueden ser valores de memoria. Por lo tanto, a continuación podemos observar las diferentes formas que puede tomar la instrucción:

```
mov{b,w,l,q} <registro>, <registro>
mov{b,w,l,q} <memoria>, <registro>
mov{b,w,l,q} <registro>, <memoria>
mov{b,w,l,q} <valor immediato>, <memoria>
mov{b,w,l,q} <valor immediato>, <registro>
```

Ejemplos

```
movb $65, %al              # al=65='A'
movq %rax, %rcx            # rcx=rax
movw (%rax), %dx           # Copia en dx dos bytes comenzando en la
                           # dirección guardada en rax.
movw %dx, (%rax)           # Copia dx en la dirección guardada en rax.
movl 16(%rbp), %ecx        # Copia en ecx cuatro bytes (debido al
                           # sufijo l) comenzando en la dirección rbp+16.
movb $45, a                # Copia el valor 45 en la dirección de memoria
                           # con etiqueta a
```

Nota: Algunos de estos ejemplos se comprenderán mejor luego de ver la Sección 6.

Observación

Una limitación de la arquitectura x86-64 es que no permite utilizar dos referencias a memoria en una misma instrucción. Por ejemplo:

`movq a, b` ✗

Por lo tanto, las operaciones que involucran dos valores almacenados en memoria requieren instrucciones adicionales para cargar alguno de ellos en un registro antes de realizar la operación.

`movq a, %rax` ✓

`movq %rax, b` ✓

Observación

La relación entre subregistros de diferentes tamaños es la siguiente:

1. La carga de un valor en un subregistro de 32 bits establece los 32 bits superiores del registro en cero. Por ejemplo, después de `movl $-1, %eax`, el registro `%rax` tiene el valor `0x00000000ffffffff`, independientemente del valor anterior.

2. La carga de un valor en un subregistro de 16 u 8 bits deja todos los demás bits sin cambios. Por ejemplo, si el valor de `%rax` es `0xffffffffffffffff`, luego de `movw $0, %ax` el registro `%rax` tiene el valor `0xffffffffffff0000`.

Esto puede parecer un poco arbitrario pero es así por una cuestión de compatibilidades a medida que fueron apareciendo procesadores con registros con mayor cantidad de bits.

3.1.2. Instrucción XCHG

Esta instrucción intercambia el contenido de los operandos:

```
xchg{b,w,l,q} <operando fuente>, <operando destino>
```

Ejemplo

```
movq $34, %rax
movq $0xf3fa, %rbx
xchgq %rax, %rbx     # rax=0xf3fa y rbx=34
```

3.2. Instrucciones aritméticas

La familia de procesadores x86 ofrece múltiples instrucciones para realizar operaciones numéricas, entre ellas:

3.2.1. Instrucción ADD

Esta instrucción realiza la suma aritmética de los dos operandos:

```
add{b,w,l,q} <operando fuente>, <operando destino>
```

El resultado queda en el operando destino:

```
<operando destino=operando fuente + operando destino>
```

Ejemplo

```
movb $5, %al
movb $4, %bl
addb %al, %bl     # al=5 y bl=9
```

La instrucción `add` realiza la suma entera. Notar que evalúa el resultado tanto para la operación sin signo como con signo y establece las banderas `CF` y `OF` para indicar si el resultado es correcto. La bandera `SF` indica el signo del resultado signado y la bandera `ZF` si el resultado es nulo.

Ejemplo

```
movb $45, %al
addb $100, %al
```

Luego de realizarse la suma resulta `al=0x91=-111`, `SF=1`, `CF=0` y `OF=1`. El estado de las banderas indica que el resultado de la suma con los operando interpretados como números con signo es negativo y es incorrecto. En cambio, si los operandos se toman como números sin signo, el resultado es correcto (`0x91=145`).

3.2.2. Instrucción ADC

Esta instrucción realiza la suma aritmética de los dos operandos más el bit de acarreo (CF del rflags):

```
adc{b,w,l,q} <operando fuente>, <operando destino>
```

Resulta:

```
<operando destino=operando fuente + operando destino + acarreo>
```

Ejemplo

```
movb $0, %dl
movb $0xff, %al
addb $0xff, %al   # al=0xfe, CF=1
adcb $0, %dl      # dl=1
```

3.2.3. Instrucción SUB

Esta instrucción realiza la resta aritmética de los dos operandos:

```
sub{b,w,l,q} <operando fuente>, <operando destino>
```

La operación realizada es:

```
operando destino = operando destino - operando fuente
```

Ejemplo

```
movq $45, %rbx
movq $23, %rax
subq %rax, %rbx    # rbx = rbx - rax =22
```

La instrucción sub realiza una resta entera y actualiza las banderas del registro de estado en función del resultado obtenido. La bandera CF indica un préstamo (*borrow*) en la operación, lo que permite detectar un desborde en la interpretación sin signo. Por otro lado, la bandera OF indica un desborde en la interpretación con signo. La bandera SF toma el valor del bit más significativo del resultado, indicando su signo en la representación en complemento a dos, mientras que la bandera ZF se activa cuando el resultado es cero.

Observación

Aunque la instrucción SUB realiza una resta, las unidades aritmético-lógicas (ALU) no suelen implementar un circuito restador independiente. En su lugar, reutilizan el circuito sumador mediante la representación en complemento a dos:

$$A - B = A + C_2(B) = A + C_1(B) + 1$$

Para ello, el segundo operando se invierte bit a bit y se utiliza un acarreo inicial igual a 1.

Ejemplo

La operación:

```
movb $23, %al
subb $45, %al
```

se implementa internamente como:

$$\begin{array}{rll} & 00010111_2 & (23) \\ + & 11010011_2 & (-45) \\ \hline & 11101010_2 & (-22) \end{array}$$

El resultado almacenado en el registro es entonces `11101010`, que representa el valor -22 en complemento a dos.

Es importante notar que, aunque la resta se implementa como una suma, la bandera `CF` no representa el acarreo de esta suma interna. En una instrucción `SUB`, la arquitectura x86 define `CF` como un indicador de préstamo (*borrow*) en una operación sin signo. En este ejemplo, como $23 < 45$, se requiere un préstamo y por lo tanto `CF = 1`.

3.2.4. Instrucción SBB

Resta aritmética de los dos operandos considerando el bit de acarreo:

```
sbb{b,w,l,q} <operando fuente>, <operando destino>
```

Resulta:

```
operando destino = operando destino - operando fuente - acarreo
```

Ejemplo

```
movl $1, %edx
movl $0, %eax
subl $1, %eax   # CF=1
sbbl $0, %edx   # edx=0
```

3.2.5. Instrucción INC

Incrementa el operando en una unidad (`operando=operando+1`):

```
inc{b,w,l,q} <operando>
```

Ejemplo

```
movq $56, %rax
incq %rax         # rax=57
```

Esta instrucción es equivalente a `addq $1, %rax`. Sin embargo, la instrucción `inc` no modifica el valor de la bandera `CF`. El resto de las banderas son modificadas de acuerdo al resultado.

3.2.6. Instrucción DEC

Decrementa el operando en una unidad:

```
dec{b,w,l,q} <operando>
```

Ejemplo

```
movq $45, %rax
decq %rax          # rax=:44
```

Esta instrucción es equivalente a `subq $1, %rax`. Sin embargo, la instrucción `dec` no modifica el valor de la bandera `CF`. El resto de las banderas son modificadas de acuerdo al resultado.

3.2.7. Instrucción IMUL

La instrucción `IMUL` realiza la multiplicación entera con signo. La instrucción `imul` tiene tres formatos posibles:

Con un operando:

```
imul{b,w,l,q} <operando>
```

El operando puede ser un registro o una dirección de memoria, pero no un valor inmediato.

El formato con un operando utiliza los registros `rax` y `rdx` (o una parte de los mismos) de forma implícita. Es decir, si el operando es de 64 bits multiplica el valor del operando con `rax` y el resultado queda en `rdx:rax`. Notar que el resultado es de 128 bits y los 64 bits menos significativos quedan en `rax` mientras que los 64 más significativos en `rdx`:

```
rdx:rax ← rax · operando
```

De manera análoga, se puede trabajar con operandos de 32 y 16 bits. Es decir, si se multiplica el valor de un operando de 32 con `eax`, el resultado queda en `edx:eax`. Si se multiplica el valor de un operando de 16 con `ax`, el resultado queda en `dx:ax`. Sin embargo, si se multiplica el valor de un operando de 8 bits con `al`, el resultado queda en `ah:al`.

Ejemplos

```
imulb %cl      # AX = AL * CL
imulw %bx      # DX:AX = AX * BX
imull %ecx     # EDX:EAX = EAX * ECX
imulq %rcx     # RDX:RAX = RAX * RCX
```

Con dos operandos:

```
imul{w,l,q} <fuente>, <destino>
```

En este formato el operando destino es multiplicado por el operando fuente. El operando destino debe ser un registro de propósito general, mientras que el operando fuente puede ser un registro de propósito general, un valor inmediato o un valor en memoria. En este formato no existen registros implícitos, y el resultado intermedio (de un tamaño doble al del operando fuente) se trunca y se guarda en el operando destino.

$$\texttt{destino} \leftarrow \texttt{destino} \cdot \texttt{fuente}$$

Ejemplos

```
imulw %bx, %ax        # AX = AX*BX
imull %ebx, %eax      # EAX = EAX*EBX
imulq %rbx, %rax      # RAX = RAX*RBX
```

Con tres operandos:

```
imul{w,l,q} <fuente1>, <fuente2>, <destino>
```

Este formato requiere dos operandos fuentes y un operando destino. El segundo operando fuente (que puede ser un registro de propósito general o un valor en memoria) es multiplicado por el primer operando fuente (un valor inmediato). El resultado intermedio (el doble de tamaño que el operando fuente) es truncado y guardado en el operando destino (un registro):

$$\texttt{destino} \leftarrow \texttt{fuente2} \cdot \texttt{fuente1}$$

Ejemplos

```
imulw $10, %bx, %ax      # AX = 10*AX*BX
imull $10, %ebx, %eax    # EAX = 10*EAX*EBX
imulq $10, %rbx, %rax    # RAX = 10*RAX*RBX
```

Las tres formas de la instrucción `IMUL` difieren en cómo se trata el resultado del producto. En la forma de un operando, el producto completo —de longitud doble respecto de los operandos— se almacena explícitamente en el destino implícito.

En cambio, en las formas de dos y tres operandos, el resultado se reduce al tamaño del operando destino, descartando los bits más significativos que no entran en dicho tamaño. Debido a este truncamiento, es necesario verificar las banderas CF y OF para determinar si el resultado exacto puede representarse sin pérdida en el destino.

Ejemplos

```
movq $9, %rax
movq $-3, %rbx
imulq %rbx                      # rax=0xfffffffffffffe5 (-27)
                                # rdx=0xffffffffffffffff (-1)
movq $9, %rax
imulq %rbx, %rax                # rax=0xfffffffffffffe5 (-27)
imulq $4, %rax                  # rax=0xffffffffffffff94 (-108)
imulq $2, %rax, %rbx            # rbx=0xffffffffffffff28 (-216)
movq $0x7ffffffffffffe, %rax    # rax=9223372036854775806
imulq $2, %rax                  # rax=0xfffffffffffffffc (-4)
```

Notar que en la última multiplicación el resultado es erróneo dado que el verdadero resultado (1.8447×10^{19}) no entra en 64 bits (CF=OF=1).

Observación

En todas las variantes de `imul`, las banderas CF y OF indican desbordamiento aritmético con signo.

- En la forma de un operando, las banderas CF y OF se utilizan para indicar si el resultado podría representarse en la mitad inferior como un entero con signo.
- En las formas de dos y tres operandos, se establecen (CF = OF = 1) si el resultado no puede representarse en el tamaño del operando destino con signo.
- En todos los casos, CF y OF toman siempre el mismo valor, mientras que las demás banderas quedan indefinidas.

3.2.8. Instrucción `MUL`

La instrucción `MUL` realiza la multiplicación entera sin signo. Esta instrucción a diferencia de la `IMUL` solo admite el formato con un operando:

```
mul{b,w,l,q} <operando>
```

El operando puede ser un registro o una posición de memoria.

Ejemplos

```
mulb %bl        # AX = AL*BL
mulw %bx        # DX:AX = AX*BX
mull %ebx       # EDX:EAX = EAX*EBX
mulq %rbx       # RDX:RAX = RAX*RBX
```

De manera similar a la instrucción `IMUL` con un solo operando, el resultado se distribuye entre dos registros (`rdx` y `rax`) o sus respectivos subregistros, según el tamaño de los operandos. Si los bits de orden superior del producto son 0, las banderas CF y OF se borran; de lo contrario, las banderas se establecen (CF = OF = 1).

Ejemplo

```
movq $0xffffffffffffffff, %rax
movq $4, %rbx
mulq %rbx      # rax=0xfffffffffffffffc; rdx=3; CF=OF=1
```

3.2.9. Instrucción IDIV

La instrucción `idiv` realiza la división entera con signo. Tiene un comportamiento similar a `mul`, ya que utiliza operandos implícitos y solamente permite especificar un operando explícito. No existen formas de dos o tres operandos.

```
idiv{b,w,l,q} <operando divisor>
```

donde el operando divisor puede ser un registro o una posición de memoria, pero no un inmediato.

La instrucción `idiv` en su versión de 64 bits divide el contenido del entero de 128 bits `rdx:rax` (construido interpretando a `rdx` como los ocho bytes más significativos y a `rax` como los ocho bytes menos significativos) por el valor del operando especificado. El resultado del cociente de la división se almacena en `rax`, mientras que el resto se coloca en `rdx`[2].

Ejemplos

```
idivb %bl      # Cociente: AL = AX / BL, resto: AH = AX % BL
idivw %bx      # Cociente: AX = DX:AX / BX, resto: DX = DX:AX % BX
idivl %ebx     # Cociente: EAX = EDX:EAX / EBX, resto: EDX = EDX:EAX %
   EBX
idivq %rbx     # Cociente: RAX = RDX:RAX / RBX, resto: RDX = RDX:RAX %
   RBX
```

Ejemplo

```
movq $-65535, %rax   # rax = -65535
cqto                 # rdx:rax = extensión de signo de rax
movq $1024, %rbx     # divisor
idivq %rbx           # rax = -63 y rdx = -1023
```

El cociente entero es -63 y el resto es -1023.

3.2.10. Instrucción DIV

La instrucción `div` es similar a `idiv`, pero realiza la división entera sin signo. Al igual que `idiv`, utiliza registros implícitos para el dividendo, el cociente y el resto, y solamente permite especificar el operando divisor. No existen formas de dos o tres operandos.

[2]Esta instrucción también admite operandos de otros tamaños. Para mayor información ver [14]

```
div{b,w,l,q} <operando divisor>
```

donde el operando divisor puede ser un registro o un operando en memoria, pero no un valor inmediato.

Ejemplos

```
divb %bl      # Cociente: AL = AX / BL, resto: AH = AX % BL
divw %bx      # Cociente: AX = DX:AX / BX, resto: DX = DX:AX % BX
divl %ebx     # Cociente: EAX = EDX:EAX / EBX, resto: EDX = EDX:EAX % EBX
divq %rbx     # Cociente: RAX = RDX:RAX / RBX, resto: RDX = RDX:RAX % RBX
```

Ejemplo

```
movq $0xffff, %rax    # rax=65535
movq $0, %rdx
movq $1024, %rbx
divq %rbx      # rax=0x3f (63) y rdx=0x3ff (1023)
```

El resultado entero es 63 y el resto es 1023.

3.2.11. Instrucción NEG

La instrucción `neg` realiza la negación aritmética en complemento a 2:

```
neg{b,w,l,q} <operando>
```

Ejemplo

```
movb $0xff, %al
negb %al       # al=1
```

3.3. Instrucciones lógicas

3.3.1. Instrucción AND

Operación `and` lógica bit a bit.

Ejemplo

```
movw $0xdeaa, %ax
movw $0xf0f0, %bx
andw %bx, %ax            # ax=0xd0a0
```

3.3.2. Instrucción OR

Operación `or` lógica bit a bit.

Ejemplo

```
movw $0xdeaa, %ax
movw $0xf0f0, %bx
orw %bx, %ax            # ax=0xfefa
```

3.3.3. Instrucción XOR

Operación `xor` lógica bit a bit.

Ejemplo

```
xorl %eax, %eax          # eax=0
movl $0xffffffff, %ebx   # ebx=0xffffffff
xorl %eax, %ebx          # ebx=0xffffffff
```

3.3.4. Instrucción NOT

Negación lógica bit a bit.

Ejemplo

```
movb $0xff, %al
notb %al            # al=0
```

3.4. Instrucciones rotación y desplazamiento

Las instrucciones de rotación y desplazamiento realizan una rotación cíclica o un desplazamiento no cíclico, por un número dado de bits, sobre un operando dado:

```
operación{b,w,l,q} <primer operando>, <segundo operando>
```

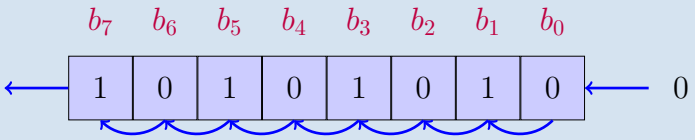
donde el primer operando es la cantidad de veces que se rota o desplaza el segundo operando.

3.4.1. Instrucción SAL/SHL

Las instrucciones `sal` (*Shift Arithmetic Left*) y `shl` (*Shift Logical Left*) son idénticas en cuanto a funcionamiento: ambas realizan un desplazamiento lógico hacia la izquierda del contenido de un operando, rellenando con ceros por la derecha y sin rotar.

Ejemplo

```
movb $0xaa, %al
salb $1, %al        # al=0x54
```

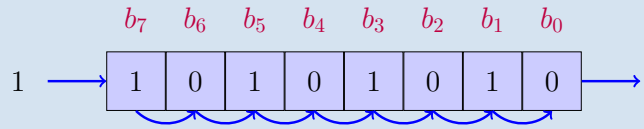


3.4.2. Instrucción SAR

La instrucción `sar` (*Shift Arithmetic Right*) realiza un desplazamiento aritmético hacia la derecha, conservando el signo del operando. Es decir, si el bit más significativo es uno, se desplaza ingresando unos; en caso contrario, se ingresan ceros.

Ejemplo

```
movb $-0xaa, %al    # al=0xaa (-86)
sarb $1, %al        # al=0xd5 (-43)
```

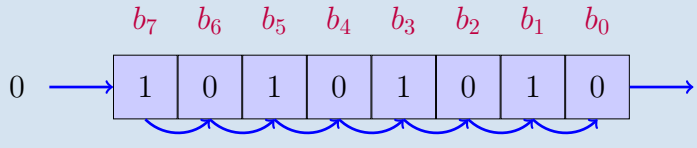


3.4.3. Instrucción SHR

La instrucción `shr` (*Shift Logical Right*) realiza un desplazamiento lógico hacia la derecha del contenido de un operando. A diferencia de la instrucción `sar`, no conserva el bit de signo: siempre inserta ceros por la izquierda.

Ejemplo

```
movb $0xaa, %al       # al=0xfc (-86)
shrb $1, %al          # al=0x55 (85)
```

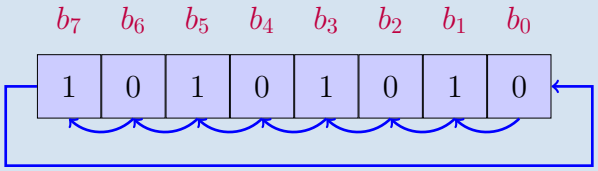


3.4.4. Instrucción ROL

La instrucción `rol` (*Rotate Left*) realiza una rotación lógica hacia la izquierda del contenido de un operando. Es decir, los bits del operando se desplazan hacia la izquierda, y los bits que "salen" por el extremo izquierdo vuelven a entrar por el extremo derecho.

Ejemplo

```
movb $0xaa, %al
rolb $1, %al        # al=0x55
```

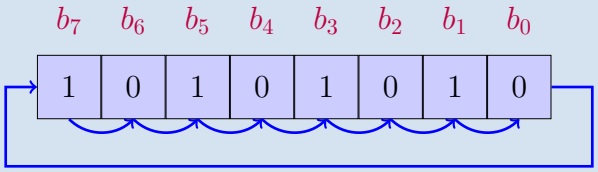


3.4.5. Instrucción `ROR`

La instrucción `ror` (*Rotate Right*) es similar a la instrucción anterior pero con rotación hacia la derecha.

Ejemplo

```
movb $0xaa, %al
rorb $1, %al        # al=0x55
```

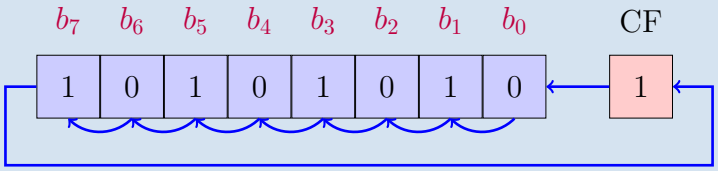


3.4.6. Instrucción `RCL`

La instrucción `rcl` rota los bits del operando hacia la izquierda, incorporando el bit de acarreo (CF) como un bit adicional en el extremo derecho. De este modo, el número total de bits rotados es igual al tamaño del operando más uno. El bit que sale por el extremo izquierdo se almacena en CF, mientras que el valor previo de CF ingresa por el extremo derecho.

Ejemplo

```
movb $0xaa, %al
stc                 # CF=1 (se enciende la bandera de acarreo)
rclb $2, %al        # al=2*al+1=0x55
```



3.4.7. Instrucción RCR

La instrucción `rcr` realiza una rotación lógica a la derecha considerando el bit de acarreo. Similar a la instrucción anterior pero con rotación hacia la izquierda.

Ejemplo

```
movb $0xaa, %al
stc                     # CF=1
rcrb $1, %al            # al=0xd5
```

3.5. Instrucciones de comparación

Una comparación entre dos valores es la forma más común de evaluarlos para decidir posteriormente la ejecución de un salto condicional. En la arquitectura x86, esta evaluación se realiza mediante instrucciones que operan sobre los operandos y actualizan las banderas del registro `RFLAGS` en función del resultado, sin almacenar dicho resultado.

3.5.1. Instrucción CMP

La instrucción `cmp` realiza la comparación de los dos operandos:

```
cmp{b,w,l,q} <operando fuente>, <operando destino>
```

Hace la resta `destino=destino-fuente` sin guardar el resultado, solo modifica las banderas correspondientes. Aunque no se escriba el resultado, el destino tiene que ser un registro y no puede ser una constante.

Ejemplo

```
movq $45, %rbx
movq $66, %rax
cmpq %rax, %rbx  # compara RBX con RAX (RBX - RAX) y actualiza las
   banderas: CF=1, SF=1, ZF=0
cmpq %rbx, %rax  # compara RAX con RBX (RAX - RBX) y actualiza las
   banderas: CF=0, SF=0, ZF=0
```

En la primera instrucción `cmp` el operando destino es menor que el operando fuente. Por lo tanto `CF=1`, `ZF=0` y `SF=1`. En la segunda instrucción `cmp` el operando destino es mayor que el operando fuente. Por lo tanto `CF=0`, `ZF=0` y `SF=0`. En ninguna de las instrucciones el operador destino fue modificado.

3.5.2. Instrucción TEST

La instrucción `test` realiza la comparación lógica de los dos operandos:

```
test{b,w,l,q} <operando fuente>, <operando destino>
```

Realiza la operación lógica `and` bit a bit sin guardar el resultado. Solo modifica las banderas:

- ZF se activa si el resultado es 0.
- SF: refleja el bit más significativo del resultado (indica si sería negativo).
- PF: indica si el número de bits en 1 del byte menos significativo es par.
- CF y OF: se limpian (quedan en 0).

Ejemplo

```
    movq $5, %rax
    test %rax, %rax    # AND bit a bit consigo mismo; actualiza flags
    jz es_cero         # salta si ZF = 1 (la última operación dio
       resultado cero)
    movq $1, %rbx      # Se ejecuta si rax no es cero
    jmp fin
es_cero:
    movq $0, %rbx      # Se ejecuta si rax es cero
fin:
```

3.6. Instrucciones de acceso y manipulación del registro de banderas

Existen instrucciones especiales para trabajar con el registro `rflags`. Entre ellas distinguimos varias clases.

3.6.1. Modificación individual de banderas

- `clc` (*clear carry flag*), limpia la bandera de acarreo (`CF`), estableciéndola en cero.
- `cmc` (*complement carry flag*), invierte el valor de la bandera de acarreo (`CF`).
- `cld` (*clear direction flag*), limpia la bandera de dirección (`DF`), estableciéndola en cero.
- `stc` (*set carry flag*), establece la bandera de acarreo (`CF`) en uno.
- `std` (*set direction flag*), establece la bandera de acarreo (`DF`) en uno.

3.6.2. Acceso al registro de banderas

- `lahf` (Load AH from Flags) y `sahf` (Store AH into Flags) copian ciertos bits del registro `ah` hacia el `rflags` y viceversa.
- `pushfq` almacena el contenido del registro RFLAGS en la pila.
- `popfq` restaura el contenido de RFLAGS desde la pila.

Ejemplos

```
movb $200, %al     # al = 200
addb $100, %al     # al = 44, CF = 1
clc                # Limpia el Carry Flag (CF = 0)
stc                # Fuerza CF = 1 manualmente
adc $0, %bl        # Suma con acarreo: bl = bl + 0 + CF
```

El uso del registro `rflags` se verá más claro en breve cuando expliquemos cómo se usa el registro para hacer saltos condicionales en la Sección 4.

3.7. Instrucciones para saltos incondicionales

3.7.1. Instrucción JMP

La instrucción `jmp` realiza un salto incondicional transfiriendo el control a la dirección indicada por su operando. Este destino puede especificarse mediante una etiqueta (salto directo) o a través de un registro (salto indirecto).

```
jmp etiqueta
jmp *%registro
```

Observación

Dado que el operando de las instrucciones de salto es siempre una dirección de memoria, estas instrucciones no llevan sufijo de tamaño.

Ejemplo

```
    jmp etiqueta
    movq %rax, %rbx   # Esta instrucción nunca se ejecuta
etiqueta:
    movq $45, %rcx    # La ejecución pasa a esta esta instrucción
```

Ejemplo

```
    movq $cont, %rax
    jmp *%rax
    movq $1, %rax
cont:
    movq $2, %rax
```

En este ejemplo `jmp *%rax` realiza un salto a la dirección contenida en `rax`, que es la dirección de `cont`. Observar el uso del `*` antes del nombre del registro, lo cual es requerido por la sintaxis.

3.8. Instrucciones para saltos condicionales

Las instrucciones para realizar saltos condicionales tienen la forma:

```
jCC etiqueta
```

donde `CC` es un sufijo que indica la condición que debe cumplirse para que se realice el salto. Si dicha condición es verdadera, se transfiere el control a la etiqueta indicada; en caso contrario, la ejecución continúa con la siguiente instrucción.

Estas instrucciones evalúan el estado de las banderas del registro de estado, por lo que previamente debe ejecutarse alguna instrucción que las modifique (por ejemplo, una instrucción de comparación o una instrucción aritmética).

En la Tabla 1 mostramos un listado completo de instrucciones `jCC` y los valores requeridos en las banderas, donde `CC` es el sufijo que depende de la condición que se debe verificar. En la Sección 4.2 se aborda en detalle la aplicación de las instrucciones para saltos condicionales.

Ejemplo

```
    movq $15, %rax      # RAX = 15
    cmp $10, %rax       # compara RAX con 10 (RAX - 10)  # actualiza
       banderas (ZF, SF, CF, OF, etc.), no guarda el resultado
    ja  mayor           # salto si "above" (sin signo); se realiza si CF
       = 0 y ZF = 0 (RAX > 10)
    decq %rax           # si NO salta: RAX = RAX - 1
    jmp fin             # salto incondicional para evitar ejecutar el
       bloque 'mayor'
mayor:
    incq %rax           # si salta: RAX = RAX + 1
fin:
```

3.9. Instrucciones de saltos iterativos

3.9.1. Instrucción `LOOP`

La instrucción

```
loop <etiqueta>
```

Tabla 1: Instrucciones `jCC` y sus correspondientes `rFLAGS`.

Mnemónico	**Estado de banderas requerido**	**Descripción**
`JO`	OF = 1	*Jump near if overflow*
`JNO`	OF = 0	*Jump near if not overflow*
`JB`	CF = 1	*Jump near if below*
`JC`		*Jump near if carry*
`JNAE`		*Jump near if not above or equal*
`JNB`	CF = 0	*Jump near if not below*
`JNC`		*Jump near if not carry*
`JAE`		*Jump near if above or equal*
`JZ`	ZF = 1	*Jump near if zero*
`JE`		*Jump near if equal*
`JNZ`	ZF = 0	*Jump near if not zero*
`JNE`		*Jump near if not equal*
`JNA`	CF = 1 or ZF = 1	*Jump near if not above*
`JBE`		*Jump near if below or equal*
`JNBE`	CF = 0 and ZF = 0	*Jump near if not below or equal*
`JA`		*Jump near if above*
`JS`	SF = 1	*Jump near if sign*
`JNS`	SF = 0	*Jump near if not sign*
`JP`	PF = 1	*Jump near if parity*
`JPE`		*Jump near if parity even*
`JNP`	PF = 0	*Jump near if not parity*
`JPO`		*Jump near if parity odd*
`JL`	SF $\neq$ OF	*Jump near if less*
`JNGE`		*Jump near if not greater or equal*
`JGE`	SF = OF	*Jump near if greater or equal*
`JNL`		*Jump near if not less*
`JNG`	ZF = 1 or SF $\neq$ OF	*Jump near if not greater*
`JLE`		*Jump near if less or equal*
`JNLE`	ZF = 0 and SF = OF	*Jump near if not less or equal*
`JG`		*Jump near if greater*

tiene dos efectos:

- Decrementa en uno el registro `rcx`. Aquí vemos que `rcx` tiene un uso especial.
- Luego, salta a la dirección de memoria indicada en la etiqueta **sólo si** el resultado de decrementar `rcx` dio distinto de cero. Si el resultado dio cero, el flujo del programa sigue en la siguiente instrucción a la instrucción `loop`.

Ejemplo

```
    movq $10, %rcx
    xorq %rax, %rax
etiqueta:
    incq %rax
    loop etiqueta
    .....
```

La instrucción `incq %rax` se ejecuta 10 veces. Por lo tanto, luego de ejecutarse el código anterior `rax=10`.

Las aplicaciones de la instrucción `loop` las veremos en la Sección 4.4.

3.10. Instrucciones para llamadas a procedimientos

3.10.1. Instrucción CALL

La instrucción `call` se utiliza para hace una llamada a subrutina:

```
call <etiqueta>
```

Su funcionamiento se basa en dos acciones principales:

- Guarda la dirección de retorno (la instrucción siguiente a `call`) en la pila.
- Transfiere el control a la dirección de la función llamada (es decir, realiza un salto la dirección de memoria indicada con la etiqueta).

Ejemplo

```
    ...
    call sumar     # Llama a la función sumar (guarda la dirección de
       retorno en la pila)
    ...

sumar:
    ...            # Cuerpo de la función sumar
    ret            # Retorna a la función llamante (usa la dirección de
       retorno en la pila)
```

También se puede hacer una llamada indirecta:

```
call <*operando>
```

En lugar de saltar a una dirección codificada explícitamente, transfiere el control a la dirección contenida en un registro. Por ejemplo:

```
call *%rax      # Llama a la dirección almacenada en %rax (llamada
   indirecta); guarda la dirección de retorno en la pila.
```

3.11. Instrucciones para retornos de procedimientos

3.11.1. Instrucción RET

La instrucción `ret` se utiliza para hacer un retorno de subrutina. No tiene operandos. Esta instrucción y la instrucción `CALL` se ven en detalle en la Sección 8.

3.12. Instrucciones para llamadas al sistema

3.12.1. Instrucción SYSCALL

La instrucción `syscall` en x86-64 se utiliza para realizar una llamada al sistema operativo desde un programa en modo usuario. No lleva operandos: es una instrucción implícita que provoca la transición de modo usuario a modo kernel.

Ejemplo

```
movq $60, %rax    # Número de syscall (exit)
movq $0, %rdi     # Código de salida (status = 0)
syscall           # Invoca la llamada al sistema (transfiere el control
   al kernel)
```

Esta instrucción se verá en detalle al Apunte **Introducción a Flujo de Control Excepcional**.

3.13. Instrucciones de conversión

Las instrucciones de conversión de datos realizan diferentes transformaciones de datos. En particular, la arquitectura x86-64 ofrece numerosas instrucciones para convertir entre enteros de distintos tamaño.

3.13.1. Instrucciones CXX/CXXE

Existe un conjunto de instrucciones que doblan el tamaño del registro correspondiente, extendiendo con el signo el valor almacenado, que tienen la siguiente forma:

```
cXX
cXXe
```

donde `XX` son dos sufijos de tamaño de acuerdo al tamaño del origen y del destino. Estas instrucciones no tienen operandos explícitos y operan de manera implícita con el registro `rax` o sus subregistros.

Aquí vemos algunas de las instrucciones disponibles:

Instrucción	Descripción
`cbw`	Extiende (con signo) `al` a `ax`.
`cwde`	Extiende (con signo) `ax` a `eax`.
`cwd`	Extiende (con signo) `ax` a `dx:ax`.
`cdq`	Extiende (con signo) `eax` a `edx:eax`.
`cdqe`	Extiende (con signo) `eax` a `rax`.
`cqo`	Extiende (con signo) `rax` a `rdx:rax`.

Observación

Notar que las instrucciones anteriores trabajan con operados implícitos. Notar también que hay instrucciones muy parecidas que difieren en que terminan con el sufijo `e`. Es decir, son instrucciones con diferentes nombres que hacen la misma conversión pero en el caso de las instrucciones que termina con `e` el resultado queda todo en un subregistro y no repartido en dos subregistros. Para un listado completo de las instrucciones de conversión consultar [14].

Ejemplo

```
movw $-34, %ax
cwd
```

Luego de ejecutarse el código anterior, `ax=0xffde` y `dx=0xffff`. En cambio, si hacemos `cwde` el resultado de la conversión queda en el subregistro `eax=0xffffffde`.

3.13.2. Instrucciones `CXTX`

Existe otro grupo de instrucciones de conversión de datos con la forma

`CXTX`

que se utilizan para realizar conversiones en las que se amplía el tamaño del dato (extensión). De manera análoga al conjunto de instrucciones visto en la sección anterior, se emplean dos sufijos que indican el tamaño del operando de origen y del de destino. Sin embargo, en este caso los sufijos están separados por una `t`, que corresponde a la palabra en inglés "*to*".

Aquí vemos algunas de las instrucciones disponibles:

Instrucción	Descripción
cwtl	Extiende (con signo) `ax` en `eax`.
cltq	Extiende (con signo) `eax` en `rax`.
cqto	Extiende (con signo) `rax` en `rdx:rax`.

Ejemplo

```
movl $-5, %eax    # Carga -5 en eax (32 bits)
cltq              # Extiende el signo de eax a rax (64 bits)
```

Observación

Los sufijos de tamaño utilizados en las instrucciones de esta sección y de la anterior corresponden a los presentados en la tabla de la Sección 2.2. Sin embargo, notar que aquí el sufijo `d` hace referencia a "*doble word*", es decir 32 bits, y no a doble precisión. De hecho, todas estas instrucciones utilizan datos de tipo entero. Veremos en la Sección 9.2 las instrucciones de conversión para datos de tipo flotante.

3.13.3. Instrucciones MOVSXX

Las instrucciones `MOVSXX` copian un valor del origen al destino extendiendo de acuerdo al signo. Estas instrucciones se utilizan para extender datos con signo y tiene dos sufijos, el primero es el tamaño del dato origen y el segundo es el tamaño del dato destino. Estas instrucciones son similares a las `CXX/CXXE` pero tienen mucha más versatilidad dado que permiten mayor cantidad de conversiones (no solamente doblando el tamaño) y no trabajan con registros implícitos.

Ejemplos

```
movsbl %bl, %ebx   # convierte un byte a 4 bytes
movswl %cx, %ecx   # convierte un word a 4 bytes
movswq %ax, %rax   # convierte un word a 8 bytes
```

Por lo tanto si tenemos el siguiente código:

```
movb $-45, %al        # al=0xd3
movsbq %al, %rax      # rax=0xffffffffffffffd3
```

Luego de ejecutarse, `rax=0xffffffffffffffd3`, lo cual corresponde al valor -45 interpretado como un entero con signo en complemento a dos.

3.13.4. Instrucciones MOVZXX

Las instrucciones `MOVZXX` copian un valor del origen al destino extendiendo con *cero*. Estas instrucciones se utilizan para extender datos sin signo y tiene dos sufijos, el primero es el tamaño del dato origen y el segundo es el tamaño del dato destino.

Ejemplos

```
movzbl %al, %eax   # convierte un byte a 4 bytes
movzwl %ax, %eax   # convierte un word a 4 bytes
movzwq %ax, %rax   # convierte un word a 8 bytes
```

Dado el comportamiento predeterminado al trabajar con registros de 32 bits, no hay necesidad de una instrucción `movzlq` explícita. En efecto, si queremos extender sin signo el registro `eax` a `rax`, basta con hacer `movl %eax, %eax`.

3.14. Instrucciones para cálculo de direcciones

3.14.1. Instrucción LEA

La arquitectura x86-64 ofrece una instrucción similar al operador de referencia de C. Esta instrucción se denomina `lea` (por "*load effective address*")[3] y calcula la dirección efectiva del operando de origen y la almacena en el operando de destino.

```
lea{w,l,q} <operando fuente>, <operando destino>
```

El operando fuente es una dirección de memoria especificada con uno de los modos de direccionamiento de los procesadores mientras que el operando de destino es un registro de propósito general. Usando sintaxis AT&T, los modos de direccionamiento útiles con `lea` son los siguientes:

Modo	Dirección
`K(%base)`	`K + base`
`(,%índice, M)`	`índice * M`
`(%base, %índice, M)`	`base + índice*M`
`K(, %índice, M)`	`K + índice*M`
`K(%base, %índice, M)`	`K + base + índice*M`
`etiqueta`	Dirección de `etiqueta`

donde `K` es una constante entera que actúa como desplazamiento, `M` es una constante con un valor de 1, 2, 4 u 8, que funciona como multiplicador, y `%dest`, `%índice` y `%base` son registros.

Observación

La instrucción `lea` (*Load Effective Address*) no accede al contenido de memoria sino que calcula la dirección efectiva y la almacena en un registro. Su comportamiento es equivalente al operador `&` en C (obtención de direcciones) o a expresiones aritméticas sobre punteros. Por ejemplo:

- `lea (%rax), %rbx` ⟷ `rbx = rax;`
- `lea 8(%rax), %rbx` ⟷ `rbx = rax + 8;`
- `lea (%rax,%rcx,4), %rbx` ⟷ `rbx = rax + rcx * 4;`

En particular, si `rax` contiene la dirección de una variable, entonces `lea` permite construir nuevas direcciones a partir de ella sin acceder al contenido de memoria. Esto contrasta con `mov`, que sí realiza una desreferenciación cuando se utilizan operandos de memoria, como veremos en la Sección 6.6.

[3] *Effective Address* (EA): es la dirección de memoria resultante de evaluar el modo de direccionamiento; es la que se utiliza para acceder a memoria. La aplicación de la instrucción `lea` y el concepto de dirección efectiva se verá en detalle en la Sección 6.5.

Ejemplo

```
.data
num: .long 10, 20, 30, 40, 50    # arreglo de enteros de 4 bytes

.text
.global main

main:
    movl $2, %eax
    leaq num, %rbx               # Carga la dirección de 'num' en RBX
    leaq (%rbx,%rax,4), %rcx     # Calcula la dirección del tercer
       elemento
    movl (%rcx), %eax            # Retorna el valor del tercer elemento
    ret
```

Observación

```
leaq str, %rax          # En rax queda la dirección de la etiqueta str
movq $str, %rax         # Esta instrucción es equivalente a la anterior
movq (%rax), %rbx       # Se dereferencia la dirección str
```

Las dos primeras instrucciones del código anterior son equivalentes y no implican acceso a memoria. Aunque el primer operando en ambas instrucciones parece ser una referencia de memoria, en realidad la instrucción no lee desde la dirección indicada, sino que copia la dirección efectiva al destino. Esta operación es equivalente al operador `&` utilizado en el lenguaje C. En cambio, la tercera instrucción sí accede a memoria, utilizando la dirección contenida en `rax`.

La instrucción `lea` a menudo se usa como un "truco" para hacer ciertos cálculos, aunque ese no sea su propósito principal.

Ejemplos

La instrucción `lea` se puede usar para multiplicar un registro por constantes tales como 2, 3, 4, 5, 8 o 9:

```
lea k(, %src, 2), %dst          # dst <-- k + src*2
lea k(%src, %src, 2), %dst      # dst <-- k + src*3
lea k(, %src, 4), %dst          # dst <-- k + src*4
lea k(%src, %src, 4), %dst      # dst <-- k + src*5
lea k(, %src, 8), %dst          # dst <-- k + src*8
lea k(%src, %src, 8), %dst      # dst <-- k + src*9
```

Una ventaja de utilizar `lea` para realizar operaciones aritméticas es que, a diferencia de instrucciones como `add`, `sub` o `imul`, no modifica los indicadores del registro `RFLAGS`, independientemente del resultado obtenido.

Observación

En x86-64 (sintaxis AT&T), la instrucción `lea` admite los sufijos de tamaño `w`, `l` y `q`, que determinan el tamaño del registro destino (16, 32 y 64 bits, respectivamente). No obstante, en modo de 64 bits el uso más coherente es `leaq`, ya que las direcciones efectivas son de 64 bits.

En la práctica, los ensambladores suelen permitir omitir el sufijo y lo infieren a partir del registro destino (por ejemplo, `%rax` implica 64 bits). Por este motivo, es común encontrar simplemente `lea` en el código generado, aunque semánticamente corresponda a `leaq`.

3.15. Instrucciones de gestión de pila y marcos de activación

3.15.1. Instrucción PUSH

La instrucción `push` tiene la forma:

```
push{w,q} <operando fuente>
```

y produce dos efectos:

1. decrementa el registro `rsp` en una cantidad de bytes de acuerdo al sufijo `S`,
2. mueve el operando de la instrucción a la dirección apuntada por el registro `rsp` luego de que dicho registro es decrementado.

La instrucción `push` puede usarse con distintos tipos de operandos, siempre que el tamaño sea válido (16 o 64 bits en modo de 64 bits). Los operandos posibles son:

- Registros generales (de 16 o 64 bits).
- Operando inmediato.
- Operando en memoria (empuja el valor almacenado en esa dirección de memoria).

En todos los casos, el valor se almacena en la pila: primero se decrementa el puntero de pila según el tamaño del operando y luego se escribe el dato en la nueva dirección apuntada.

Ejemplos

```
# Supóngase: rsp = 0x7fffffffebc8

movq $45, %rax     # rax:=45 (no modifica rsp)
pushq %rax         # rsp:=rsp-8 = 0x7fffffffebc0; Mem[rsp]:=rax
pushw %ax          # rsp:=rsp-2 = 0x7fffffffebbe; Mem[rsp]:=ax
pushq $10          # rsp:=rsp-8 = 0x7fffffffebb6; Mem[rsp]:=10
pushq -8(%rbp)     # rsp:=rsp-8 = 0x7fffffffebae; Mem[rsp]:=Mem[rbp-8]
pushq a            # rsp:=rsp-8 = 0x7fffffffeba6; Mem[rsp]:=Mem[a]
```

La instrucción `pushq %rax` decrementa el registro `rsp` en 8 bytes y luego almacena el contenido de `rax` en la nueva dirección apuntada por `rsp`. De manera análoga, `pushw %ax`

decrementa `rsp` en 2 bytes y almacena el contenido de `ax`. La instrucción `pushq $10` empuja un valor inmediato, mientras que `pushq -8(%rbp)` y `pushq a` empujan en la pila el valor almacenado en las direcciones de memoria correspondientes.

Observación

En la arquitectura x86-64, la instrucción `push` puede llevar los sufijos `b` (byte, 8 bits), `w` (word, 16 bits), `l` (long, 32 bits) y `q` (quad, 64 bits). No obstante, en modo de 64 bits solo `pushq` y `pushw` son válidas. La primera es la forma habitual, ya que la pila se maneja en unidades de 64 bits, mientras que `pushw` se usa excepcionalmente para operaciones de compatibilidad con código de 16 bits.

3.15.2. Instrucción `POP`

La instrucción `pop` tiene la forma:

```
pop{w,q} <operando destino>
```

y tiene dos efectos:

1. copia el dato apuntado por el registro `rsp` al operando destino,
2. incrementa el valor de `rsp` en una cantidad de bytes de acuerdo al sufijo `S`.

Esta instrucción es la opuesta a `push` y, al igual que aquella, puede emplearse con distintos tipos de operandos, siempre que el tamaño sea válido (16 o 64 bits en modo de 64 bits). Los operandos posibles son:

- Registros generales (de 16 o 64 bits).
- Operandos en memoria.

Ejemplo

```
popq %rax
```

Copia en `%rax` el valor almacenado en la dirección de memoria apuntada por `%rsp` y luego incrementa `%rsp` en 8 bytes.

Por ejemplo, si antes de ejecutar la instrucción se tiene `rsp = 0x7fffffffebc0`, entonces luego `rsp = 0x7fffffffebc8`. El valor previo de `%rax` es irrelevante, ya que es sobrescrito.

Observaciones

- Las instrucciones `push` y `pop` pueden interpretarse como instrucciones complementarias. Es decir, en general se las utiliza de manera conjunta en concordancia.

- En la arquitectura x86-64 las instrucciones `push` y `pop` solo admiten los sufijos `w` y `q`. Sin embargo, lo usual es solo usar dichas instrucciones con el sufijo `q` para mantener la alineación de la pila.
- En la Sección 7.1 veremos en detalle cómo se utilizan las instrucciones `push` y `pop` para manejar la "pila".

3.15.3. Instrucción ENTER

La instrucción `ENTER` pertenece a la categoría de instrucciones de gestión de pila y del entorno de ejecución de funciones. Recibe dos operandos inmediatos y se utiliza para crear un nuevo marco de activación en la pila. En particular, el primer operando indica la cantidad de bytes a reservar para variables locales, mientras que el segundo operando especifica el nivel de anidamiento léxico.

Se trata de una instrucción compuesta que encapsula las operaciones típicas del prólogo de una función, incluyendo el almacenamiento del valor del registro base previo y la inicialización de un nuevo marco de pila. En su forma más simple (con nivel de anidamiento igual a cero), su comportamiento es equivalente a la siguiente secuencia de instrucciones:

```
pushq %rbp        # Guarda el valor del registro base del llamador en la
   pila
movq %rsp, %rbp   # Establece el nuevo marco de activación (rbp apunta al
    tope de la pila)
subq $N, %rsp     # Reserva N bytes en la pila para variables locales
```

El comportamiento de la secuencia anterior puede expresarse de manera equivalente mediante la instrucción `ENTER` en su forma más simple (nivel de anidamiento igual a cero):

```
enter $N, $0      # Reserva N bytes para variables locales y establece el
    nuevo marco de activación
```

El segundo operando de `ENTER` especifica el nivel de anidamiento léxico. Este mecanismo es característico de lenguajes con funciones anidadas y tiene escaso uso en la práctica moderna. Por lo tanto, es poco frecuente que el segundo operando sea mayor que cero en código real.

3.15.4. Instrucción LEAVE

La instrucción `LEAVE` también pertenece a la categoría de instrucciones de gestión de pila y del entorno de ejecución de funciones. No recibe operandos explícitos y opera implícitamente sobre los registros `rbp` y `rsp`. Se trata de una instrucción compuesta, equivalente a la siguiente secuencia de instrucciones:

```
movq %rbp, %rsp   # Restaura el puntero de pila (rsp) al inicio del marco
    actual.
popq %rbp         # Recupera el valor previo del registro base (rbp), es
   decir, el marco del llamador.
```

Se utiliza típicamente al final de una función, justo antes de `RET`, en el epílogo.

3.16. Instrucciones de entrada/salida por puertos

Las instrucciones de entrada/salida por puertos realizan lecturas y escrituras desde y hacia el espacio de direcciones de E/S (I/O). Este espacio de direcciones se utiliza para acceder y controlar dispositivos externos. Las operaciones de entrada/salida pueden realizarse mediante llamadas al sistema operativo (por ejemplo, usando `syscall`) o mediante instrucciones específicas de acceso a puertos (`in`, `out`). Estas últimas están típicamente restringidas a código con privilegios.

- `in <fuente>, <destino>`: lee un dato desde el puerto de E/S especificado en el operando fuente y lo almacena en el operando destino.
- `out <fuente>, <destino>`: escribe el dato especificado en el operando fuente en el puerto de E/S indicado por el operando destino.

El dato siempre está en el registro `AL/AX/EAX` y el puerto está en `DX` o es un valor inmediato.

Ejemplo

```
movw $0x3F8, %dx  # Puerto COM1 (serie)
movb $'A', %al    # Dato a enviar
outb %al, %dx     # Escribe en el puerto
inb %dx, %al      # Lee un byte desde el puerto
```

4. Comparaciones, Saltos y Estructuras de Control

Cualquier código estructurado requiere que la ejecución no siempre siga con la siguiente instrucción escrita, sino que ciertas veces el procesador debe continuar la ejecución en otra porción de código (por ejemplo, al llamar a una función o en distintas ramas de una estructura *if*). Para ello, todas las arquitecturas incluyen funciones de salto, tanto incondicionales como condicionales.

4.1. Saltos incondicionales

Los saltos incondicionales en ensamblador x86-64 permiten modificar el flujo de ejecución de un programa sin necesidad de evaluar ninguna condición previa. Utilizando la instrucción `jmp`, el control se transfiere directamente a otra parte del código, especificada mediante una dirección o una etiqueta. Este tipo de salto es esencial para implementar bucles, ramificaciones y llamadas indirectas, y es una herramienta clave en la programación de bajo nivel para estructurar el comportamiento del programa.

Como hemos visto, la instrucción `jmp` toma como único operando una dirección a la cual "saltar". El efecto que tiene este salto es que la próxima instrucción a ejecutar no será la siguiente al `jmp` sino la indicada en su operando. La dirección del salto en general se da usando etiquetas (ver Sección 2.4).

Ejemplo

```
    movq $0, %rax
    jmp cont
    movq $1, %rax
cont:
    movq $2, %rax
```

En el fragmento de código anterior la instrucción `movq $1, %rax` **nunca** es ejecutada ya que la instrucción `jmp` hace que el procesador salte a la instrucción en la dirección `cont`. Notar aquí que aunque `cont` es una constante (la dirección de memoria donde está la instrucción `movq 2, %rax`) ésta no va prefijada por `$`.

La instrucción `jmp` permite hacer saltos y es el equivalente a un `goto` de un lenguaje de alto nivel. Pero ¿cómo podemos implementar estructuras de control como bucles y condicionales con ella? Respuesta: no se puede. Para ello debemos introducir los saltos condicionales.

4.2. Saltos condicionales

Los saltos condicionales en ensamblador x86-64 permiten desviar el flujo de ejecución según el resultado de una operación previa, típicamente una comparación o una instrucción aritmética. Estas instrucciones evalúan los indicadores (flags) del registro de estado y realizan el salto solo si se cumple cierta condición, como igualdad, desigualdad o signo. Los saltos condicionales son fundamentales para implementar decisiones, bucles y estructuras de control en programas escritos en bajo nivel.

Los saltos condicionales en Assembler x86-64 utilizan una variedad de instrucciones que dependen del estado de los flags (indicadores) establecidos por instrucciones anteriores, como `cmp` o `test`. Estas instrucciones de salto comienzan con la letra `j` (de *jump*) seguida de un sufijo o abreviatura que indica la condición. En la Tabla 1 se mostró un listado completo de instrucciones `jCC` y los valores requeridos en las banderas.

Observación

Tanto los saltos condicionales como los incondicionales no llevan sufijo ya que su operando es siempre una dirección de memoria (dentro del segmento de código).

Junto con los saltos condicionales la arquitectura x86-64 incluye instrucciones para comparar dos valores. Una de estas instrucciones es la instrucción `cmp`. Como ya se mencionó, esta instrucción realiza una diferencia (resta) entre sus dos operandos, descartando el resultado pero **prendiendo los bits del registro** `rflags` acorde al resultado obtenido.

Siguiendo la lógica de la instrucción `sub`,

```
cmpq %rax, %rbx
```

realiza la resta `rbx-rax`, prende el bit `SF` (que indica negatividad) si `rax` es mayor que `rbx` pero a diferencia de `sub`, **no modifica el valor del registro destino** `rbx`. Notar que si ambos valores son iguales la resta tendrá un resultado nulo, prendiendo el bit `ZF`.

Como la relación que guardan dos valores (cuál es menor y cuál es mayor) depende de si dichos números se asumen con signo o sin signo, existen dos versiones de saltos condicionales por comparación de desigualdad. Por ejemplo:

- `jl` y `jg` (por *lower* y *greater*, respectivamente) para datos con signo.
- `jb` y `ja` (por *below* y *above*, respectivamente) para datos sin signo.

Ejemplo

```
    movq $45, %rbx
    movq $-66, %rcx
    cmpq %rbx, %rcx     # compara rcx - rbx --> (-66) - 45 = -111 --> SF
       =1, OF=0
    jl menor            # salta si (SF != OF), es decir, si rcx < rbx (
       comparación con signo)
    ....
    ....
menor:
    ....
```

Observación

Es necesario que la instrucción de comparación esté ubicada inmediatamente antes que la instrucción de salto condicional. Si se colocan otras instrucciones entre la comparación y el salto condicional, el registro `rFlag` puede ser alterado y por lo tanto es posible que el salto condicional no refleje la condición correcta.

4.3. Estructuras de Control

Tratemos ahora de traducir el siguiente fragmento de función C en ensamblador:

```
long a=0;
if (a==100) {
    a++;
}
// seguir
......
```

cuya estructura se ilustra en la Figura 6.

Teniendo en cuenta lo que vimos sobre saltos y comparaciones, una posible traducción sería:

```
.global main
main:
    movq $0, %rax
    cmpq $100, %rax
```

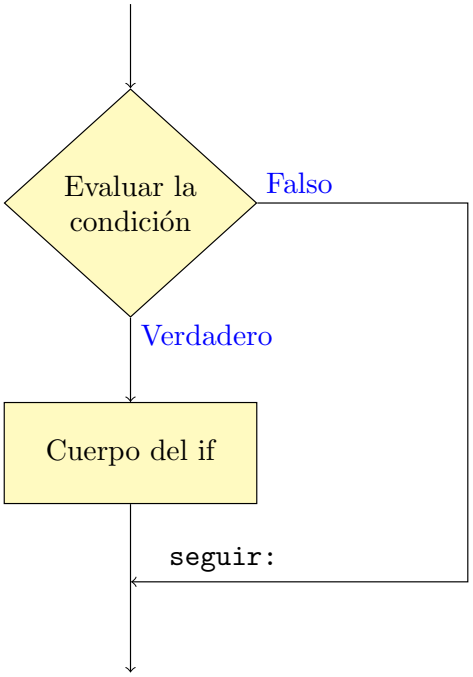


Figura 6: Estructura tipo `if`.

```
    jne seguir          # Si rax != 100, salta
    incq %rax           # Cuerpo del if
seguir:
    ...
```

En este código comparamos el valor de `rax` con la constante 100. Si el resultado dio cero (`rax-100`) es porque son iguales. En este caso debemos incrementar `rax`.

Veamos en el fragmento anterior varias cosas:

- El orden de los argumentos en la instrucción `cmp` es importante ya que la resta no es conmutativa. Notar también que esta instrucción necesita un sufijo de tamaño.
- Inmediatamente después de hacer la comparación realizamos el salto condicional. De tener más instrucciones en el medio, éstas podrían modificar el estado del registro `rflags`.
- Por la naturaleza del `if`, debemos definir dos etiquetas, una para saltar cuando la condición es verdadera (`igual_a_cien`) y otra para continuar la ejecución tanto si la condición fue verdadera o no (`seguir`). Notar que si la condición resulta falsa el programa salteará el bloque `igual_a_cien`.

Vemos ahora cómo traduciríamos el siguiente fragmento de código:

```
if (a==100) {
  a++;
} else {
  a--;
}
```

```
// seguir
```

Una posible traducción sería:

```
    cmpq $100, %rax
    je igual_a_cien     # Si rax == 100, salta al cuerpo del if
    decq %rax           # Cuerpo del else
    jmp seguir          # Salta al final de la estructura
igual_a_cien:
    incq %rax           # Cuerpo del if
seguir:
    ...
```

Vemos en el fragmento anterior varias cosas:

- En este caso si el salto condicional no se realiza (porque la condición resultó falsa) se ejecutará el decremento.
- Como ambas ramas del if deben unificarse, luego de hacer el decremento saltamos a `seguir` "salteando" la rama verdadera del if.
- Notar que como la etiqueta `seguir` está a continuación del bloque `igual_a_cien` el salto puede ser obviado.

4.4. Iteraciones

Otra estructura de control muy común en los lenguajes de alto nivel son las iteraciones, también denominadas bucles o lazos. Estas estructuras permiten ejecutar repetidamente un bloque de instrucciones mientras se cumpla una determinada condición o durante un número establecido de iteraciones.

En lenguaje ensamblador, las estructuras iterativas se implementan mediante instrucciones de comparación y saltos condicionales e incondicionales. Con las instrucciones estudiadas hasta ahora, es posible implementar la mayoría de las estructuras iterativas presentes en los lenguajes de alto nivel.

En primer lugar, consideraremos la estructura de control `while`, cuyo funcionamiento se representa en la Figura 7. En esta estructura, la condición se evalúa antes de ejecutar el cuerpo del bucle. Si la condición es verdadera, se ejecuta el bloque de instrucciones y se vuelve a evaluar la condición. Este proceso se repite hasta que la condición resulta falsa, momento en el cual se continúa con la instrucción siguiente al bucle.

Ejemplo

Supongamos que queremos traducir la siguiente estructura tipo `while`:

```
long int i;
while (i!=0) {
    cuerpo_del_while();
    i--;
}
```

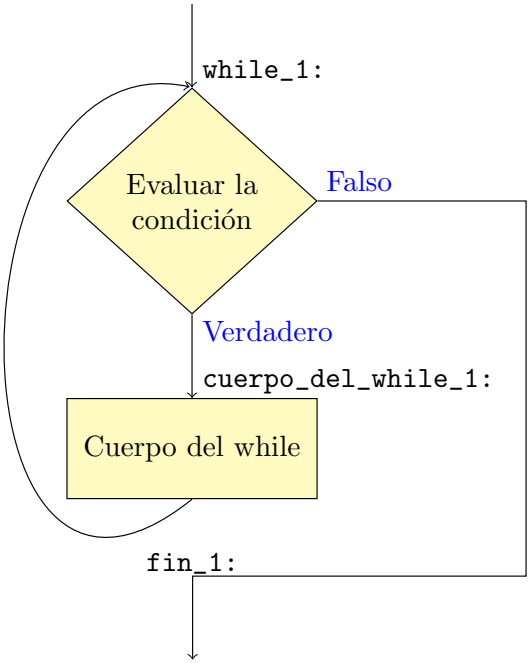


Figura 7: Estructura de control tipo `while`.

```
    cmpq                # Evaluar la condición
    je                  # Si resulta falsa, el lazo termina
                        # Cuerpo del while


    decq
    jmp
```

Las estructuras de repetición del tipo `for` son muy comunes en los lenguajes de alto nivel. Una de sus aplicaciones más habituales consiste en ejecutar un bloque de instrucciones un número determinado de veces.

Considere la siguiente estructura `for` en lenguaje C:

```
    int i;
for (i = 100; i > 0; i--) {
    cuerpo_del_for();
}
```

Este bucle ejecuta la función `cuerpo_del_for()` 100 veces, decrementando la variable `i` en cada iteración.

Una posible implementación en lenguaje ensamblador x86-64, utilizando la sintaxis AT&T, es la siguiente:

```
    movq $100, %rcx   # Inicialización del contador
bucle:
    cmpq $0, %rcx     # Se compara el contador con cero
    jle fin           # Si rcx <= 0, se termina el bucle
    # Instrucciones del cuerpo del bucle
    decq %rcx         # Se decrementa el contador
    jmp bucle         # Se repite el bucle
fin:
    ...
```

En esta implementación, el registro `%rcx` se utiliza como contador de iteraciones. Antes de ejecutar el cuerpo del bucle, se comprueba si su valor es mayor que cero. En caso contrario, se abandona el bucle.

Alternativamente, se puede utilizar la instrucción específica `loop`, que permite simplificar la implementación:

```
    movq $100, %rcx   # Inicialización del contador
bucle:
    # Instrucciones del cuerpo del bucle
    loop bucle
```

La instrucción `loop` decrementa automáticamente el registro `%rcx` y transfiere el control a la etiqueta indicada si el resultado es distinto de cero. De esta manera, las instrucciones comprendidas entre la etiqueta `bucle` y la instrucción `loop` se ejecutan exactamente 100 veces, siempre que el cuerpo del bucle no modifique `%rcx`.

5. Manejo de Arreglos y Cadenas

Un arreglo es una estructura de datos que almacena una colección de elementos del mismo tipo (por lo tanto del mismo tamaño) y le asigna un índice entero a cada uno. Existen distintas variantes de arreglos (largo fijo/variable, uni/multi-dimensional) pero en este apunte nos centraremos en arreglos a la "C", esto es, un arreglo `a` será la dirección del primer elemento (el de índice 0). Como cada elemento del arreglo tiene tamaño fijo al que llamaremos `s`, podemos calcular la dirección del elemento `i` del arreglo `a` como `a+i*s`.

Como los arreglos son estructuras de datos muy utilizadas, la arquitectura x86-64 incluye varias instrucciones (llamadas de cadena) para realizar copias, comparaciones, búsquedas, etc. Esta familia de instrucciones hace uso especial de dos registros: `rsi` (*source index*) y `rdi` (*destination index*) [4]. Cuando el procesador ejecuta una instrucción de cadena, éste incrementa/decrementa automáticamente esos registros [5] para apuntar al próximo elemento del arreglo. La cantidad incrementada/decrementada depende del tamaño del dato en cuestión. Además, el bit DF (*direction flag*) del registro `rflags` le indica al procesador si debe incrementar o decrementar los registros de índice (se puede apagar con `cld` para que se incrementen o prender con `std` para que se decrementen).

[4] Aunque su nombre sugieren que son índices, estos registros se utilizan como apuntadores en estas instrucciones.

[5] Algunas instrucciones solo incrementan/decrementan uno de estos registros.

A continuación veremos las diferentes instrucciones de manejo de arreglos y cadenas con sus respectivos ejemplos.

5.1. Copia y manipulación de datos

El procesador ofrece tres instrucciones para la copia y manipulación de datos almacenados en arreglos.

5.1.1. Instrucción LODS

La instrucción `lods` (de *load string*) copia en el registro `rax` (o en su sub-registro correspondiente) el valor apuntado por `rsi` e incrementa o decrementa `rsi` (dependiendo del valor de la bandera `DF`) en la cantidad de bytes indicada por el sufijo de tipo.

Así la instrucción `lodsw` (asumiendo `DF=0`) es equivalente a:

```
movw (%rsi),%ax
addq $2,%rsi
```

Notar que aquí se utiliza el subregistro `ax` para compatibilizar con el sufijo `w` de word y que por lo tanto el incremento es dos bytes.

5.1.2. Instrucción STOS

La instrucción `stos` (de *store string*) almacena el valor del registro `rax` (o su sub-registro correspondiente) en la dirección apuntada por `rdi` y luego incrementa/decrementa el valor de `rdi` en la cantidad de bytes indicada por el sufijo de tipo. Así, la instrucción `stosl` (asumiendo `DF=1`) equivale a:

```
movl %eax, (%rdi)
subq $4, %rdi
```

5.1.3. Instucción MOVS

La instrucción `movs` (de *move string*) realiza las acciones de `lods` y `stos` aunque sin utilizar el registro `rax`, esto es, copia el valor apuntado por `rsi` en la posición de memoria apuntada por `rdi` e incrementa/decrementa **ambos** en la cantidad de bytes indicada por el sufijo de tipo. Así la instrucción `movsb` (asumiendo `DF=0`) es equivalente a

```
movb (%rsi),%regtemp
movb %regtemp, (%rdi)
addq $1, %rsi
addq $1, %rdi
```

siendo `regtemp` un registro temporario del procesador (en realidad no existe ese registro).

Observación

Notar que las instrucciones para manejo de arreglos y cadenas trabajan con operandos implícitos, es decir, los operandos no se declaran explícitamente sino que ya viene prefijado con que operandos se trabaja.

Un caso típico de uso de estas instrucciones de cadena es para traducir el siguiente fragmento C:

```
int f(char *a, char *b) {
    int i;
    for (i=0;i<100;i++)
    a[i]=b[i];
}
```

que puede ser implementado en ensamblador como

```
.global f
f:
    # por convención de llamada tenemos en rdi el puntero al arreglo "a"
       y en rsi el puntero al arreglo "b"
    movq $100, %rcx      # debemos iterar 100 veces
    cld                  # iremos incrementando rsi y rdi (DF=0)
sigue:
    movsb
    loop sigue
    ret
```

Al repetir 100 veces la instrucción `movsb` copiamos los 100 bytes de `b` hacia `a`. El mismo efecto se podría haber obtenido copiando 50 veces un word (con `movsw`), 25 veces un long (con `movsl`) o 12 veces un quad (con `movsq`) y un long **extra**.

Supongamos que ahora debemos modificar el arreglo como sigue:

```
int f(int *a){
int i;
for (i=0;i<100;i++)
    a[i]++;
}
```

Esto puede ser escrito utilizando instrucciones de cadena como sigue:

```
.global f
f:
    # suponemos que rdi tiene el puntero al arreglo "a"
    movq %rdi, %rsi      # el origen y el destino son el mismo arreglo
    movq $100, %rcx      # iteramos 100 veces
    cld                  # iremos incrementando rsi y rdi (DF=0)
l:
    lodsl          # cargamos en eax el elemento del arreglo (apuntado por
        rsi)
    incl %eax      # lo incrementamos
    stosl          # lo guardamos en el arreglo (apuntado por rdi)
    loop l         # pasamos al siguiente elemento
    ret
```

Vemos que en este caso el uso del registro `eax` es útil para obtener el valor original del elemento (con `lodsl`), modificar el registro (con `incl`) y luego guardarlo de nuevo (con `stosl`). Notar también que en este caso el arreglo destino y origen son el mismo, por ello copiamos `rdi` en `rsi` al iniciar la función.

5.2. Búsquedas y Comparaciones

Una operación común es buscar un elemento dentro de un arreglo o comparar dos arreglos. La arquitectura ofrece para esto dos instrucciones.

5.2.1. Instrucción SCAS

La instrucción `scas` (de *scan string*) compara lo apuntado por `rdi` con el valor del registro `rax` (o del sub-registro según corresponda) e incrementa/decrementa `rdi` en la cantidad de bytes dada por el sufijo de tipo.

5.2.2. Instrucción CMPS

La instrucción `cmps` (de *compare string*) compara el valor apuntado por `rsi` con el valor apuntado por `rdi` e incrementa/decrementa ambos registros en la cantidad de bytes dada por el sufijo de tipo.

Al igual que la instrucción `cmp` estas comparaciones prenden los bits correspondiente en el registro `rflags`.

Ejemplo

Veamos un caso de uso de las instrucciones de búsquedas y comparaciones de cadenas. Supongamos que queremos implementar en ensamblador la siguiente función C que busca un elemento en un arreglo.

```
int find(int *a, int k){
int i;
for (i=0;i<100;i++)
    if (a[i]==k) return 1;
return 0;
}
```

Esta función puede ser implementada en ensamblador como sigue:

```
.global find
find:
    cld                 # iremos incrementando rdi (DF=0)
    movq $100, %rcx     # iteramos 100 veces
    movl %esi, %eax     # buscamos el 2do argumento
sigue:
    scasl               # comparamos el elemento actual con eax
    je found            # si lo encontramos terminamos
    loop sigue          # si no seguimos
    movq $0, %rax       # no lo encontramos, retornar 0
    jmp fin
found:
    movq $1, %rax       # lo encontramos, retornar 1
fin:
    ret
```

5.3. Iteraciones con instrucciones de cadena

Como vimos en los ejemplos anteriores, es lógico que una instrucción de cadena se repita muchas veces. Por ejemplo, una por cada elemento del arreglo o cadena. Para facilitar la escritura de estas estructuras iterativas la arquitectura ofrece la familia de **prefijos** `rep` que pueden ser antepuestos a cualquier instrucción de cadena. Al igual que la instrucción `loop` el prefijo repite la instrucción la cantidad de veces indicada por `rcx`. Así, el ejemplo de copia de un arreglo a otro de la Sección 5.1 puede ser reescrito en ensamblador como:

```
.global find
.global f
f:
    # por convención de llamada tenemos en rdi el puntero al arreglo a y
       en rsi el puntero al arreglo b
    movq $100, %rcx           # debemos iterar 100 veces
    cld                       # iremos incrementando rsi y rdi (DF=0)
    rep movsb                 # repite movsb 100 veces
    ret
```

Al igual que existen los saltos condicionales, existen los prefijos de repetición condicionales. Así, los prefijos `repe` y `repne` repiten la instrucción mientras el bit `Z` esté prendido/apagado a lo sumo `rcx` veces. El ejemplo de la búsqueda de un entero de la Sección 5.2 puede ser reescrito utilizando prefijos de repetición condicional como:

```
.global find
find:
    cld                 # iremos incrementando rdi (DF=0)
    movq $100, %rcx     # iteramos 100 veces
    movl %esi, %eax     # buscamos el 2do argumento
    repne scasl         # repetimos mientra sea distinto o a lo sumo 100
       veces
    je found            # si lo encontramos terminamos
    movq $0, %rax       # no lo encontramos, retornar 0
    jmp fin
found:
    movq $1, %rax       # lo encontramos, retornar 1
fin:
    ret
```

Observación

Notemos que el prefijo `repne` repite la instrucción mientras la comparación resulte distinta y a lo sumo `rcx` veces, pero ¿cómo saber por cuál de las dos causas finalizó la repetición?

Cuando la condición del prefijo resulta falsa los registros `rsi,rdi` son incrementados o decrementados según corresponda y el registro `rcx` es decrementado pero los bits del registro `rflags` quedan intactos dejando allí el valor de la última comparación. Por lo tanto, podemos realizar un salto condicional para ver si la última comparación dio igual o distinto.

6. Acceso a datos en memoria

La memoria principal constituye el espacio donde se almacenan los datos y las instrucciones que utiliza un programa durante su ejecución. Desde el punto de vista del procesador, la memoria es un conjunto lineal de posiciones identificadas mediante direcciones, sin información adicional sobre el tipo o significado de los datos almacenados. Por lo tanto, el acceso correcto a la información requiere conocer tanto la dirección de memoria como la representación utilizada para almacenar los datos.

En esta sección se estudian los mecanismos utilizados para acceder a datos en memoria en la arquitectura x86-64. En primer lugar, se analiza la definición y disposición de variables en memoria, junto con el concepto de *endianness*, que determina cómo se representan los datos de múltiples bytes. Luego se aborda la diferencia entre acceder al valor almacenado y acceder a su dirección, para finalmente estudiar los principales modos de direccionamiento y la desreferenciación de memoria.

6.1. Definición de variables globales

Para acceder a datos almacenados en memoria, los programas en lenguaje ensamblador utilizan etiquetas que identifican posiciones de memoria reservadas para almacenar información. Estas etiquetas cumplen un rol similar al de las variables en los lenguajes de alto nivel.

La definición de variables globales inicializadas en un programa en ensamblador se realiza en la sección `.data`. Los datos globales que no requieren un valor inicial suelen ubicarse en la sección `.bss`. Los datos en estas secciones se definen utilizando las directivas vistas en la Sección 2.3, las cuales permiten reservar espacio de memoria y especificar sus valores iniciales.

Ejemplo

```
.data
a: .byte 5     # Reserva 1 byte en la dirección etiquetada como 'a' e
   inicializa su contenido con el valor 5
var: .long 0x12345678     # Reserva 4 bytes (32 bits) en la dirección
   etiquetada como 'var' e inicializa su contenido con 0x12345678
f:   .quad 0x1122334455667788 # Reserva 8 bytes (64 bits) en la dirección
    etiquetada como 'f' e inicializa su contenido con 0x1122334455667788

.bss
c:   .quad                  # Reserva 8 bytes (64 bits) en la dirección
   etiquetada como 'c' sin inicialización explícita (.bss)
```

Es importante destacar que en ensamblador hay que estar muy alerta cuando accedemos a las variables que hemos definido previamente, como veremos a continuación.

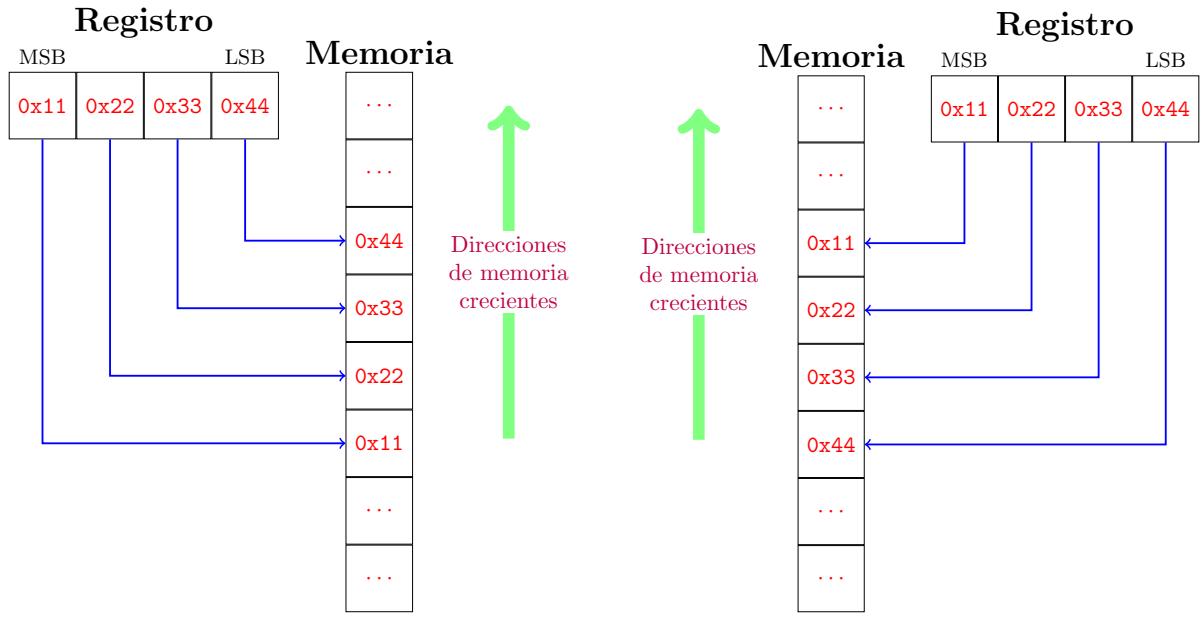
6.2. Endianness

"¿Qué ocurre si se escribe una palabra y, al observar cómo queda almacenada en memoria, sus bytes aparecen en un orden diferente al esperado? ¡Endianness!"

El término inglés *endianness* designa el formato en el que se almacenan en memoria los datos de más de un byte. El problema es similar a los idiomas en los que se escribe de derecha a izquierda, como el árabe, o el hebreo, frente a los que se escriben de izquierda a derecha, pero trasladado de la escritura al almacenamiento en memoria.

Supongamos que tenemos que almacenar el entero `0x11223344` en la dirección de memoria `a`. Este valor se representa mediante los cuatro bytes 0x11 0x22 0x33 0x44, escribiendo más a la izquierda el byte más representativo (MSB) y más a la derecha el byte menos representativo (LSB).

Una opción es guardar el byte **más** significativo (`0x11`) en la dirección `a`, el segundo (`0x22`) en la dirección `a+1`, y así sucesivamente. Esto se conoce como convención *Big-Endian*.



(a) Big-Endian (b) Little-Endian

Figura 8: Convenciones de *Endianness*.

Otra opción es almacenar en la dirección `a` el byte **menos** significativo (`0x44`), el siguiente (`0x33`) en la dirección `a+1` y así sucesivamente. Esta convención se denomina *Little-Endian* y es la utilizada por las arquitecturas x86, y por lo tanto, también por x86-64. La Figura 8 ilustra ambas convenciones.

Ejemplo

```c
#include <stdio.h>

int main(void){
    unsigned int x = 0x41424344; // 'A' 'B' 'C' 'D'

    printf("Valor de x: 0x%X\n", x);
    printf("Bytes en memoria: ");
    unsigned char *p = (unsigned char *)&x;

    for (int i = 0; i < 4; i++)
        printf("%c ", p[i]);
    printf("\n");
    return 0;
}
```

En una máquina x86-64, que utiliza little-endian, la salida será:

```
Valor de x: 0x41424344
Bytes en memoria: D C B A
```

6.3. Organización de datos en memoria

Los datos se almacenan en memoria de manera contigua a medida que se declaran en el código. Cada dato se identifica mediante una **etiqueta** (label), que el ensamblador asocia con una dirección de memoria concreta. No existe, sin embargo, ningún delimitador implícito entre datos consecutivos: la separación entre ellos está dada únicamente por su tamaño y por el orden en que se ubican en memoria.

En el caso de los **vectores** (arreglos), la situación es análoga: todos sus elementos se almacenan en posiciones de memoria contiguas y bajo una única etiqueta que referencia al primer elemento. El acceso a cada componente se realiza mediante desplazamientos (offsets) respecto de dicha dirección base, típicamente en función del tamaño de cada elemento. Por ejemplo, si cada elemento ocupa k bytes, el elemento i se encuentra en la dirección base más un desplazamiento de $i \cdot k$ bytes.

A continuación se presenta un ejemplo ilustrativo.

Ejemplo

```
.data
vec: .byte 10, 20, 30, 40
var: .long 0x11223344
str: .asciz "Hola\n"
```

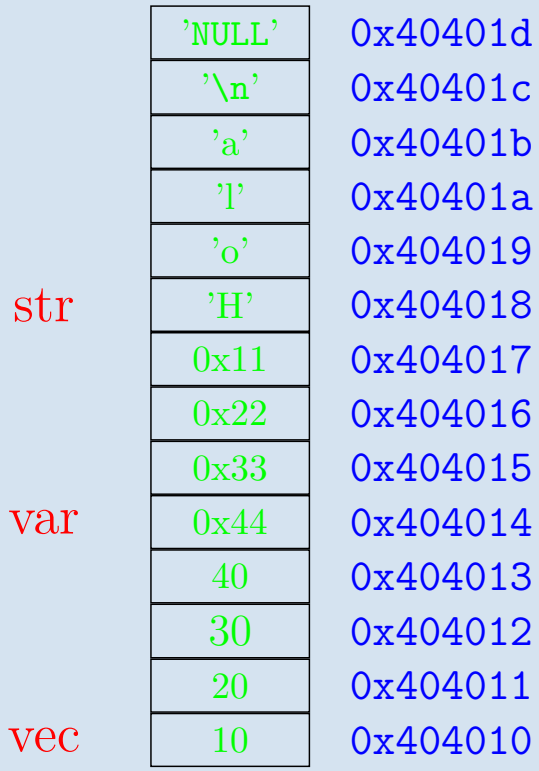
Las datos se encontrarán en memoria tal como se muestra en el siguiente esquema (suponiendo que la etiqueta `vec` corresponde a la dirección `0x404010`).

Debemos tener cuidado con el acceso a los datos en memoria. Por ejemplo, la instrucción `movq var, %rax` copia 8 bytes a partir de la dirección asociada a `var`. Es decir, el procesador lee 8 bytes consecutivos comenzando en `var`. Por lo tanto, dado que los

datos se almacenan en formato *little-endian*, en el registro `%rax` quedará cargado el valor `0x616c6f4811223344`.

Si este acceso no es el deseado, el ensamblador no reportará ningún error, ni tampoco se producirá necesariamente una falla durante la ejecución; en general, este tipo de problemas solo puede detectarse mediante pruebas y depuración.

Conclusión: El acceso a los datos es muy flexible, pero si no se controla adecuadamente, esta flexibilidad puede dar lugar a errores sutiles y difíciles de detectar.



Observación

Las cadenas de caracteres se almacenan en memoria de manera secuencial, byte a byte, desde el primer carácter hasta el último, ocupando direcciones de memoria crecientes. Es decir, cada carácter se guarda en un byte consecutivo respecto del anterior. El concepto de *endianness* no aplica a las cadenas de caracteres, ya que este criterio solo afecta la forma en que se ordenan los bytes que componen datos de longitud fija mayores a un byte (por ejemplo, enteros o números en punto flotante). En el caso de las cadenas, cada carácter constituye una unidad independiente de un byte, por lo que su orden en memoria coincide directamente con el orden lógico de la cadena.

Ejemplo

Veamos ahora un ejemplo completo que engloba todos los conceptos vistos en las secciones anteriores:

```
.data
```

```
i: .long 0
f: .double 3.14
str: .asciz "Hola mundo"

.bss
a: .quad

.text
.global main
main:
    movq $40, %rax      # rax=40
    movl i, %ebx        # ebx=0
    movq $-1, a         # a = -1 (0xffffffffffffffff en complemento a
       dos, 64 bits)
    movq f, %rdx        # rdx=0x40091eb851eb851f (3.14)
    movl str, %ecx      # ecx=0x616c6f48 ("aloH")
    ret
```

Aquí vemos que la etiqueta `i` (dentro del segmento de datos `.data`) define la posición de memoria donde el ensamblador alojará un entero inicializado en 0 (4 bytes). Luego en `f` un valor de punto flotante inicializado en 3.14 (8 bytes). Luego en `str` arranca una cadena de caracteres de 11 bytes (el byte final corresponde al cero final). En el segmento `.bss` se crea una variable tipo `quad` sin inicializar. Finalmente, vemos que dentro del segmento de código se define una etiqueta global llamada `main`. Este será el punto de inicio del programa. Luego, a medida que se vayan ejecutando las instrucciones siguientes los registros y locaciones de memoria irán quedando con los valores indicados en los comentarios.

Notar que luego de ejecutarse la instrucción `movl str, %ecx` el valor del subregistro `ecx` es `"aloH"`, que corresponde a los primeros 4 bytes de la cadena `str` pero con los caracteres en orden invertido debido al formato *little-endian*. Sin embargo, hay que notar que las cadenas de caracteres se almacenan en la memoria "concatenando" los caracteres consecutivamente desde el primer carácter hasta el último comenzando en las posiciones más bajas de memoria. Por lo tanto, el carácter `'H'` estará almacenado en la posición `str`, el carácter `'o'` en la posición `str+1`, y así sucesivamente hasta llegar al último carácter que es el `null`.

6.4. Acceso a memoria: valor vs dirección

Como hemos visto, podemos acceder a un dato en memoria utilizando la etiqueta que define la dirección de memoria donde dicho dato comienza. Ahora supongamos que queremos incrementar el valor de una variable definida por la etiqueta `i`, esto podemos hacerlo simplemente escribiendo:

```
incq i
```

Si antes era `i=23`, ahora es `i=24`. Es decir,

$$\texttt{M[i]} \leftarrow \texttt{M[i]} + 1$$

Es importante notar que aunque la etiqueta `i` es una constante, es decir, la dirección de memoria donde se aloja ese valor, la etiqueta `i` no lleva el signo `$`.

Si ahora quisiéramos sumar `i` con el registro `rax` podemos escribir:

```
addq i, %rax
```

En `%rax` queda la suma de su valor con el valor almacenado en `i`. Es decir,

$$\texttt{\%rax} \leftarrow \texttt{\%rax} + \texttt{M[i]}$$

Muchas veces es útil conocer la dirección de memoria donde está alojado un valor. Esto en C se conoce como obtener un puntero al dato. Así, si tenemos una variable `long int i;` podemos obtener un puntero a dicha variable utilizando el operador de referencia, escribiendo `&i`. Como antes mencionamos, en ensamblador una etiqueta es una dirección de memoria constante. Por ello si quisiéramos obtener el valor de esa dirección podríamos escribir:

```
movq $i, %rax
```

Luego `%rax` guardará la dirección de memoria asociada a la etiqueta `i`, es decir, su dirección efectiva. En notación formal, esto puede expresarse como:

$$\texttt{\%rax} \leftarrow \&i$$

Esta instrucción es equivalente a:

```
leaq i, %rax
```

La instrucción `lea` se analiza en profundidad en la Sección 3.14.1.

Ejemplo

Veamos un ejemplo interesante para ver la diferencia entre usar una etiqueta y el valor allí guardado.

```
.data
str: .asciz "hola mundo"

.text
.global main
main:
    movq str , %rax   # Instruccion 1
    movq $str, %rax   # Instruccion 2
    ret
```

¿Qué diferencia hay entre la instrucción 1 y la 2? Aunque muy similares, las dos instrucciones son muy distintas entre sí. Ambas son un movimiento con destino a `rax`, pero veamos qué mueven.

Al ejecutar la primera, `rax` toma el valor de 7959387902288097128. ¿Qué ha ocurrido aquí? La instrucción le indica al procesador que debe copiar 8 bytes (ya que es un quad) desde la región de memoria indicada por la etiqueta `str` a `rax`. Como en esa región de memoria se aloja la cadena de caracteres `"hola mundo"` los primeros 8 bytes son `hola mun` y de allí el valor tan extraño. El valor 7959387902288097128 se puede descomponer en hexadecimal en los siguientes bytes `0x6e 0x75 0x6d 0x20 0x61 0x6c 0x6f 0x68`, donde

cada uno corresponde en decimal a `110 117 109 32 97 108 111 104` y al convertirlo en caracteres ASCII son "num aloh" (notar que la frase aparece al revés por ser x86-64 little endian).

Al ejecutar la segunda lo que ocurrirá es que en `rax` se guardará la **dirección de memoria** donde está guardada la cadena de caracteres. Este valor dependerá del proceso de compilación y carga. Notemos que en este caso ningún carácter de esa cadena será copiado a `rax`. De hecho esa instrucción no accede a la memoria.

6.5. Modos de direccionamiento de memoria

Las instrucciones pueden acceder a memoria de distintas formas, según cómo se especifique la dirección del operando. Estas formas se conocen como modos de direccionamiento. Es decir, los modos de direccionamiento describen cómo obtener una dirección efectiva (EA, *Effective Address*) a partir de los operandos de la instrucción.

Definición

Dirección efectiva (*Effective Address*, EA): Es la dirección de memoria resultante de evaluar el modo de direccionamiento. Es decir, no es la forma simbólica o parcial que aparece en la instrucción, sino el valor concreto (numérico) que el procesador usa para acceder a memoria.

Observación

Es importante notar que el uso de un modo de direccionamiento no implica necesariamente un acceso a memoria. Por ejemplo, la instrucción `lea` (vista en la Sección 3.14.1) utiliza modos de direccionamiento para calcular direcciones, pero no desreferencia memoria.

En la Sección 2.7 se mostró el acceso directo mediante etiquetas. A continuación, se presentan los modos de direccionamiento más habituales en sintaxis AT&T, que permiten construir direcciones de memoria de manera más general y flexible.

6.5.1. Modo de direccionamiento directo (absoluto)

En el modo de **direccionamiento directo (o absoluto)**, la dirección efectiva del operando se obtiene directamente a partir de un valor codificado en la propia instrucción. Este valor puede escribirse como una constante numérica, un símbolo (por ejemplo, una etiqueta) o una expresión aritmética formada por constantes, símbolos y operadores; en este último caso, la expresión la evalúa el ensamblador o el enlazador, de modo que en la instrucción siempre queda codificado un valor constante.

En x86-64, la forma habitual de codificar este direccionamiento utiliza un campo de desplazamiento de 32 bits que se extiende con signo a 64 bits para obtener la dirección efectiva, por ejemplo `movq 0x1000, %rax`. No se trata de una limitación de la capacidad de direccionamiento del procesador, sino del formato de codificación de las instrucciones.

Existe, no obstante, una variante de `mov` que permite codificar directamente una dirección absoluta de 64 bits: la instrucción `movabsq`, cuando se usa con un operando de memoria, como en `movabsq 0x123456789a, %rax`. Esta forma solo admite como

otro operando el registro acumulador (`%al`, `%ax`, `%eax` o `%rax`). Debe distinguirse de `movabsq $imm64, %reg`, que carga un valor inmediato de 64 bits en un registro y no accede a memoria.

Ejemplos

```
movq 0x404028, %rax      # Carga en rax los 8 bytes almacenados a partir
   de la dirección 0x404028
movq a, %rax             # Carga en rax los 8 bytes almacenados a partir
   de la dirección de la etiqueta a
movq a+8, %rax           # Ídem, a partir de la dirección a+8 (calculada
   al ensamblar/enlazar)
movq a+8*2, %rax         # Ídem, a partir de la dirección a+16 (calculada
    al ensamblar/enlazar)
movabsq 0x123456789a, %rax # Carga en rax los 8 bytes almacenados a
   partir de la dirección de 64 bits 0x123456789a
```

Observación

Los ejemplos que utilizan etiquetas requieren generar un ejecutable **no independiente de la posición** (no PIE), con la opción `-no-pie` de `gcc`. En instrucciones como `movq a, %rax`, la dirección de la etiqueta se codifica en un campo de 32 bits extendido con signo. En un ejecutable no PIE, el programa se carga en una dirección fija y baja, y esa dirección cabe en 32 bits. En un ejecutable PIE, en cambio, la dirección de carga no se conoce hasta el momento de la ejecución y normalmente no cabe en 32 bits, por lo que el enlazador no puede completar la instrucción y termina con un error. Como muchas distribuciones generan ejecutables PIE por defecto, sin `-no-pie` estos ejemplos no enlazan. En código PIE se utiliza en su lugar el direccionamiento relativo a RIP, por ejemplo `movq a(%rip), %rax`.

6.5.2. Modo de direccionamiento general

El modo de direccionamiento más general en x86-64 permite calcular la dirección efectiva combinando un desplazamiento inmediato, un registro base, un registro índice y un factor de escala. Los demás modos de direccionamiento se obtienen de manera más simple al omitir uno o más de estos términos.

En el caso general , el operando que hace referencia a memoria tiene la siguiente forma general:

`desplazamiento(%base,%índice, escala)`

donde la base y el índice pueden ser cualquier registro de propósito general, la escala puede ser 1, 2, 4 u 8 y el desplazamiento ha de ser un número representable con 32 bits. De esta manera la dirección especificada resulta:

$$\text{EA} = \text{desplazamiento} + \text{base} + \text{índice} \times \text{escala}$$

Ejemplos

```
3(%rbx, %rcx, 4)        # EA = 3 + rbx + rcx*4
a(%rbx, %rax, 2)        # EA = a + rbx + rax*2
```

Modo base En este modo, la dirección efectiva está contenida únicamente en un registro base. Es decir, el operando de memoria se obtiene a partir del contenido de dicho registro.

```
EA = base
```

Ejemplo

```
(%rbx)    # EA = rbx
```

Modo base + desplazamiento En este modo, la dirección efectiva se calcula como la suma del contenido de un registro base y un desplazamiento constante.

```
EA = base + desplazamiento
```

Ejemplo

```
8(%rax)    # EA = rax+8
```

Modo indexado En este modo, la dirección efectiva se calcula como la suma del contenido de un registro base, un desplazamiento y el producto de un registro índice por un factor de escala.

```
EA = base + índice*escala
```

Ejemplo

```
(%rax, %rcx, 4)   # EA = rax+rcx*4
```

Observación

En la práctica, todos los modos de direccionamiento de memoria en x86 pueden verse como casos particulares de una única forma general. Las distintas variantes que aparecen en los ejemplos no son más que simplificaciones donde algunos de los componentes no están presentes.

6.6. Desreferenciación de memoria

Para desreferenciar una dirección de memoria se utilizan los modos de direccionamiento vistos en la sección anterior. Una instrucción de la forma:

```
mov{b,w,l,q} <src>, <dest>
```

puede involucrar acceso a memoria si alguno de los operandos es un operando de memoria. En ese caso, la dirección efectiva (*Effective Address*, EA) se calcula utilizando los modos de direccionamiento vistos previamente, y se accede al contenido almacenado en dicha dirección.

Casos posibles

- Lectura desde memoria:

  `mov{b,w,l,q} EA, REG` $\Rightarrow$ `REG = Mem[EA]`

- Escritura en memoria:

  `mov{b,w,l,q} REG, EA` $\Rightarrow$ `Mem[EA] = REG`

- No se permite que ambos operandos sean referencias de memoria.

Observación

Este mecanismo es equivalente al operador de desreferenciación en el lenguaje C. Por ejemplo:

- `movq (%rax), %rbx` $\longleftrightarrow$ `rbx = *rax;`
- `movq %rbx, (%rax)` $\longleftrightarrow$ `*rax = rbx;`
- `movq 8(%rax), %rbx` $\longleftrightarrow$ `rbx = *(rax + 8);`

En todos los casos, el operando entre paréntesis indica que el valor del registro se interpreta como una dirección de memoria, y no como un dato inmediato.

Ejemplos

El sufijo de la instrucción `mov` determina la cantidad de bytes transferidos: `b` indica 1 byte, `w` indica 2 bytes, `l` indica 4 bytes y `q` indica 8 bytes.

```
movq (%rax), %rbx              # rbx <--- Mem64[rax] (lee 8 bytes)
movl %eax, 8(%rsp)             # Mem32[rsp+8] <--- eax (guarda 4 bytes)
movb 8(%rbp), %al              # al <--- Mem8[rbp+8]
movw -16(%rbp), %ax            # ax <--- Mem16[rbp-16]
movl %eax, 0x20(%rsp)          # Mem32[rsp+32] <--- eax
movq (%rax,%rax,2), %rbx       # rbx <--- Mem64[rax+rax*2]
movq -4(%rbp,%rdx,4), %rbx     # rbx <--- Mem64[rbp+rdx*4-4]
movq 8(,%rax,8), %rbx          # rbx <--- Mem64[rax*8+8]
movb a(,%rax,1), %bl           # bl <--- Mem8[a+rax*1]
```

Ejemplo

Si tenemos un arreglo de enteros de 32 bits (4 bytes) cuya dirección inicial está almacenada en `rax`, y queremos acceder a uno de sus elementos mediante un índice, podemos utilizar el modo de direccionamiento indexado:

```
.data
a: .long 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

....
leaq a, %rax      # rax <--- &a (dirección base del arreglo, = &a[0])
                  # Equivalente a movq $a, %rax; no desreferencia memoria
                      (solo calcula la dirección efectiva)
movq $5, %rcx
movl (%rax,%rcx,4), %edx   # edx <--- Mem32[rax+4*5] = a[5] = 6
```

7. Gestión de la pila

En la arquitectura x86-64, la **pila** (*stack*) es un mecanismo fundamental para el control del flujo de ejecución y la organización del espacio de memoria durante las llamadas a funciones. Se trata de una estructura de tipo **LIFO** (*Last In, First Out*) utilizada para almacenar direcciones de retorno, parámetros de funciones, variables locales y registros temporales. Su manejo correcto resulta esencial para garantizar la integridad de la ejecución del programa y la seguridad del sistema.

A diferencia de arquitecturas más simples, la arquitectura x86-64 amplía el conjunto de registros y establece convenciones específicas para la gestión de la pila, incluyendo el uso de los registros `rsp` (*stack pointer*) y `rbp` (*base pointer*), así como reglas estrictas para la alineación de la pila y el paso de argumentos. Comprender estas convenciones permite interpretar correctamente el comportamiento de programas compilados, depurar errores, y escribir código ensamblador eficiente y compatible con funciones escritas en lenguajes de alto nivel como C o C++.

7.1. Definición de pila

Una pila es una estructura de datos que permite almacenar información de manera ordenada. Su funcionamiento puede entenderse mediante la analogía con una pila de platos sobre una mesa: uno puede ir apilando platos, y la pila crecerá hacia arriba. Para retirar un plato, se debe quitar el que está en la parte superior, lo que reduce el tamaño de la pila. Como se observa, al extraer un elemento de la pila, se remueve el último que fue insertado (si lo hubiera). Por esta razón, la estructura de datos pila se conoce como **LIFO** (*Last-In, First-Out*), ya que el último en entrar es el primero en salir.

La arquitectura x86-64 permite utilizar una región de la memoria como pila (*stack*). La pila puede emplearse para distintas finalidades, tales como:

- **Almacenamiento temporal:** por ejemplo, variables locales (automáticas) en C suelen ubicarse en la pila, aunque su asignación final depende de optimizaciones del compilador.

- **Implementación de llamadas a funciones:** el orden de llamada y finalización sigue el comportamiento LIFO de una pila. Si la función `f` llama a `g` y `g` llama a `h`, la primera en finalizar es `h`, seguida de `g` y finalmente `f`. Esto resulta especialmente relevante en funciones recursivas.
- **Preservación de registros:** como se explica en la Sección 8, algunos registros pueden ser modificados durante una llamada a función. Sus valores pueden almacenarse en la pila y restaurarse posteriormente.

Aunque la pila puede emplearse con distintos propósitos, su uso principal es dar soporte a las llamadas a funciones: cada invocación construye sobre ella una estructura bien definida, denominada **marco de activación**. El marco de activación es la porción de la pila asociada a una ejecución particular de una función y contiene la información necesaria para gestionarla: las variables locales, los parámetros (cuando no se pasan íntegramente en registros), la dirección de retorno, los registros que deban preservarse y, en general, cualquier dato que deba conservarse desde la llamada hasta el retorno.

Cada llamada a función crea su propio marco de activación, que se apila sobre el de la función invocante y se libera al retornar. Los marcos se gestionan, por tanto, en orden LIFO: el último en crearse es el primero en destruirse. Gracias a ello, varias invocaciones simultáneas de una misma función, como ocurre en la recursión, disponen de marcos independientes. En un instante dado, el **marco de activación activo** es el que se encuentra en el tope de la pila y corresponde a la invocación en curso. La Figura 9 ilustra un posible estado de la pila con varios marcos de activación apilados.

Observación

En la Figura 9 se observa que el último elemento insertado en la pila se ubica en direcciones **más bajas de memoria**. Es decir, en x86-64 la pila crece hacia direcciones decrecientes. Este comportamiento responde a razones históricas de diseño y a la conveniencia de permitir que las distintas regiones del espacio de direcciones crezcan en sentidos opuestos, maximizando el uso del espacio disponible: típicamente, el segmento de datos crece desde direcciones bajas hacia altas, mientras que la pila lo hace en sentido inverso. Este aspecto se analiza con mayor detalle en el apunte **Organización y gestión de la memoria**.

Observación

Si se compila con `-fomit-frame-pointer` (opción que GCC y Clang activan por defecto a partir de `-O1`), el registro `%rbp` no se utiliza como puntero de marco y los accesos al marco de activación se realizan mediante direcciones relativas a `%rsp`. Por este motivo, en el diagrama de la Figura 9 el valor guardado de `%rbp` figura como opcional. Esta la cuestión la veremos con mayor detalle en la Sección 8.4.

7.2. Uso de registros en la gestión de la pila

La arquitectura x86-64 dispone de dos registros que desempeñan funciones específicas en la gestión de la pila:

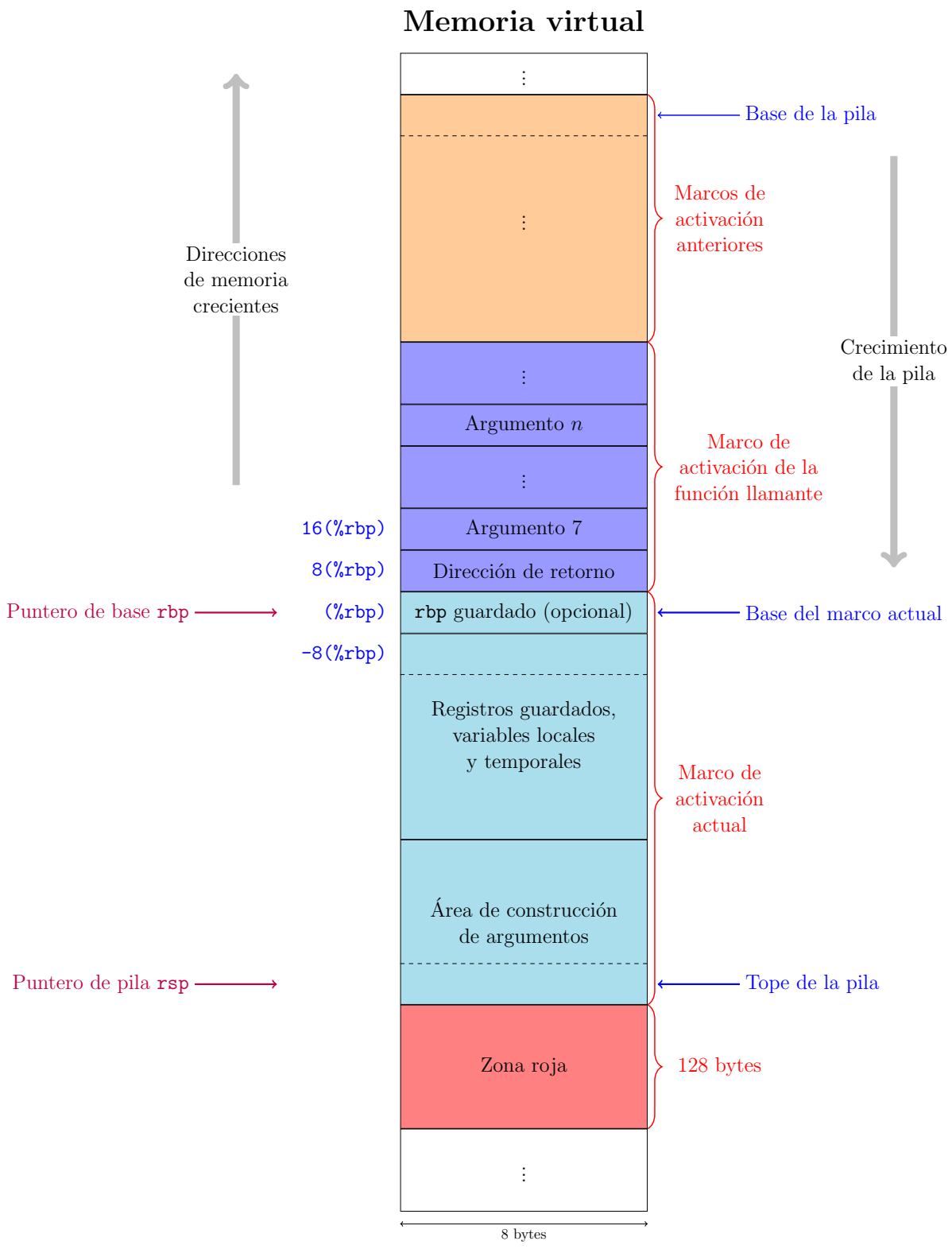


Figura 9: Diagrama típico de la estructura de pila en arquitectura x86-64.

rsp (*stack pointer*): registro de 64 bits que contiene la dirección del tope de la pila. Dado que la pila crece hacia direcciones de memoria decrecientes, `rsp` apunta al último dato apilado.

rbp (*base pointer*): registro de 64 bits que se utiliza, por convención, como referencia fija

dentro del marco de activación de una función. Su valor suele mantenerse constante durante la ejecución de la función, lo que facilita el acceso a variables locales y otros datos del marco mediante desplazamientos constantes.

Aunque ambos registros cumplen roles específicos en la gestión de la pila (como se verá a continuación), no son registros privilegiados: pueden manipularse mediante instrucciones generales como `add`, `sub`, `mov`, entre otras. No obstante, su modificación directa debe realizarse con cuidado para preservar la coherencia del marco de activación y de la propia pila.

7.3. Operaciones sobre la pila

Como se mencionó en las Secciones 3.15.1 y 3.15.2, la arquitectura x86-64 también provee instrucciones específicas para operar sobre la pila: `push`, que inserta (apila) un elemento, y `pop`, que extrae (desapila) el elemento ubicado en el tope.

La instrucción `pushq` primero decrementa el registro `rsp` en 8 (recordemos que la pila crece hacia direcciones más bajas) y luego almacena su operando en los 8 bytes que comienzan en la nueva dirección apuntada por `rsp`. Así, la instrucción

```
pushq %rax
```

es equivalente, en cuanto a su efecto sobre la pila y la memoria, a

```
subq $8, %rsp          # Reserva 8 bytes en la pila (RSP <-- RSP - 8)
movq %rax, (%rsp)      # Guarda RAX en los 8 bytes apuntados por RSP
                       # (M[RSP] <-- RAX)
```

como se observa en la Figura 10.

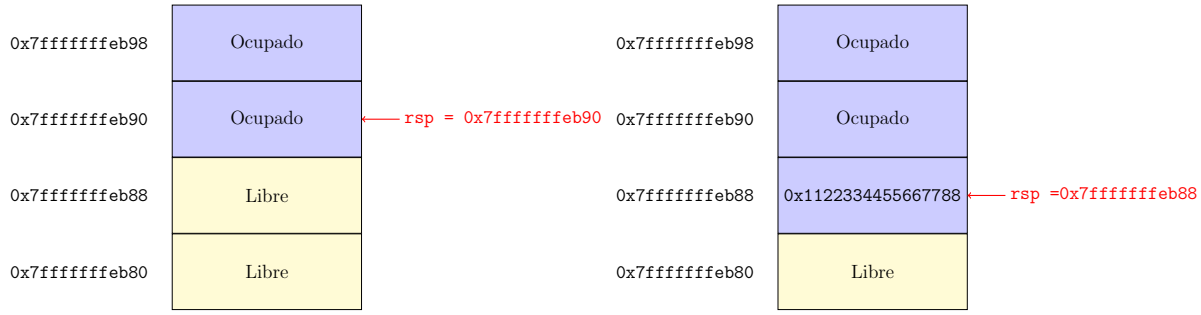


Figura 10: Diagrama de la pila antes y después de ejecutar la instrucción `pushq %rax`.

Por otra parte, la instrucción `popq` primero copia en su operando los 8 bytes ubicados en el tope de la pila y luego incrementa el registro `rsp` en 8, de modo que el tope se desplaza hacia direcciones más altas. Así, la instrucción

```
popq %rax
```

es equivalente a

```
movq (%rsp), %rax      # Carga en RAX los 8 bytes del tope de la pila
                       # (RAX <-- M[RSP])
addq $8, %rsp          # Libera el espacio que ocupaba el dato
                       # (RSP <-- RSP + 8)
```

La Figura 11 muestra el comportamiento de `popq`, suponiendo que el tope de la pila contiene el valor `0x1122334455667788` (por ejemplo, como resultado del `pushq` anterior). Nótese que, aunque ese valor deja de estar en el tope de la pila tras la operación, **permanece almacenado en la dirección** `0x7fffffffeb88`. Es decir, `popq` no borra el contenido de la memoria: además de escribir el operando destino, solo actualiza el registro `rsp`, con lo que cambia la posición lógica del tope de la pila.

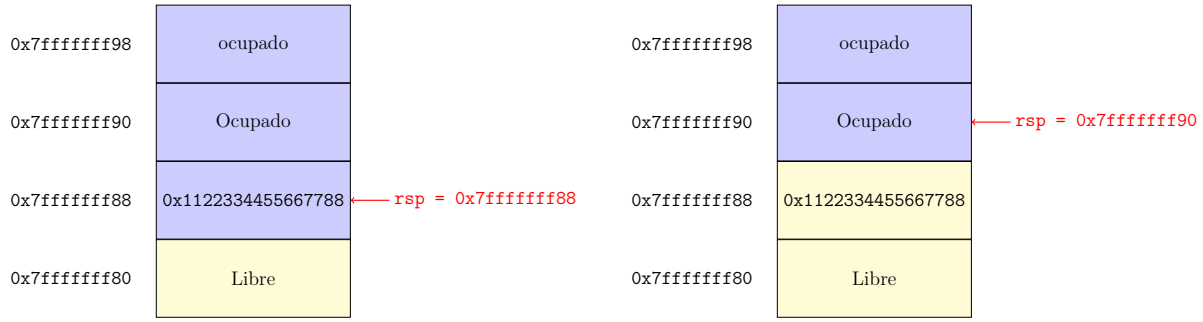


Figura 11: Diagrama de la pila antes y después de ejecutar la instrucción `popq %rax`.

7.4. Alineación de la pila en x86-64

En la arquitectura x86-64, la pila debe mantenerse alineada a 16 bytes antes de realizar una llamada a función (instrucción `call`). Esta restricción forma parte de la convención de llamadas (ABI) y es necesaria para garantizar el correcto funcionamiento de ciertas instrucciones, en particular aquellas que operan con registros SIMD (como las de SSE/AVX), las cuales requieren direcciones de memoria alineadas.

Observación

Decir que la pila (o, más precisamente, el tope de la pila) está alineada a una cierta cantidad de bytes significa que la dirección contenida en el registro `%rsp` es múltiplo de dicha cantidad.

Por ejemplo, la dirección `0x7fffffffebb8` está alineada a 8 bytes (ya que es múltiplo de 8), mientras que `0x7fffffffebb0` está alineada a 16 bytes.

En particular, muchas convenciones de llamada en x86-64 requieren que, en ciertos puntos del programa (por ejemplo, antes de una llamada a función), el valor de `%rsp` sea múltiplo de 16.

Formalmente, una dirección está alineada a 16 bytes si `dirección mod 16 = 0`. En representación hexadecimal, esto equivale a que el último dígito sea `0`.

Dado que la instrucción `call` apila la dirección de retorno (8 bytes), el programador debe asegurarse de que la pila esté correctamente alineada antes de la llamada. En caso contrario, el programa puede comportarse de manera incorrecta o incluso producir fallos en tiempo de ejecución. Por este motivo, es habitual ajustar explícitamente el valor de `%rsp` (por ejemplo, reservando espacio en la pila en múltiplos de 16 bytes) al comienzo de una función, como veremos en la próxima sección.

8. Funciones

Una parte fundamental del código estructurado son los procedimientos y funciones. Estas construcciones permiten dividir un programa en partes más pequeñas, comprensibles y reutilizables. En particular, una función en un programa puede compararse con una función matemática: recibe ciertos argumentos y devuelve un valor de retorno.

Así vemos por ejemplo que la función en C:

```
long int sum(long int a, long int b);
```

toma dos enteros largos como argumentos y devuelve otro entero largo.

Desde el punto de vista del procesador, una llamada a función es similar a un salto, ya que implica modificar el flujo del programa para ejecutar el código de la función llamada. La diferencia principal es que, dado que el código se ejecuta secuencialmente, tras la llamada a función el control debe regresar al punto siguiente a dicha llamada para continuar con la ejecución normal del programa. Veamos esto en un ejemplo en C:

```
...
i++;
printf("%d\n",i);
i--;
printf("%d\n",i);
...
```

Aquí vemos tres instrucciones. La segunda es una llamada a la función `printf` con dos argumentos, la cadena de formato `"%d\n"` y el valor de `i`. Luego de finalizada la impresión por parte de `printf` el código debe seguir con el decremento de `i`. Pero, ¿cómo sabe `printf` a qué instrucción debe retornar, considerando que puede ser llamada desde múltiples lugares distintos? La respuesta es que no lo sabe: es el código que realiza la llamada quien debe indicarle adónde continuar la ejecución una vez finalizada. Esta dirección se conoce como **dirección de retorno**.

Para realizar llamadas a función, la arquitectura x86-64 provee dos instrucciones: la instrucción `call`, que transfiere el control a la función llamada, y `ret`, que devuelve el control a la función llamante.

8.1. Llamada a funciones

La instrucción `call` realiza la llamada a la función indicada como operando (la etiqueta que la define) guardando en la pila la dirección de retorno (la dirección de la próxima instrucción al `call`):

```
call f
```

Así, la instrucción `call f` sería equivalente a

```
    pushq $direccion_de_retorno
    jmp f
direccion_de_retorno:
    .....
```

donde la etiqueta `dirección_de_retorno` indica la dirección de la próxima instrucción a la llamada a función.

8.2. Retorno de funciones

La instrucción `ret` retorna de una función sacando la dirección de retorno que se encuentra en el tope de la pila (puesto allí por la instrucción `call`) y salta a ese lugar. Así, la instrucción `ret` equivale a

```
popq %r11
jmp *%r11
```

Sin embargo, `ret` no modifica registros de propósito general; únicamente actualiza `%rip` con la dirección de retorno almacenada en la pila e incrementa `%rsp` en 8 bytes en modo x86-64. El uso de `%r11` en este código es solo ilustrativo para mostrar el comportamiento de la instrucción mediante un código equivalente. En este ejemplo, el asterisco es necesario debido a la sintaxis utilizada.

8.3. Convención de llamada en x86-64

Al invocar una función en un programa, es necesario acordar cómo se pasan los argumentos, cómo se devuelve el resultado y quién se encarga de preservar el contenido de los registros. Estas reglas forman lo que se conoce como **convención de llamada**.

En la arquitectura x86-64, se sigue una convención específica que define, entre otras cosas, qué registros se utilizan para pasar los primeros argumentos, cómo se debe alinear la pila, y qué registros deben ser preservados por la función llamada. Esta convención garantiza que funciones escritas en distintos lenguajes o por distintos compiladores puedan interoperar correctamente.

En una llamada a función, se distinguen dos partes: el **caller** (llamador) y el **callee** (llamada).

- El **caller** es el código que invoca la función.
- El **callee** es el código de la función que se ejecuta como resultado de esa llamada.

Ambos tienen responsabilidades específicas dentro de la convención de llamada: por ejemplo, quién debe preservar ciertos registros, cómo se pasan los argumentos y cómo se retorna el control al final de la ejecución.

Convención de llamada para x86-64 en Linux

- Los seis primeros argumentos a la función son pasados por registro en el siguiente orden: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9` (cuando los argumentos son valores enteros o direcciones de memoria).
- Si los argumentos son valores de punto flotantes pueden utilizarse hasta 8 de los registros `xmm` en el siguiente orden: `%xmm0`, `%xmm1`, `%xmm2`, `%xmm3`, `%xmm4`, `%xmm5`, `%xmm6` y `%xmm7`.
- Parámetros grandes mayores a 64 bits, por ejemplo estructuras pasadas por valor, se pasan utilizando la pila.

- Cuando la función toma como argumento una mezcla de valores enteros y flotantes `rdi` será el primer valor entero, `xmm0` el primer valor flotante, y así sucesivamente. Así, en la función `void f(int, double, int, double)` los argumentos irán en `rdi,xmm0,rsi,xmm1`.
- Si hubiera más argumentos de los que se pueden pasar por registros, éstos son pasados a la función utilizando la pila.
- Los argumentos que van a la pila se colocan de derecha a izquierda, es decir, el último argumento de la lista se empuja primero.
- Los valores escalares (como enteros y punteros) se devuelven en el registro `rax`.
- Los valores en punto flotante (`float` y `double`) se devuelven en `xmm0`.
- El llamado **se compromete** a preservar el valor de los registros `%rbx`, `%rbp`, `%rsp`, y `%r12` a `%r15`. Esto no quiere decir que no los pueda usar sino que al retornar deben tener el mismo valor que al comenzar la función. La función podría guardarlos temporalmente en memoria o pila y restaurarlos antes de retornar. Estos registros se conocen como *callee saved* ya que es responsabilidad del llamado preservarlos.
- Los otros registros (incluso los utilizados para pasar los argumentos) pueden ser modificados libremente por la función sin necesidad de restaurar sus valores. Si el llamante desea preservar sus valores es responsabilidad de él, por lo cual estos registros se conocen como *caller saved*. En la Tabla 2 se puede observar el rol de los registros en la llamada a función.
- El bit `DF` de `rflags` está inicialmente apagado (esto incrementará los punteros en instrucciones de manejo de cadena) y debe ser apagado al finalizar la función (y antes de llamar a otra función).
- El puntero de la pila (`rsp`) debe estar alineado a 16 bytes antes de realizar una llamada a función. Esto asegura que cualquier instrucción de operaciones con registros SIMD (usados para procesar datos en paralelo) funcione correctamente.
- Como `%rbp` y `%rsp` son preservados durante una llamada a función, el estado de la pila del llamante se mantiene.
- Si la función llamada es variádica, entonces el número de argumentos de punto flotante que se pasan a la función en los registros vectoriales debe ser proporcionado por la función llamante en el subregistro `al`.

En la Tabla 2 se presentan los registros de propósito general de la arquitectura x86-64 y su uso según la convención de llamadas System V AMD64. Asimismo, se indica cuáles deben ser preservados durante una llamada a función.

Según esta convención, los registros se clasifican en *caller-saved* y *callee-saved*. Los primeros pueden ser modificados por la función llamada, por lo que la función llamadora debe guardar sus valores si necesita conservarlos. En cambio, los registros *callee-saved* deben ser preservados por la función llamada, que tiene la responsabilidad de restaurar

sus valores originales antes de retornar.

Tabla 2: Registros de propósito general de x86-64 y su uso según la convención de llamadas System V AMD64.

Registro	Uso habitual	Convención	¿Debe preservarse?
`rax`	Valor de retorno	*Caller saved*	No
`rbx`	Propósito general	*Callee saved*	Sí
`rcx`	4º argumento	*Caller saved*	No
`rdx`	3º argumento	*Caller saved*	No
`rsi`	2º argumento	*Caller saved*	No
`rdi`	1º argumento	*Caller saved*	No
`rbp`	Puntero base (opcional)	*Callee saved*	Sí*
`rsp`	Puntero de pila	*Callee saved*	Sí
`r8`	5º argumento	*Caller saved*	No
`r9`	6º argumento	*Caller saved*	No
`r10`	Temporal	*Caller saved*	No
`r11`	Temporal	*Caller saved*	No
`r12`	Propósito general	*Callee saved*	Sí
`r13`	Propósito general	*Callee saved*	Sí
`r14`	Propósito general	*Callee saved*	Sí
`r15`	Propósito general	*Callee saved*	Sí

*El registro `rsp` debe recuperar su valor correspondiente al retornar de la función, teniendo en cuenta el efecto de las instrucciones `call` y `ret`.

8.4. Prólogo y epílogo de una función

El **prólogo** de una función es la secuencia de instrucciones ejecutada al comienzo de esta para establecer su marco de pila, reservar espacio para variables locales y, cuando corresponda, preservar los registros que deben conservar su valor. El prólogo clásico completo tiene la siguiente forma:

```
pushq %rbp          # Guarda el rbp de la función llamadora
movq  %rsp, %rbp    # Establece el nuevo marco de pila
subq  $N, %rsp      # Reserva N bytes en la pila
```

donde `N` es un entero no negativo que determina la cantidad de bytes reservados para las variables locales y otros datos temporales, teniendo en cuenta los requisitos de alineación de la pila.

En la secuencia de instrucciones anterior, conviene distinguir dos operaciones:

1. **Establecimiento del marco de pila:** las instrucciones `pushq %rbp` y `movq %rsp, %rbp` permiten utilizar

2. **Reserva de espacio local:** la instrucción `subq $N, %rsp` desplaza el puntero de pila para disponer de N bytes adicionales.

La tercera instrucción no es obligatoria. Puede omitirse si la función no necesita reservar espacio en la pila.

El **epílogo** de una función es la secuencia de instrucciones ejecutada al finalizar esta, cuya finalidad es liberar el espacio reservado en la pila, restaurar los registros que deban conservar su valor y retornar el control a la función llamadora.

```
movq %rbp, %rsp # Libera el espacio reservado en la pila
popq %rbp       # Restaura el valor anterior de rbp
ret             # Retorna a la función llamadora
```

Las dos primeras instrucciones pueden reemplazarse por la instrucción `leave`:

```
leave
ret
```

Observación

Al igual que el prólogo, el epílogo no tiene una estructura obligatoria. Su implementación depende de cómo se haya organizado el marco de pila y de los registros que la función deba restaurar.

Ejemplo

En este ejemplo el prólogo guarda el valor de `rbp` del llamante, establece `rbp` como base del nuevo marco de activación y reserva espacio en la pila para las variables locales. El epílogo realiza las operaciones inversas en orden contrario: libera el espacio reservado y restaura el valor original de `rbp`. Al finalizar el epílogo, `rsp` apunta a la dirección de retorno, de modo que `ret` puede extraerla de la pila y devolver el control al llamante, que encuentra `rbp` y `rsp` tal como estaban antes de la llamada.

```
funcion:
    # Prólogo
    pushq %rbp           # Guarda en la pila el rbp del llamante
    movq  %rsp, %rbp     # rbp apunta a la base del nuevo marco de
       activación
    subq  $16, %rsp      # Reserva espacio suficiente para almacenar las
       variables locales y mantener la alineación de la pila

    # Cuerpo de la función
    movl  $5, -4(%rbp)   # Inicializa una variable local de 4 bytes con
       el valor 5
    movl  -4(%rbp), %eax # Copia la variable local en eax (valor de
       retorno)

    # Epílogo
    movq  %rbp, %rsp     # Libera las variables locales: rsp apunta al
       rbp guardado
    popq  %rbp           # Restaura el rbp del llamante
    ret                  # Retorna al punto de llamada
```

8.5. Ejemplo completo de llamada a función

Como se mencionó en las secciones anteriores, una llamada a función en la arquitectura x86-64, implica una serie de convenciones y mecanismos que permiten el paso de argumentos, la transferencia del control de ejecución y el posterior retorno al punto de invocación. Estas llamadas siguen el estándar de llamada conocido como System V AMD64 ABI en sistemas tipo Unix.

En este esquema, los primeros seis argumentos enteros de una función se pasan a través de registros —en orden: `rdi`, `rsi`, `rdx`, `rcx`, `r8` y `r9`—, mientras que los restantes se colocan en la pila (stack). Los primeros ocho argumentos de tipo flotante, por su parte, se pasan en los registros `xmm0` a `xmm7`, y los restantes se colocan en la pila.

Para transferir el control, se utiliza la instrucción `call`, que guarda la dirección de retorno en la pila y salta a la dirección de la función. Al finalizar, la función ejecuta `ret`, que recupera dicha dirección y continúa la ejecución del programa. Además, deben respetarse convenciones sobre qué registros deben preservarse entre llamadas (`rbx`, `rbp`, `rsp`, `r12-r15`) y cuáles pueden ser modificados libremente (`rax`, `rcx`, `rdx`, `rdi`, `rsi` y `r8-r11`).

La Figura 12 ilustra la función `suma`, invocada por `main`, que recibe tres números enteros como argumentos. Esta función los suma y devuelve el resultado a `main`, la cual a su vez invoca a la función `printf` para mostrar el resultado por pantalla. En la figura se muestra, a la izquierda, el código en lenguaje C, y a la derecha, un código equivalente en Assembler x86-64. Obsérvese que cada función se define en un archivo distinto. Si bien esto no es un requisito, es una práctica altamente recomendable.

8.6. Zona roja en x86-64

En la Figura 9 se muestra un área de la memoria denominada ***"zona roja"***. La arquitectura x86-64 especifica que los programas pueden utilizar los 128 bytes situados por debajo del puntero de la pila actual (es decir, en direcciones más bajas que el valor de `rsp`). Esta área se considera reservada y no es modificada por señales o manejadores de interrupciones.

Debido a esto, las funciones pueden aprovechar la zona roja para almacenar datos temporales que no sean necesarios en llamadas a otras funciones. En particular, las funciones de "hoja" (*leaf functions*[6]) pueden usar esta área directamente en lugar de ajustar el puntero de la pila en el prólogo y el epílogo.

La zona roja es una optimización. El código puede asumir que los 128 bytes por debajo de `rsp` no serán alterados por señales o manejadores de interrupciones, lo que permite usarlos para datos temporales sin necesidad de mover explícitamente el puntero de la pila. Sin embargo, dado que esta área es utilizada por llamadas a función, su principal utilidad se encuentra en funciones que no invocan a otras funciones.

Ejemplo

En este ejemplo, presentaremos una función llamada `suma`, que recibe dos números enteros largos como argumentos. Estos números son sumados, y la función retorna el resultado de la suma. A continuación, se muestra cómo invocar la función `suma` con los argumentos de 8 bytes 40 y 45:

[6]Una *leaf function* (función hoja) es una función que no realiza llamadas a otras funciones durante su ejecución.

suma.s

```
.text
.global suma
suma:
    # Prólogo
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp
    # Cuerpo de la función
    movl %edi, -4(%rbp)
    movl %esi, -8(%rbp)
    movl %edx, -12(%rbp)
    movl -4(%rbp), %edx
    movl -8(%rbp), %eax
    addl %edx, %eax
    movl -12(%rbp), %edx
    addl %edx, %eax
    # Epílogo
    movq %rbp, %rsp
    popq %rbp
    ret
```

suma.c

```
int suma (int a, int b, int c){
    return a+b+c;
}
```

main.c

```
# include<stdio.h>
int main (){
    int a=suma(10, 20, 30);
    printf ("Suma: %d\n", a);
    return 0;
}
```

main.s

```
.data
msg: .asciz "Suma: %d\n"

.text
.global main
main:
   # Prólogo
   pushq %rbp
   movq %rsp, %rbp
   subq $16, %rsp
   # Construcción de argumentos
   movl $10, %edi     # arg. 1
   movl $20, %esi     # arg. 2
   movl $30, %edx     # arg. 3
   # Llamada a la función suma
   call suma
   movl %eax, -4(%rbp)
   leaq msg, %rdi
   movl -4(%rbp), %esi
   xorl %eax, %eax
   # Llamada a printf
   call printf
   # Epílogo
   movq %rbp, %rsp
   popq %rbp
   ret
```

Figura 12: Ejemplo de llamada a función.

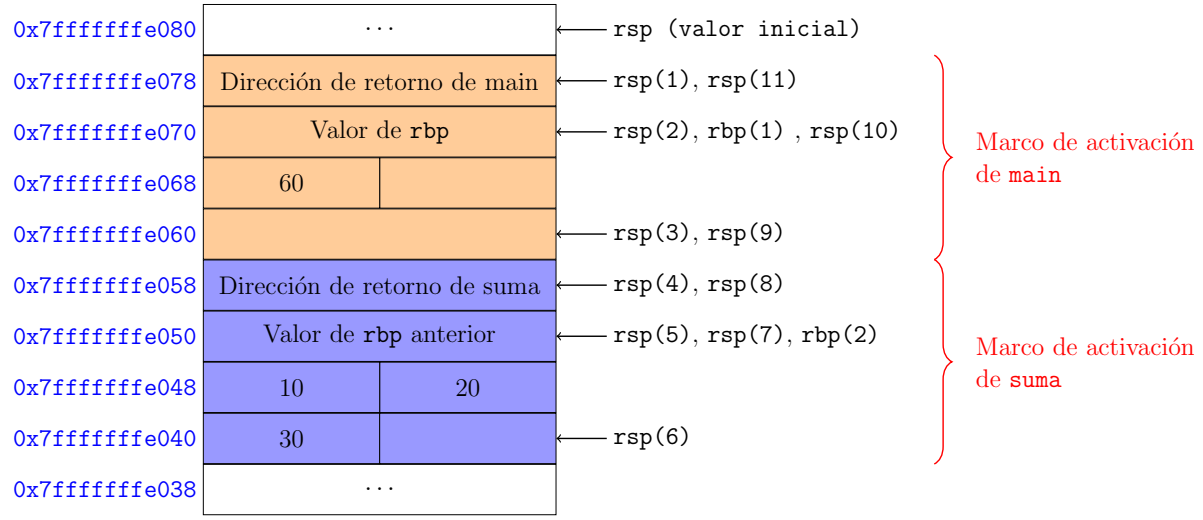


Figura 13: Diagrama de pila correspondiente al ejemplo de la Figura 12.

```
.data
i: .space 8

.text
.global main
main:
    # Prólogo
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp          # Se reserva espacio en la pila

    # Cuerpo de la función main
    movq $40, -8(%rbp)      # Variable local en el marco de activación
       de main
    movq $45, -16(%rbp)     # Variable local en el marco de activación
       de main
    movq -8(%rbp), %rdi     # Primer argumento en rdi
    movq -16(%rbp), %rsi    # Segundo argumento en rsi
    call suma               # Guarda la dirección de retorno en pila y
       salta a "suma"
    movq %rax, i            # Aquí rax contiene el resultado (85)

    # Epílogo
    movq %rbp, %rsp
    popq %rbp
    ret

suma:
    # Prólogo
    pushq %rbp              # Preserva el registro rbp
    movq %rsp, %rbp         # rbp "apunta" al inicio de la pila
```

```
    # Cuerpo de la función suma
    movq %rdi, -8(%rbp)      # Variable local en zona roja
    movq %rsi, -16(%rbp)     # Variable local en zona roja
    movq -8(%rbp), %rax
    addq -16(%rbp), %rax     # Aquí el resultado ya está en rax
    movq %rax, -24(%rbp)     # Variable local en zona roja

    # Epílogo
    movq %rbp, %rsp      # rsp "apunta" nuevamente al inicio de la pila
    popq %rbp            # Se restaura el registro rbp
    ret                  # Retorna de la función "suma" a "main"
```

En la Figura 14, se presenta un esquema de la pila que muestra cómo se almacenan las direcciones de retorno, cómo se preserva el registro `rbp` y cómo se almacenan las variables locales. También se puede observar la evolución del valor del registro puntero de pila `rsp`.

En este diagrama, vemos que las variables locales 40 y 45 se almacenan en el marco de activación de la función `main`, espacio que se reserva mediante la instrucción `subq $16, %rsp`. Sin embargo, en la función `suma`, las variables locales no requieren espacio reservado. ¿Por qué? Porque la "zona roja"garantiza que los 128 bytes por debajo del valor apuntado por el registro `rsp` no se modificarán por señales o interrupciones.

Es importante destacar que esto es posible porque `suma` es una función "hoja". En cambio, si dentro de `suma` se llamara a otra función, lo anterior ya no sería válido, y sería necesario reservar espacio adicional mediante un nuevo decremento del valor del registro `rsp`.

9. Instrucciones de punto flotante

La arquitectura x86-64 soporta instrucciones para operar con datos de punto flotante siguiendo el estándar IEEE 754, tanto en precisión simple como en doble precisión. Estas operaciones se ejecutan mediante una extensión específica de la arquitectura, conceptualmente separada de la ALU, conocida como SSE (Streaming SIMD Extensions).

Para estas instrucciones se utilizan registros y operaciones específicas. La arquitectura dispone de 16 registros de 128 bits (16 bytes), denominados `xmm0` a `xmm15` (ver Figura 3). Cada registro puede almacenar un único valor de punto flotante en precisión simple o doble, caso en el cual el dato se considera escalar y sólo se usa una parte del registro; o bien múltiples valores de igual tamaño en formato empaquetado (packed). Por ejemplo, en `xmm0` pueden almacenarse cuatro valores de precisión simple o dieciséis enteros de 1 byte. El formato empaquetado permite que ciertas instrucciones apliquen la misma operación sobre varios datos simultáneamente (SIMD: *Single Instruction, Multiple Data*).

El formato general de las instrucciones de punto flotante es el siguiente:

```
operadorS <operando origen>, <operando destino>
```

donde el sufijo `S` contiene en general dos letras:

- La primera letra indica el formato: `s` ("scalar") o `p` ("packed").

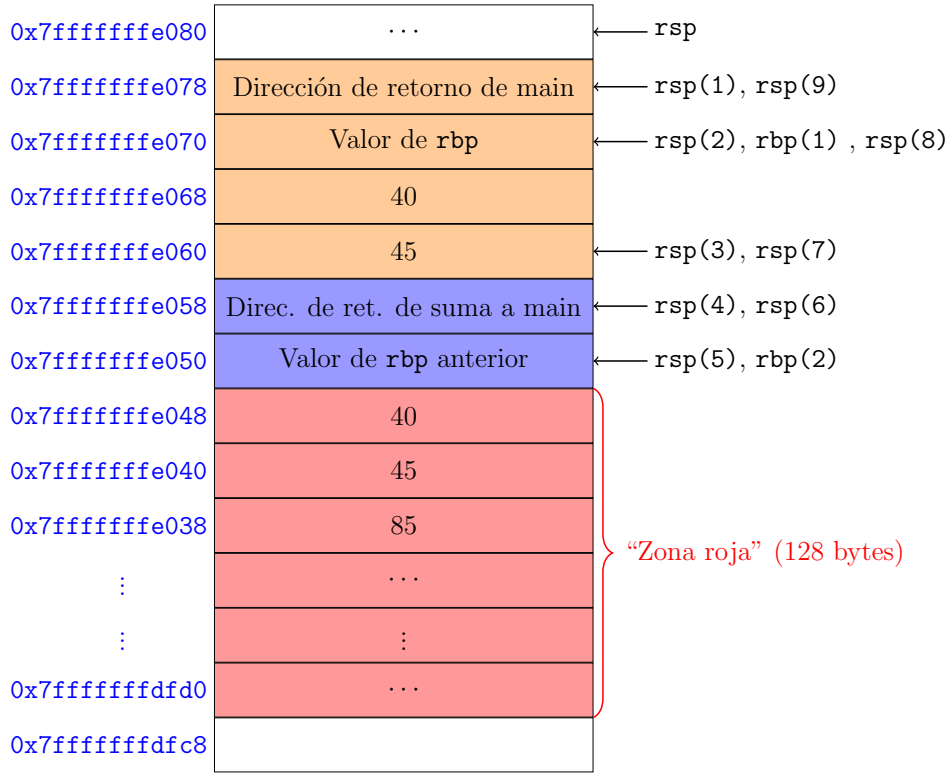


Figura 14: Ejemplo de esquema de pila con utilización de la "zona roja".

- La segunda letra indica el tipo de dato: `s` ("single"), `d` ("double") o `i` ("integer").
- Cuando el dato es entero, puede añadirse un sufijo de tamaño. Por ejemplo, `q` indica que el entero ocupa un *quadword* (8 bytes).

9.1. Copias de datos

Al igual que con los registros de propósito general, existen instrucciones para copia de datos. Para los registros de punto flotante existen las instrucciones `movss` y `movsd` que copian un dato de precisión simple (*float*) y doble precisión (*double*), respectivamente, de un registro `xmm` a otro o desde/hacia la memoria. En la Tabla 3 se recopilan las instrucciones de copia de datos escalares.

Ejemplos

```
movss %xmm0, %xmm1  # copiar valor escalar en simple precisión (32 bits)
    de xmm0 a xmm1
movsd %xmm0, %xmm1  # copiar valor escalar en doble precisión (64 bits)
   de xmm0 a xmm1
movss a, %xmm0    # cargar en xmm0 un valor escalar en simple precisión
   desde la dirección 'a'
movsd a, %xmm0    # cargar en xmm0 un valor escalar en doble precisión
   desde la dirección 'a'
movss %xmm1, b    # almacenar en 'b' un valor escalar en simple precisión
    tomado de xmm1
```

Tabla 3: Instrucciones de movimiento de datos en punto flotante [5].

Instrucción	S	D	Descripción
`movss S, D`	M32 / X	X / M32	Copia un valor de precisión simple (32 bits)
`movsd S, D`	M64 / X	X / M64	Copia un valor de precisión doble (64 bits)

X: registro XMM (`xmm0-xmm15`)
M32: operando en memoria de 32 bits
M64: operando en memoria de 64 bits

```
movsd %xmm2, b    # almacenar en 'b' un valor escalar en doble precisión
   tomado de xmm2
movss (%rdi), %xmm0  # carga 32 bits (float) desde memoria a %xmm0
movss %xmm0, (%rdi)  # almacena 32 bits (float) desde %xmm0 a memoria
```

9.2. Conversiones de datos

A su vez existen múltiples instrucciones para convertir entre enteros y datos de punto flotante. Por ejemplo, `cvtsi2sdq` se interpreta de la siguiente manera:

- `cvt`: convert (convertir)
- `si`: scalar integer (entero escalar con signo)
- `2`: to ("a")
- `sd`: scalar double (flotante escalar de doble precisión)
- `q`: quadword (el entero de origen es de 64 bits)

Por lo tanto, esta instrucción convierte un entero con signo almacenado en un *quadword* a un valor *double* en formato escalar. En la Tabla 4 se recopilan las instrucciones de conversión.

Ejemplo

```
movq $10, %rax
cvtsi2sdq %rax, %xmm0   # Convierte el entero con signo en rax a double
   en xmm0
```

Ejemplo

Veamos el procedimiento para inicializar una variable de tipo double (en el registro `xmm0`) con el valor 1.0:

```
movq $1, %rax            # Copiar un 1 entero a rax
cvtsi2sdq %rax, %xmm0    # Convierte el 1 de rax al double 1.0 en xmm0
movsd %xmm0, %xmm1       # Copia el valor escalar 1.0 en doble precisión
   del registro xmm0 al registro xmm1
```

Tabla 4: Instrucciones de conversiones de datos para punto flotante [5].

`cvtss2sd S, D`	M32/X	X	Convertir de simple a doble precisión
`cvtsd2ss S, D`	M64/X	X	Convertir de doble a simple precisión
`cvtsi2ss S, D`	M32/R32	X	Convertir entero a simple precisión
`cvtsi2sd S, D`	M32/R32	X	Convertir entero a doble precisión
`cvtsi2ssq S, D`	M64/R64	X	Convertir quadword entero a simple precisión
`cvtsi2sdq S, D`	M64/R64	X	Convertir quadword entero a doble precisión
`cvttss2si S, D`	X/M32	R32	Convertir (truncado) simple precisión a entero
`cvttsd2si S, D`	X/M64	R32	Convertir (truncado) doble precisión a entero
`cvttss2siq S, D`	X/M32	R64	Convertir (truncado) simple precisión a quadword entero
`cvttsd2siq S, D`	X/M64	R64	Convertir (truncado) doble precisión a quadword entero

X: Registro XMM (e.g., `%xmm3`)
R32: Registro de propósito general de 32 bits (e.g., `%eax`)
R64: Registro de propósito general de 64 bits (e.g., `%rax`)
M32: 32 bits de memoria
M64: 64 bits de memoria

9.3. Operaciones en punto flotante

Las operaciones entre valores de punto flotante siempre involucran dos operandos, el operando fuente puede ser tanto un registro `xmm` como un valor almacenado en memoria. El destino debe ser un registro `xmm`. La Tabla 5 resume las operaciones más utilizadas para simple y doble precisión.

Ejemplo

Vimos en la Sección 8 que la convención de llamada indica que los argumentos de punto flotante se pasan por los registros `xmm` y el valor de retorno se deja en el registro `xmm0`. Sabiendo esto podemos escribir el siguiente código que es equivalente a la función en C:

Tabla 5: Instrucciones aritméticas en punto flotante[5].

Simple precisión	Doble precisión	Efecto	Descripción
`addss S, D`	`addsd S, D`	$D \leftarrow D + S$	Suma en punto flotante
`subss S, D`	`subsd S, D`	$D \leftarrow D - S$	Resta en punto flotante
`mulss S, D`	`mulsd D, D`	$D \leftarrow D \times S$	Multiplicación en punto flotante
`divss S, D`	`divsd S, D`	$D \leftarrow D \div S$	División en punto flotante
`maxss S, D`	`maxsd S, D`	$D \leftarrow \text{máx}(D,\ S)$	Máximo en punto flotante
`minss S, D`	`minsd S, D`	$D \leftarrow \text{mín}(D,\ S)$	Mínimo en punto flotante
`sqrtss S, D`	`sqrtsd S, D`	$D \leftarrow \sqrt{S}$	Raíz cuadrada en punto flotante

```
double convert(double t){
    return t*1.8 + 32;
}

.global convert
convert:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp

    # en xmm0 viene t por convención de llamada
    movq $0x3ffcccccccccccd, %rax  # 1.8 en IEEE754 doble precisión. El
       valor inmediato es la representacion de 1.8 según IEEE 754
    movq %rax, -8(%rsp)
    movsd -8(%rsp), %xmm1     # Carga el valor 1.8 en xmm1
    movq $32, %rax
    cvtsi2sdq %rax, %xmm2
    # Carga el valor 32.0 convirtiendo el valor entero 32 de rax a xmm2
    mulsd %xmm1, %xmm0       # xmm0=xmm0*xmm1 => xmm0=t*1.8
    addsd %xmm2, %xmm0       # xmmo=xmm0+xmm2 => xmmo=t*1.8+32
    # como el valor de retorno se escribe en xmm0 hemos terminado

    movq %rbp, %rsp
    popq %rbp
    ret
```

9.4. Comparaciones en punto flotante

Al igual que con los valores enteros la arquitectura ofrece comparaciones de valores de punto flotante. Las instrucciones de comparación comparan dos valores (haciendo una resta virtual) y prenden las banderas correspondientes en el registro `rflags`. La comparación se comporta como una comparación de datos unsigned (i.e.: conviene utilizar `jae` para saltar por mayor o igual). Además, si los valores son incomparables (alguno es NaN) se prende la bandera `PF (Parity Flag)`.

Las instrucciones de comparación en punto flotante se muestran en la Tabla 6. Estas instrucciones permiten comparar valores almacenados en registros de tipo `xmm`, produciendo resultados que afectan las banderas del registro `eflags`. Entre ellas, `ucomisd` y `comisd` se utilizan para comparar valores de doble precisión, mientras que `ucomiss` y `comiss` realizan la comparación en precisión simple. Las instrucciones `ucomis*` (*Unordered Compare Scalar*) y `comis*` son casi equivalentes. Sin embargo, `comis*` puede generar una excepción de punto flotante si alguno de los operandos es NaN, mientras que `ucomis*` no lo hace: solo actualiza las banderas, estableciendo PF = 1 para indicar una comparación no ordenada debido a un NaN. Por ello, `ucomis*` se considera la versión "segura" para comparar valores que podrían incluir NaN.

Las instrucciones de comparación de punto flotante establecen tres banderas de condición: la bandera cero `ZF`, la bandera de acarreo `CF` y la bandera de paridad `PF`. Las banderas de condición se establecen de la siguiente manera:

Tabla 6: Instrucciones de comparación en punto flotante[5].

Instrucción	Basada en	Descripción
`ucomiss S2, S1`	$S_1 - S_2$	Comparación de precisión simple ("desordenado")
`ucomisd S2, S1`	$S_1 - S_2$	Comparación de precisión doble ("desordenado")
`comisd S2, S1`	$S_1 - S_2$	Comparación de precisión doble ("ordenado")
`comisd S2, S1`	$S_1 - S_2$	Comparación de precisión doble ("ordenado")

Orden	CF	ZF	PF
"desordenado"	1	1	1
$S_1 < S_2$	1	0	0
$S_1 = S_2$	0	1	0
$S_1 > S_2$	0	0	0

El caso "desordenado" ocurre cuando cualquiera de los operandos es NaN. Esto se puede detectar con la bandera de paridad. Comúnmente, la instrucción `jp` (para "saltar en paridad") se usa para saltar condicionalmente cuando la comparación en punto flotante arroja un resultado desordenado. Por otra parte, `ZF` se establece cuando los dos operandos son iguales y `CF` cuando $S_1 < S_2$. Las instrucciones `ja` y `jb` se usan para saltar condicionalmente en estos casos.

Ejemplo

```
.data
a: .double 50.0
b: .double 30.0
msg_gt: .asciz "a > b\n"
msg_lt: .asciz "a < b\n"
msg_eq: .asciz "a = b\n"

.text
.global main
main:
    pushq %rbp
    movq %rsp, %rbp

    # cargar valores en registros XMM
    movsd a, %xmm0    # %xmm0 = a
    movsd b, %xmm1    # %xmm1 = b

    # comparar (no lanza excepción si hay NaN)
    ucomisd %xmm1, %xmm0   # compara %xmm0 vs %xmm1 (a ? b)

    # saltos según flags
    ja   mayor             # a > b
```

```
    jb   menor              # a < b
    je   iguales            # a == b

mayor:
    leaq msg_gt, %rdi
    xorl %eax, %eax
    call printf
    jmp fin

menor:
    leaq msg_lt, %rdi
    xorl %eax, %eax
    call printf
    jmp fin

iguales:
    leaq msg_eq, %rdi
    xorl %eax, %eax
    call printf

fin:
    movq %rbp, %rsp
    popq %rbp
    ret
```

9.5. Convención de llamada en punto flotante

Como hemos visto, en la arquitectura x86-64, la convención de llamada a funciones define cómo se pasan los parámetros y cómo se gestionan los registros durante las invocaciones. Aquí vemos algunas particularidades para el caso de funciones con argumentos de punto flotante:

- Para las funciones que utilizan números en punto flotante, los primeros ocho argumentos de tipo flotante se pasan a través de los registros `xmm0` a `xmm7`.
- Si hay más argumentos flotantes, se pasan en la pila.
- Una función que devuelve un valor en punto flotante lo hace en el registro `xmm0`.
- Todos los registros `xmm` son preservados por el llamador. Por lo tanto, la función llamada puede sobrescribir cualquiera de estos registros sin guardarlos previamente.

Ejemplo

En este ejemplo vemos cómo llamar a la función `printf` para imprimir un entero y un flotante doble precisión:

```
.data
str: .asciz "%d %f\n"
```

```
a: .long 45
f: .double 3.14

.text
.global main
main:
    pushq %rbp          # Alineamos la pila a 16 bytes
    leaq str, %rdi      # Le pasamos la direc. de la cadena de formato
    movl a, %esi        # Le pasamos el segundo argumento
    movsd f, %xmm0      # le pasamos el tercer argumento
    movb $1, %al        # Cantidad de argumentos de punto flotante
    call printf         # Llamamos a la función printf
    popq %rbp           # Desapilamos para preservar el valor de rsp
    xorl %eax, %eax     # Retornamos cero
    ret
```

Notar que para poder utilizar la función `printf` la convención de llamada AMD64 System V ABI[11] requiere varias cuestiones:

- Justo antes de una instrucción `call` la pila debe estar alineada al menos con 16 bytes.
- La convención de llamada también requiere que que el subregistro `al` contenga el número de registros vectoriales utilizados para una función de argumento variable. `printf` es una función de argumento variable, por lo que es necesario configurar `al`. En este ejemplo la cantidad de argumentos de punto flotante es uno.
- `rdi` debe ser un puntero a la cadena de formato.
- También debemos terminar la cadena de formato con NULL. Por lo tanto, en lugar de utilizar `.ascii`, utilizar `.asciz`.

9.6. Almacenamiento de registros XMM en la pila

En la arquitectura x86-64 no existe una instrucción `push` que opere directamente sobre registros `xmm`; por lo tanto, una instrucción como:

```
pushq %xmm0     ✗
```

no es válida.

Sin embargo, es posible emular un `push` almacenando el contenido del registro `xmm` en la pila mediante memoria. Si solo interesa la parte baja (64 bits, por ejemplo un *double* o un entero de 64 bits), puede hacerse:

```
subq $8, %rsp
movq %xmm0, (%rsp)
```

El proceso inverso permite recuperar el valor:

```
movq (%rsp), %xmm0
addq $8, %rsp
```

Si se desea guardar el registro completo (128 bits), puede utilizarse:

```
subq $16, %rsp        # reservar 16 bytes (128 bits)
movdqu %xmm0, (%rsp) # almacenar xmm0 en la pila
```

La instrucción `movdqu` transfiere 128 bits entre memoria y un registro `xmm`. Admite direcciones no alineadas a 16 bytes, por lo que es adecuada cuando no se garantiza alineación.

10. Instrucciones SIMD

Los programas de procesamiento de señales multimedia (audio, imágenes, video, etc.) suelen requerir la repetición de una misma operación sobre una gran cantidad de datos, como aplicar un cálculo específico a cada píxel. Para este tipo de tareas, las arquitecturas modernas incluyen las instrucciones conocidas como Streaming SIMD Extensions (SSE), donde SIMD significa Single Instruction, Multiple Data. Estas instrucciones permiten aplicar una única operación de manera simultánea sobre múltiples datos. La mayoría de las instrucciones aritméticas SIMD ejecutan operaciones paralelas con vectores, también denominadas operaciones "empaquetadas" (*packed*, en inglés), ya que utilizan operandos que contienen varios elementos procesados en paralelo.

Recordemos que los registros `xmm0-xmm15` son de 128 bits (ver Figura 3) por lo cual pueden alojar 4 valores de precisión simple o 2 de precisión doble o también 16 bytes, 8 words, 4 enteros de 32 bits o 2 de 64 bits.

Hay varios tipos de instrucciones *packed*:

- Instrucciones de transferencia de datos.
- Instrucciones de conversión.
- Instrucciones aritméticas.
- Instrucciones lógicas.

10.1. Instrucciones para operaciones con flotantes empaquetados

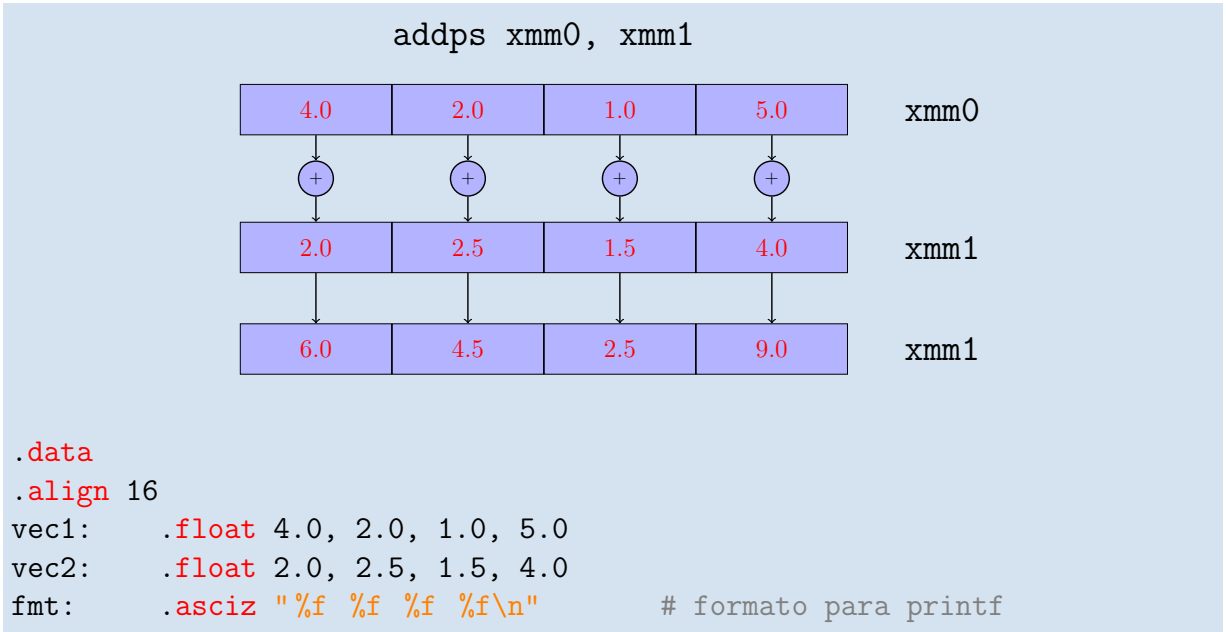
La Tabla 7 muestra algunas instrucciones para operaciones con flotantes empaquetados. Sin embargo, las extensiones SSE incluyen muchas más instrucciones. En este apunte, solo se presenta una introducción. Un listado completo puede consultarse en [8] o [9].

Ejemplo

La siguiente figura y código ilustra cómo sumar dos vectores de cuatro elementos de tipo `float` en una sola operación:

Tabla 7: Instrucciones SIMD para operaciones con flotantes empaquetados.

Instrucción	**Descripción**
`movaps`	Mueve cuatro flotantes simple precisión alineados entre registros XMM o memoria, asumiendo que la dirección de memoria está alineada a 16 bytes ("aps" significa *Aligned, Packed, Single-precision*).
`movapd`	Similar a la anterior pero mueve dos flotantes dobles precisión alineados entre registros XMM o memoria.
`movups`	Mueve cuatro flotantes simple precisión alineados entre registros XMM o memoria, sin requerir alineación de 16 bytes en la dirección de memoria ("ups" significa *Unaligned, Packed, Single-precision*).
`movupd`	Similar a la anterior pero mueve dos flotantes dobles precisión alineados entre registros XMM o memoria.
`addps`	Suma flotantes simple precisión empaquetados.
`divps`	Divide flotantes simple precisión empaquetados.
`divss`	Divide flotantes simple precisión escalares.
`mulps`	Multiplica flotantes simple precisión empaquetados.
`subps`	Resta flotantes simple precisión empaquetados.
`cmpps`	Compara flotantes simple precisión empaquetados.
`andnps`	Realiza la operación AND NOT bit a bit de flotantes simple precisión empaquetados.
`andps`	Realiza la operación AND bit a bit de flotantes simple precisión empaquetados.
`orps`	Realiza la operación OR bit a bit de flotantes simple precisión empaquetados.
`xorps`	Realiza la operación XOR bit a bit de flotantes simple precisión empaquetados.




```
.data
.align 16
vec1:    .float 4.0, 2.0, 1.0, 5.0
vec2:    .float 2.0, 2.5, 1.5, 4.0
fmt:     .asciz "%f  %f  %f  %f\n"        # formato para printf
```

```
.bss
result:  .space 16, 0                    # 4 floats = 16 bytes

.text
.global main
main:
    pushq   %rbp
    movq    %rsp, %rbp

    # Cargar los vectores
    movaps vec1, %xmm0
    movaps vec2, %xmm1

    # Sumar elemento a elemento
    addps   %xmm0, %xmm1

    # Guardar resultado en memoria
    movups %xmm1, result

    # Preparar argumentos para printf
    leaq    fmt, %rdi              # 1er argumento: dirección del formato
    movss   result, %xmm0          # cargar result[0]
    cvtss2sd %xmm0, %xmm0          # convertir a doble precisión
    movss   result+4, %xmm1        # result[1]
    cvtss2sd %xmm1, %xmm1          # convertir a doble precisión
    movss   result+8, %xmm2        # result[2]
    cvtss2sd %xmm2, %xmm2          # convertir a doble precisión
    movss   result+12, %xmm3       # result[3]
    cvtss2sd %xmm3, %xmm3          # convertir a doble precisión
    movl    $4, %eax               # número de argumentos de punto flotante
    call    printf

    xorl    %eax, %eax
    leave
    ret
```

10.2. Instrucciones para operaciones con enteros empaquetados

Por otra parte, existen instrucciones que forman parte del conjunto de extensiones SIMD y están diseñadas para realizar operaciones eficientes sobre **enteros empaquetados**. Estas instrucciones permiten efectuar comparaciones, movimientos y multiplicaciones en paralelo sobre múltiples datos en registros vectoriales, lo que optimiza el rendimiento en aplicaciones que requieren procesamiento intensivo de datos.

En particular, las instrucciones `pmov` facilitan movimientos con extensión de signo o ceros, mientras que la familia de instrucciones `padd` y `pmul` ofrece distintas variantes de suma y multiplicación, algunas con almacenamiento del resultado alto o bajo. Las

instrucciones `pmax` y `pmin` permiten determinar el valor máximo o mínimo entre elementos de enteros empaquetados, con o sin signo. Estas operaciones son fundamentales en aplicaciones de procesamiento de imágenes, criptografía y cálculos científicos de alto rendimiento.

Ejemplo

Veamos un ejemplo de instrucciones *packed*:

```
.data
.align 16
a: .float 1.0, 2.0, 3.0, 4.0
b: .float 1.0, 2.0, 3.0, 4.0
c: .byte 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
d: .byte 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16

.text
.global main
main:
    movq $a, %rdi               # rdi apunta a "a"
    movq $b, %rsi               # rsi apunta a "b"
    movaps (%rdi), %xmm0        # copia 4 floats en xmm0
    movaps (%rsi), %xmm1        # copia 4 floats en xmm1
    addps %xmm0, %xmm1          # suma los dos vectores
    movaps %xmm1, (%rdi)        # guarda el resultado en "a"

    movq $c, %rdi               # rdi apunta a "c"
    movq $d, %rsi               # rsi apunta a "d"
    movaps (%rdi), %xmm0        # copia 16 bytes en xmm0
    movaps (%rsi), %xmm1        # copia 16 bytes en xmm1
    paddb %xmm0, %xmm1          # suma los dos vectores
    movaps %xmm1, (%rsi)        # guarda el resultado en "d"

    xorl %eax, %eax
    ret
```

La instrucción relevante en este caso es `addps`, que suma simultáneamente cuatro valores en punto flotante de precisión simple. Es importante notar que, para utilizar `movaps`, los datos deben estar alineados a 16 bytes, lo cual se puede garantizar mediante la directiva `.align`. Por otro lado, la instrucción `paddb` permite sumar en paralelo 16 bytes empaquetados en registros `xmm`.

10.3. Otras extensiones

Por otra parte, en 2011 se introdujo una nueva extensión de instrucciones SIMD denominada AVX, que incorpora registros de 256 bits (`ymm0-ymm15`). Algunos procesadores implementan además AVX-512, que agrega los registros `zmm0-zmm31` de 512 bits, ampliando la cantidad total de registros vectoriales a 32. Estas extensiones no se abordan en este apunte.

Es importante notar que cada registro `ymm` amplía a su correspondiente `xmm`, pasando de 128 a 256 bits, mientras que los registros `zmm` amplían nuevamente estos registros hasta 512 bits. De este modo, los registros `ymm` y `zmm` conservan la compatibilidad con las instrucciones que operan sobre los `xmm`, permitiendo ejecutar operaciones vectoriales de mayor ancho con un mismo esquema de registros.

Apéndice A: Compilando código ensamblador con GCC

Como vimos, es necesario traducir el código desde un lenguaje de alto nivel al lenguaje de máquina que el procesador puede ejecutar. Para ello, se utiliza un software específico denominado **compilador**.

Por ejemplo, el compilador GCC realiza esta traducción en varias etapas. Primero, convierte el código fuente a un código intermedio en lenguaje ensamblador (Assembler), una representación textual del lenguaje de máquina que describe instrucciones individuales de manera más legible.

Luego, el **ensamblador** toma este código ensamblador y lo convierte en código de máquina puro, es decir, en instrucciones binarias que pueden almacenarse en memoria.

Finalmente, el **enlazador** (o linker) se encarga de combinar este código con otros módulos o bibliotecas necesarias, resolviendo las referencias entre ellos y generando un archivo ejecutable que puede ser cargado y ejecutado por el sistema operativo.

La Figura 15 ilustra este proceso completo. Dado un archivo que contiene el código fuente `hola.c`, el comando `gcc -S hola.c -o hola.s` traduce el código C a código ensamblador, almacenándolo en el archivo `hola.s`. Este comando no genera todavía código máquina ni un archivo ejecutable. El resultado puede visualizarse mediante el comando `cat hola.s`.

Luego, el comando `gcc -c hola.s -o hola.o` traduce el código ensamblador de `hola.s` a un archivo objeto binario (`hola.o`). Este archivo contiene código máquina, datos, símbolos e información de relocación, pero todavía no constituye un ejecutable y algunas referencias a símbolos externos aún deben resolverse durante el enlazado.

Finalmente, el comando `gcc hola.o -o hola` enlaza el archivo objeto con los objetos y bibliotecas necesarios —por ejemplo, la biblioteca estándar de C (`libc`), utilizada por la función `printf`— y genera el archivo ejecutable `hola`, que puede ejecutarse mediante `./hola`.

Es importante destacar que, en la práctica, este proceso suele realizarse en un solo paso. En el ejemplo mostrado, basta con ejecutar el comando `gcc -o hola hola.c`.

Por otra parte, un programador puede escribir un programa completamente en lenguaje ensamblador. Uno de los requisitos para ello es definir una etiqueta global llamada `main` dentro del segmento de código. Sin embargo, escribir todo el programa en ensamblador no suele ser la mejor opción. Es preferible limitar su uso a aquellas partes que realmente lo requieran, por ejemplo, para optimizar secciones críticas o acceder directamente al hardware. Por esta razón, es común mezclar código en C con ensamblador, siempre que este último respete la convención de llamada presentada en la Sección 8.

```
// Este archivo es main.c
#include<stdio.h>
double suma(double a, double b);
int main(){
  printf("La suma es: %f\n", suma(12, 3.14));
```

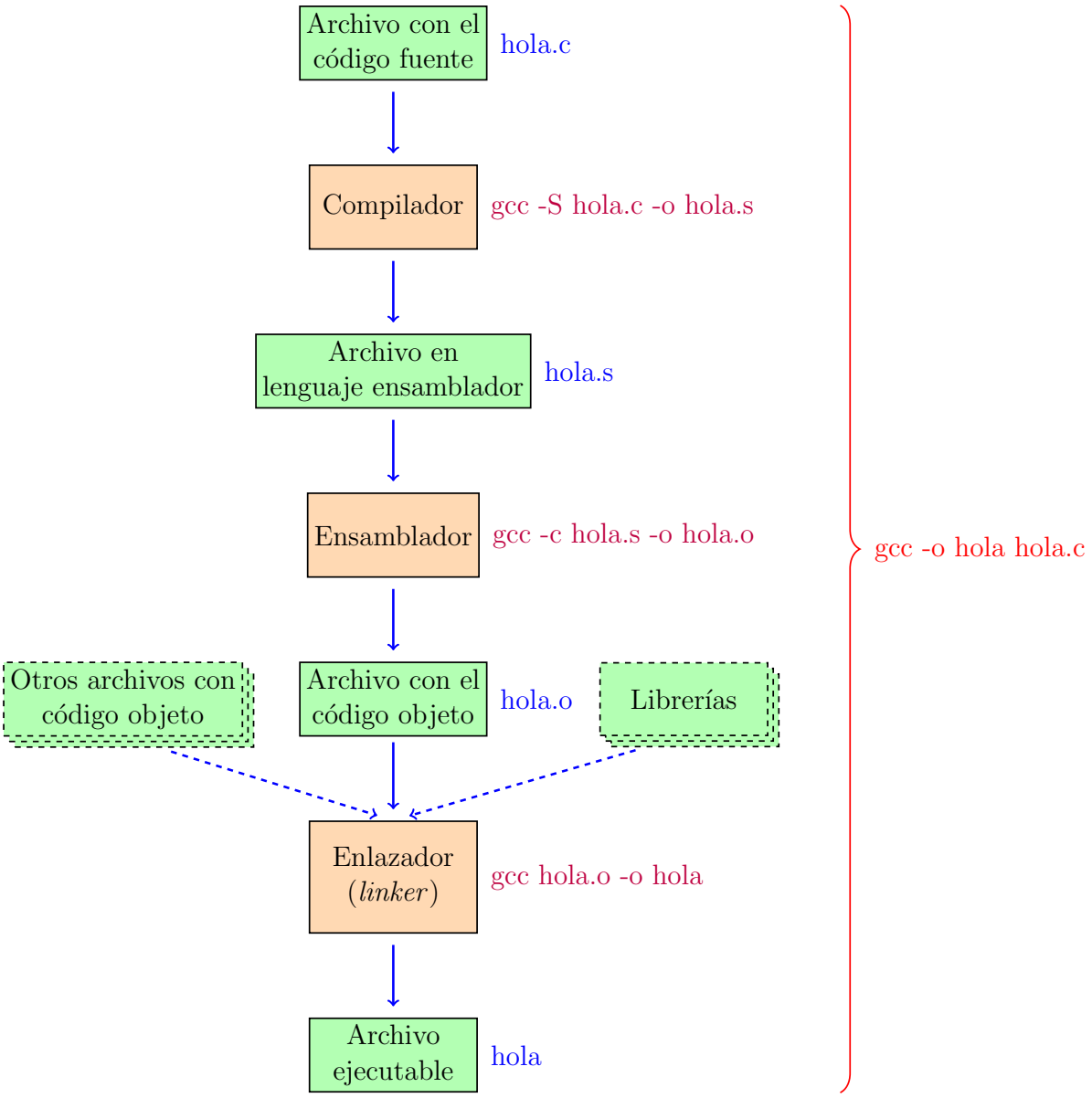


Figura 15: Etapas de transformación de un programa en C desde el código fuente hasta el archivo ejecutable.

```
  return 0;
}
```

donde la implementación de `suma` en ensamblador sería

```
# Este archivo es suma.s
.global suma
suma:
   addsd %xmm1, %xmm0   # por convención de llamada el primer argumento
      viene en xmm0 y el segundo en xmm1. El valor de retorno ya está en
       xmm0
   ret
```

Luego podemos compilar todo junto:

```
gcc -o main main.c suma.s
```

Luego podemos ejecutar mediante:

```
./main
```

y veremos impreso por pantalla el resultado:

```
15.140000
```

El enlazador se encargará de que la llamada a `suma` se corresponda con su implementación en ensamblador.

Observación

GCC permite compilar cada archivo fuente de manera independiente y luego enlazar los archivos objeto para generar el programa ejecutable:

```
gcc -c main.c -o main.o
gcc -c funciones.c -o funciones.o
gcc main.o funciones.o -o programa
```

Este procedimiento resulta especialmente útil en proyectos grandes, ya que permite recompilar únicamente los archivos fuente que hayan sido modificados, evitando tener que compilar todo el proyecto cada vez que se realiza un cambio.

Apéndice B: Depurando el código con GDB

GDB o GNU Debugger es el depurador estándar para el compilador GNU. Se puede utilizar tanto para programas escritos en lenguajes de alto nivel como C y C ++ como para programas de código ensamblador.

Continuando con el ejemplo anterior, compilamos de la siguiente manera agregando la opción `-g` para incluir información en el archivo objeto para relacionarlo con el archivo fuente:

```
gcc -g -o main main.c suma.s
```

Luego podemos iniciar la sesión de depuración con GDB:

```
gdb ./main
```

Una vez iniciada la sesión, tenemos comandos para ejecutar el código línea por línea, de a tramos, visualizar contenido de memoria, registros, etc.

Para ver una guía detallada de los comandos consultar los documentos [12] y [13]. Ambos se encuentran en la Sección Apuntes varios del Campus Virtual de la asignatura. También hay disponible un vídeo tutorial en la Sección Ejemplos.

Apéndice C: Otras opciones útiles para compilar

Al momento de compilar se pueden usar diferentes banderas. A continuación, presentaremos algunas de las más útiles:

- `-S`

  Esta bandera permite obtener el código ensamblador

  Ejemplo

  ```
  gcc -S -o hola hola.c
  ```

  Se obtiene un archivo `hola.s` con el código equivalente en ensamblador.

- `-g`

  Incluye información de depuración para herramientas como gdb.

- `-O0, -Og, -O1, -O2, -O3, -Ofast`

  Controlan el nivel de optimización del compilador. La opción por defecto es `-O0`.

Opción	Optimización	Apta para depuración	Uso típico
`-O0`	Ninguna	Sí	Desarrollo inicial, pruebas simples.
`-Og`	Básica	Sí	Depuración con algo de rendimiento.
`-O1`	Ligera	No siempre	Código final con bajo impacto.
`-O2`	Moderada	No	Código de producción estándar.
`-O3`	Alta	No	Código muy intensivo en CPU.
`-Ofast`	Máxima (sin estándares)	No	Cálculo numérico rápido, sin portabilidad garantizada.

- `-fverbose-asm`

  Se utiliza para generar un archivo de ensamblador que contiene comentarios adicionales explicativos.

- `-fomit-frame-pointer`

  Elimina el uso del puntero base (*frame pointer*) para todas las funciones. Puede ser útil en funciones pequeñas, pero puede dificultar la depuración de funciones complejas.

- `-pie`

  Compila el binario como Position Independent Executable.

- `-no-pie`

  Se usa para desactivar la generación de ejecutables como PIE (Position Independent Executable). Es decir, le indica al compilador que genere un binario con direcciones fijas, en lugar de uno que se cargue en una dirección aleatoria cada vez que se ejecuta. En general, lo utilizaremos siempre que exista un segmento `.data` para simplificar el debugging.

- `-z noexecstack`

  Se utiliza para marcar la pila como no ejecutable, lo que mejora la seguridad del programa al prevenir la ejecución de código en la pila.

- `-Wall`

  Sirve para activar un conjunto amplio de advertencias (warnings) que ayudan a detectar posibles errores o malas prácticas en el código fuente.

Referencias

[1] Andrew S. Tanembaum, *Organización de computadoras: Un enfoque estructurado*, cuarta edición, Pearson Education, 2000

[2] Paul A. Carter, *PC Assembly Language*, Disponible en formato electrónico: http://www.drpaulcarter.com/pcasm/, 2006.

[3] M. Morris Mano, *Computer system architecture*, tercera edición, Prentice-Hall, 1993.

[4] Randall Hyde, *The art of assembly language*, segunda edición, No Starch Pr, 2003.

[5] Randal E. Bryant - David R. O'Hallaron, *X86-64 Machine-Level Programming*, 2005.

[6] Bryant, Randal E, David Richard, O'Hallaron y David Richard, O'Hallaron, *Computer systems: A programmer's perspective*, Prentice Hall, 2003.

[7] *AMD64 Architecture Programmer's Manual Volume 1: Application Programming*, AMD64 Technology, 2015.

[8] *AMD64 Architecture Programmer's Manual Volume 4: 128-Bit and 256-Bit Media Instructions*, AMD64 Technology, 2015.

[9] *X86 Assembly Language Reference Manual*, Oracle, 2012.

[10] Miquel Albert Orenga y Gerard Enrique Manonellas, *Programación en ensamblador (x86-64)*, Universitat Oberta de Catalunya (UOC), 2011.

[11] M. Matz, J. Hubicka, A. Jaeger, M. Mitchell, *System V Application Binary Interface: AMD64 Architecture Processor Supplement*, Draft Version 0.99.7, 2014.

[12] *Debugging Assembly Code with GDB.*

[13] *GDB Tutorial, A Walkthrough with Examples*, 2009.

[14] *Intel 64 and IA-32 Arquitectures Software Developer's Manual, Volume 2*, Intel, diciembre 2021.

[15] Richard Blum, *Professional Assembly Language*, Wiley Publishing, Inc., 2005.

[16] Ray Seyfarth, *Introduction to 64 Bit Intel Assembly Language Programming*, 2011.